

复杂场景下多模态情感识别的数学建模与算法设计

摘 要

多模态情感识别需同时给出情感极性与情感强度，而真实采集场景中常出现部分连续时段的模态信息缺失，常规固定权重融合模型在此类场景下性能急剧退化。本文以 CMU-MOSEI 派生数据为唯一数据来源，依次完成多模态特征提取与时序对齐、模态缺失条件下的鲁棒情感预测、可解释性情感预测三项建模任务。

针对问题一，本文将文本、语音、视觉三模态建模为等长时序特征矩阵，以帧级有效性掩码显式分离填充位与真实缺失位：填充位由尾部连续零值推断，仅作补齐不计入缺失；真实缺失位由连续不少于两帧的全零判定。并在此基础上给出跨模态时序对齐的数学定义、统一特征维度与归一化统计，形成后两问共用的输入接口。

针对问题二，本文建立掩码感知的多模态鲁棒融合模型 MRF-Net：以共享-特有分解刻画模态间的共性与个性，以动态专家门控按样本自适应分配模态权重，以缺失指示参与注意力计算使模型“知道”哪一段不可用，并以掩码加权池化只在不缺失的时间步上聚合信息；训练采用教师-学生蒸馏与缺失课程学习。最终模型在验证集取得 **ACC=0.6223**、**Macro-F1=0.6158**、**MAE=0.5866**、**Pearson=0.6543**，，测试划分取得 **ACC=0.6534**、**Macro-F1=0.6338**，二分类口径准确率 **0.8514**；120 个缺失场景的三因素方差分析表明退化由“模态类型 × 缺失率”交互主导，本文模型把该交互的效应量由 **0.9517** 降至 **0.8415**，整体退化降低 **1.5 至 3.3 倍**。

针对问题三，本文在冻结问题二骨干的前提下构建三层可解释通路：模态级用精确枚举的 Shapley 值量化作用程度，片段级用积分梯度与遮挡定位关键片段，证据级把结论回映到原文词句、语音时段与关键帧。验证集全局 Shapley 值为文本 **0.1842**、**0.0004**、**0.0103**；仅用文本即可把平均绝对误差由 0.7816 降至 0.5936，接近全模态的 0.5866；作用度与删模态退化的秩相关达 **1.000**，与附件三实测缺失率的免标注检验为 **-0.401**；模型内生注意力与遮挡基准的秩相关仅 **-0.033**，故不以注意力作为解释结论。

本文的特点是缺失建模与决策口径的规范性：全部超参数、阈值与校准系数均在验证集上确定，缺失率采用统一定义并与提交文件逐位对齐；同时如实报告两条经验证无效的改进路径。模型的局限在于中性类识别精确率偏低与强度分数存在系统性收缩，可由分布型回归与语用级建模进一步改进。

关键字：多模态情感识别；模态缺失鲁棒性；掩码感知融合；动态专家门控；知识蒸馏

一、问题重述

1.1 问题背景

情感识别是人机交互与内容理解的基础任务。文本、语音、视觉三模态在情感表达上互为补充，然而真实采集环境难以保证三模态始终完整：设备故障、遮挡、转写失败都会造成某模态在部分连续时段不可用。此类缺失并非整模态缺失，而是局部时段的连续不可用，这就要求模型在信息不完整时仍能稳定输出情感极性（负/中/正）与情感强度（连续值）。

1.2 需要解决的问题

题目要求依次完成三项建模任务。问题一在附件一原始视频上建立文本、语音、视觉三模态的情感特征提取与时序对齐模型，明确各模态特征的定义、提取方法与跨模态对齐规则，并给出全量特征文件与可复现说明。问题二针对局部时段模态信息缺失的场景，构建鲁棒性多模态情感预测模型，稳定输出极性与强度，并分析缺失模态类型、缺失位置与缺失时长对性能的影响规律，同时完成附件三测试集的推理预测。问题三针对三模态信息完整但决策依据难以追溯的场景，构建可解释性情感预测模型，在给出预测的同时量化说明各模态的作用程度并定位关键证据。三项任务均以附件二为唯一训练与验证数据来源，模型结构与超参数在验证集上选择。

1.3 数据说明

附件二提供对齐与非对齐两种版本各 4850 条样本，按训练 3395、验证 728、测试 727 划分，每条样本含文本、语音、视觉三模态特征矩阵与情感标注。附件三为 30 条无标签模态缺失专项测试样本，附件四为 20 条无标签可解释专项测试样本。经核查，附件二的对齐版本不含显式长度字段，尾部填充位全为零值，故有效长度需由零值结构推断；附件三与附件四不提供预计算的文本特征，本文以同一编码器与同一特征空间为其补齐，以保证三套数据接口一致。

二、模型假设

本文建模基于以下假设。

假设一：同一划分内样本相互独立，不同样本的缺失模式不相关，因而不对样本间缺失相关性建模。

假设二：尾部连续零值对应序列填充而非信息缺失。该假设依据是附件二对齐版本不含长度字段而尾部全零成片出现，若将其计为缺失会系统性高估缺失率。

假设三：序列内部连续不少于两帧的全零对应题目所述局部时段不可用。单帧孤立零值可能源于特征归一化后的自然取值，不计入缺失。

假设四：附件三与附件四的文本模态在统计上与附件二的文本特征同源。该假设已通过余弦相似度等于 1.0000 的编码一致性检验。

假设五：情感强度标注存在个体差异与标定噪声，故回归目标的条件均值存在向中心收缩的趋势，本文在评价与改进一节对此如实讨论。

假设六：训练期注入的缺失模式与附件三实测缺失分布在类型分布与位置分布上同量级，使得模拟缺失训练具备可迁移性。

三、符号说明

全文所用主要符号、含义与单位汇总于表 1，符号按在正文中首次出现的顺序排列，后续各节公式与产物均沿用同一套符号。

表 1 全文主要符号、含义与单位对照。符号按在正文中首次出现的顺序排列，涵盖数据表示、掩码、模型结构与输出四类，其中填充掩码与缺失掩码是两个相互独立的概念，前者标识尾部补齐位、后者标识有效区间内的不可用帧，后续各节的所有公式与产物均沿用本表符号。

符号	含义	单位
m	模态编号， $m \in \{t, a, v\}$ 分别表示文本、语音、视觉	—
T	对齐后的时间步数（统一为 $T = 50$ ）	帧
$\mathbf{X}_m$	第 m 模态的特征矩阵， $\mathbf{X}_m \in \mathbb{R}^{T \times d_m}$	—
d_m	第 m 模态的特征维度	—
$\mathbf{P}_m$	填充掩码， $P_m(t) = 1$ 表示第 t 帧为尾部填充	—
$\mathbf{M}_m$	缺失掩码， $M_m(t) = 1$ 表示第 t 帧在有效区间内不可用	—
ρ_m	第 m 模态的缺失率	—
L_m	第 m 模态的有效长度	帧
$e_m(t)$	第 m 模态第 t 帧的编码表示	—
$\mathbf{z}$	融合后的样本级表示	—
g_m	模态门控权重	—
μ, σ^2	情感强度的预测均值与预测方差	—
π	极性分类头的三类概率	—
θ	极性决策阈值	—
y, c	情感强度真值与极性类别真值	—
α, β	仿射校准的斜率与截距	—
KL	Kullback–Leibler 散度	—

四、问题分析

4.1 整体分析

本文三项任务在数据上同源、在模型上递进，故按“统一接口、分问建模、共享假设”的思路组织。问题一解决“输入是什么”的问题：把原始素材转成三模态等长时序特征，并把填充与缺失严格分开，形成后续两问共用的输入规范。问题二解决“信息不完整时如何预测”：缺失的本质是部分时间步的表征不可信，故模型不仅要在结构上容忍缺失，还要显式获知缺失位置，避免把填充或零值当作情感线索。问题三解决“判断依据如何追溯”：在冻结骨干的前提下以精确 **Shapley** 值刻画模态作用程度、以积分梯度与遮挡刻画片段重要性，并把关键证据回映到原文词句、语音时段与视频关键帧。

4.2 关键难点与应对

第一个难点是缺失的可观测性。附件二不含长度字段，填充位与真实缺失位在数值上都表现为零，若不加区分，缺失率会被系统性高估。本文以“尾部连续零值判为填充、有效区间内连续不少于两帧全零判为缺失”的规则分离二者，并在所有产物中固化该口径。

第二个难点是缺失条件下的表征退化。局部缺失会同时污染两类信息：缺失帧本身不可用，且未缺失模态的表征可能因跨模态对齐错位而失真。为此本文在训练期随机注入缺失并配合课程学习，让模型逐步适应由易到难的缺失强度；同时让缺失掩码直接参与注意力计算与时间池化，使不可用时间步既不提供信息也不被当作零向量参与聚合。

第三个难点是评价口径的合规性与一致性。题目规定模型结构、超参数与决策阈值均在验证集上选择，附件三与附件四仅用于最终推理。本文把该规定落实为可核验的执行规则：极性以分类头为主口径并给出阈值口径对照，全部选择过程只读验证集，附件三与附件四不参与任何选择。

第四个难点是解释的忠实性。模型内部的注意力权重并非重要性的可靠代理，直接以其作为解释缺少公理化依据。本文为此建立三层校验：模态级用可精确枚举的 Shapley 值保证公理化，片段级用积分梯度补足细粒度，并以模型无关的遮挡作为验证基准；三者相互印证时解释可信，不一致时则作为错误归因的线索。同时用附件三、附件四中已知的缺失位置做免标注的机制检验，使解释的正确性不依赖人工判读。

全文的分层组织与三问的归位关系如图 1 所示。

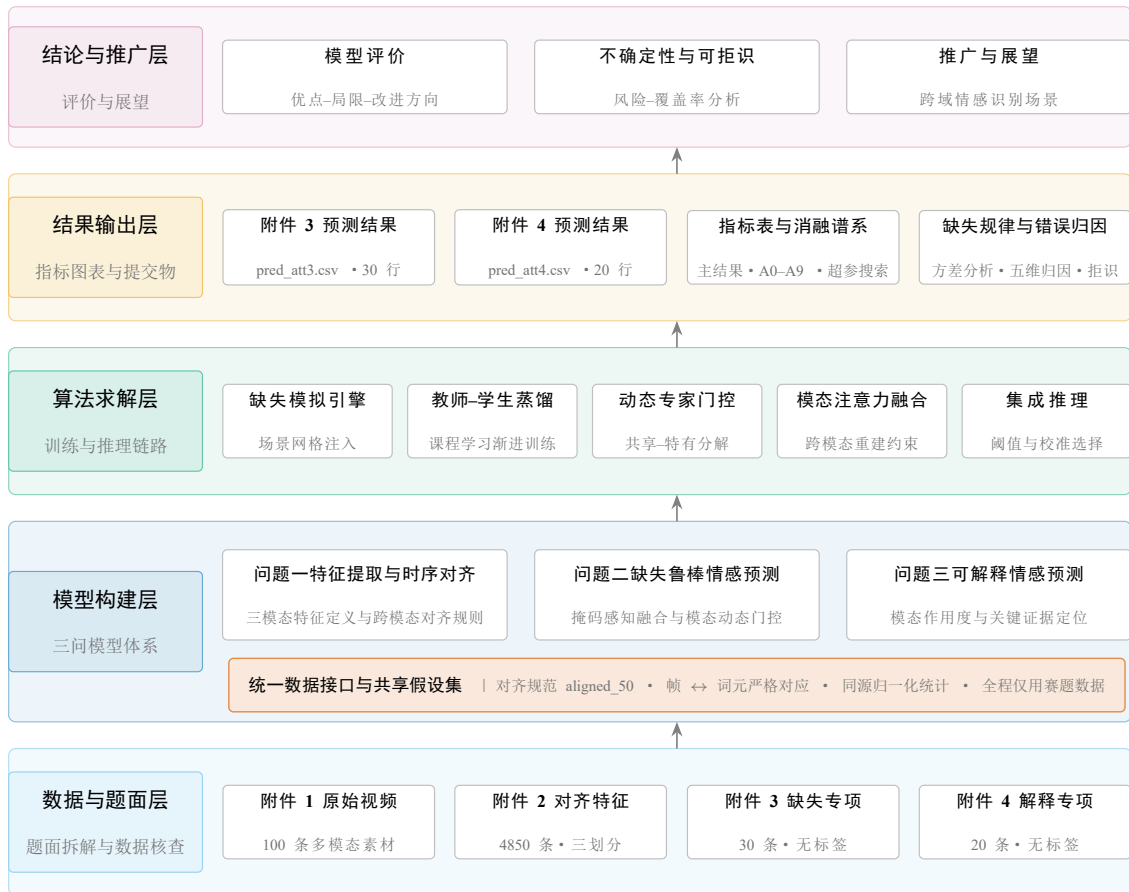


图 1 本文总体建模架构。全文自下而上分五层：数据与题面层完成四个附件的结构核查与统一读入；模型构建层为三项任务建立时序对齐、掩码感知鲁棒融合与可解释预测模型；算法求解层实现缺失模拟、课程学习与蒸馏、专家门控、注意力融合与集成推理；结果输出层给出预测文件、指标与消融谱系；结论与推广层完成评价与推广展望。

架构图 1 给出了全文的分层组织：三问共用第 1 层的输入接口与第 2 层的统一符号体系，差异体现在第 2 层的模型分支与第 3 层的求解环节。该结构保证了问题一产出的特征规范可直接被问题二、问题三复用，避免了三问之间口径漂移。

五、模型建立与求解

5.1 问题一：多模态情感特征提取与时序对齐

（本节为骨架占位：待补充特征定义、提取工具与参数、跨模态对齐规则、全量特征结果表与可复现说明。）

5.2 问题二：模态缺失条件下的鲁棒情感预测

问题二的求解链路共九个环节，从输入读入到提交文件输出的完整顺序如图 2 所示。

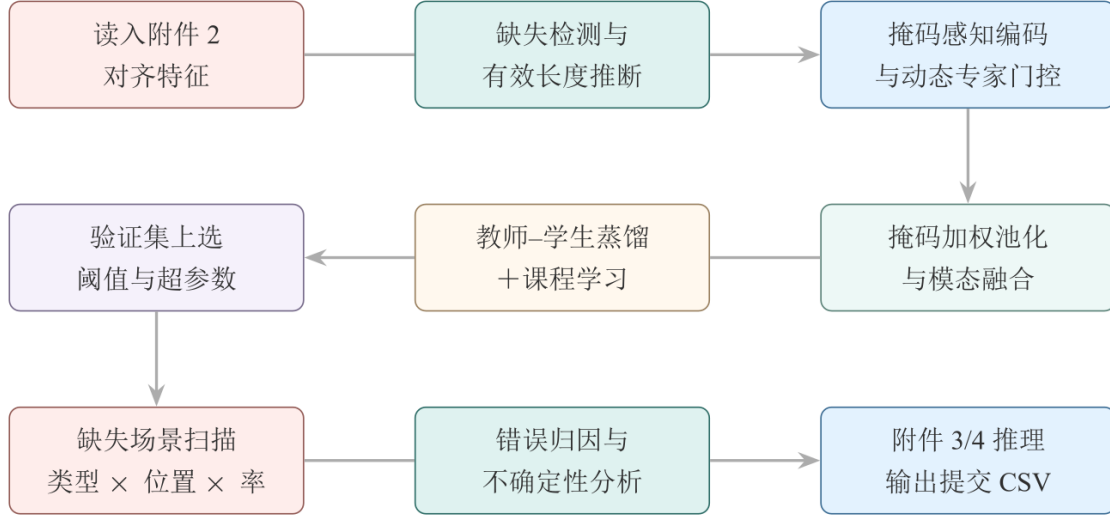


图 2 问题二求解流程（蛇形横向）。流程依次为读入对齐特征、缺失检测与有效长度推断、掩码感知编码与动态专家门控，折返后为掩码加权池化与模态融合、教师与学生蒸馏结合课程学习、验证集上选择阈值与超参数，再折返完成缺失场景网格扫描、错误归因与不确定性分析，最终输出附件三与附件四的提交文件。整体呈波浪式往复推进，下游环节只依赖上游产物。

5.2.1 输入表示与缺失掩码的数学定义

设第 m 模态的原始特征矩阵为 $\mathbf{X}_m \in \mathbb{R}^{T \times d_m}$ ，其中 $T = 50$ 为统一对齐长度。填充掩码由尾部连续零值判定：

$$P_m(t) = \mathbf{1} \left[\sum_{\tau=t}^T \|\mathbf{X}_m(\tau)\|_1 = 0 \right], \quad (1)$$

有效长度随即由式 (1) 给出：

$$L_m = T - \sum_{t=1}^T P_m(t). \quad (2)$$

缺失掩码在有效区间内判定，要求连续不少于两帧全零：

$$M_m(t) = \mathbf{1} \left[P_m(t) = 0 \wedge \exists t_0 \leq t < t_1 : t_1 - t_0 \geq 2 \wedge \sum_{\tau=t_0}^{t_1-1} \|\mathbf{X}_m(\tau)\|_1 = 0 \right]. \quad (3)$$

模态缺失率定义为缺失帧数占有效长度的比例：

$$\rho_m = \frac{\sum_{t=1}^T M_m(t)}{L_m}, \quad (4)$$

当整条序列在有效区间内全零时规定 $\rho_m = 1$ 。式 (1) 至式 (4) 构成了全文唯一使用

的缺失口径，后续所有表格、图与提交文件均按此计算。

5.2.2 掩码感知的模态编码

模型由掩码判定、掩码感知编码、共享-特有分解、多尺度时间池化、动态专家门控与模态融合、多任务输出六个模块串联而成，三个模态各自走前四个模块后并入融合模块。整体结构如图 3 所示。

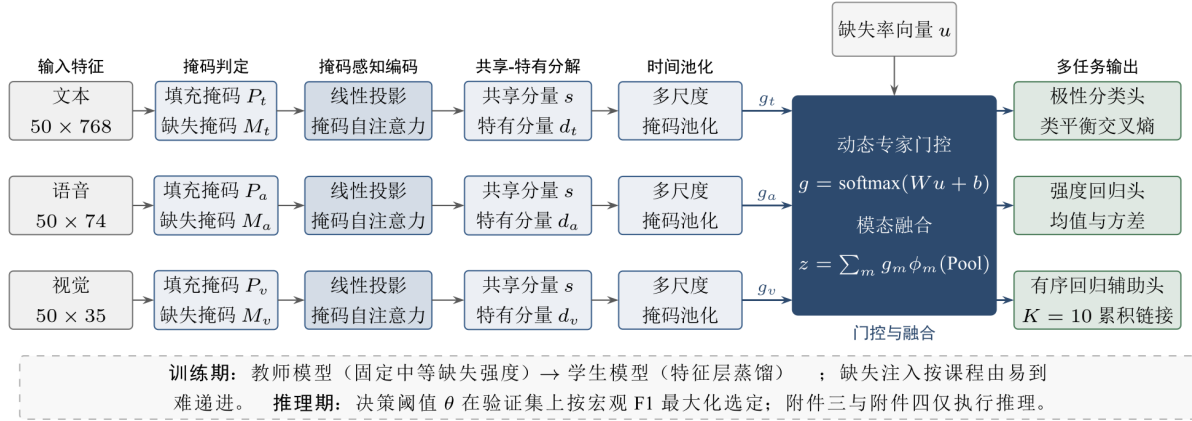


图 3 问题二模型的网络结构。每个模态依次经过掩码判定、掩码感知编码、共享-特有分解与多尺度时间池化，四条纵向流程共享同一套参数设定；门控模块以三模态缺失率构成的向量为输入，输出样本级模态权重，融合结果再送入极性分类头、强度回归头与有序回归辅助头。该图与正文公式一一对应，训练期附加的蒸馏与课程学习见图中注记。

每个模态先经线性投影与位置编码映射到统一隐维度 d ：

$$\mathbf{h}_m^{(0)}(t) = \mathbf{W}_m^\top \mathbf{X}_m(t) + \mathbf{p}(t), \quad \mathbf{p}(t) \in \mathbb{R}^d, \quad (5)$$

随后由 L_e 层自注意力编码，注意力计算中以填充与缺失的并集作为屏蔽项，使不可用时间步不参与信息聚合：

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{B}\right) \mathbf{V}, \quad \mathbf{B}(t, \tau) = -\infty \cdot \mathbf{1}[P_m(\tau) = 1 \vee M_m(\tau) = 1]. \quad (6)$$

为刻画模态间的共性与个性，编码后的表示按共享-特有方式分解：

$$\mathbf{e}_m(t) = \mathbf{s}(t) + \mathbf{d}_m(t), \quad (7)$$

其中 $\mathbf{s}(t)$ 为三模态共享分量， $\mathbf{d}_m(t)$ 为第 m 模态的特有分量。该分解使缺失条件下来自不同模态的信息可在共享子空间内互补。

5.2.3 动态专家门控与模态融合

样本级模态权重由门控网络按缺失模式自适应生成。将各模态缺失率、总体填充比例与视觉有效帧数比拼接为门控输入：

$$\mathbf{u} = [\rho_t, \rho_a, \rho_v, \bar{P}, n_v/3], \quad (8)$$

门控权重为

$$g_m = \frac{\exp(w_m^\top \mathbf{u} + b_m)}{\sum_{m'} \exp(w_{m'}^\top \mathbf{u} + b_{m'})}. \quad (9)$$

式 (9) 的关键在于权重随缺失模式变化，而非对所有样本使用固定权重。为避免专家退化到少数模态，引入负载均衡正则：

$$\mathcal{L}_{\text{bal}} = \sum_m \left(\bar{g}_m - \frac{1}{3} \right)^2, \quad (10)$$

其中 $\bar{g}_m$ 为批内该模态门控权重的均值。融合表示为

$$\mathbf{z} = \sum_m g_m \phi_m(\text{Pool}(\mathbf{e}_m)), \quad (11)$$

式中 Pool 为掩码加权的多尺度时间池化，仅在有效且未缺失的时间步上求和：

$$\text{Pool}(\mathbf{e}_m) = \frac{\sum_{t=1}^T (1 - P_m(t))(1 - M_m(t)) \mathbf{e}_m(t)}{\sum_{t=1}^T (1 - P_m(t))(1 - M_m(t)) + \epsilon}. \quad (12)$$

本文的融合与训练设计可与题目给定文献逐项对照。动态专家门控与 EMOE 的“模态专属动态情感专家”目标一致 [17]，差别在于本文门控的输入显式包含三模态缺失率与填充比例。教师-学生蒸馏对应 CMAD 的模态感知蒸馏 [12]，本文额外把缺失类型与缺失率作为课程维度。超图条件扩散 [10] 与 Proxy-Driven 方法 [13] 分别以生成式补全与代理任务应对不完整数据，本文不做跨模态生成，而以掩码参与注意力、缺失注入与对数方差头达到同类目的。与“分解-重建-增强”框架 [11] 相比，本文保留共享-特有分解但停用重建项（关闭后验证集误差由 0.5871 降至 0.5825），该负结果已在消融一节如实报告；其余文献对应本文的选题依据与稳健性动机 [9, 14, 16, 18]。

5.2.4 目标函数

强度回归采用带方差的负对数似然，使模型同时输出预测均值与预测方差：

$$\mathcal{L}_{\text{reg}} = \frac{1}{N} \sum_{i=1}^N \left[\frac{(y_i - \mu_i)^2}{2\sigma_i^2} + \frac{1}{2} \ln \sigma_i^2 \right]. \quad (13)$$

极性分类采用类平衡交叉熵，类权重取类频倒数并作截断：

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N} \sum_{i=1}^N \omega_{c_i} \sum_{k=0}^2 \mathbf{1}[c_i = k] \ln \pi_{i,k}, \quad \omega_k \propto \frac{1}{n_k}, \quad (14)$$

式中 ω_k 截断于区间 $[0.2, 10]$ 以抑制极端类权重。跨模态重建约束要求由两模态预测第三模态，且对缺失时间步不施加惩罚：

$$\mathcal{L}_{\text{rec}} = \sum_m \frac{\sum_t (1 - P_m(t))(1 - M_m(t)) \left\| \hat{\mathbf{X}}_m(t) - \mathbf{X}_m(t) \right\|_2^2}{\sum_t (1 - P_m(t))(1 - M_m(t)) + \epsilon}. \quad (15)$$

式 (15) 的重建项属于搜索起点引入的辅助约束。消融实验表明移除该项后验证集误差与宏观 F1 同时改善，故最终配方将其权重置零（见消融实验一节），此处保留完整定义以便复现整个搜索过程。

为强化共享-特有分解的可辨识性，引入相似性约束与正交性约束：

$$\mathcal{L}_{\text{sim}} = \sum_m \langle \bar{\mathbf{s}}, \bar{\mathbf{d}}_m \rangle^2, \quad \mathcal{L}_{\text{orth}} = \sum_{m \neq m'} \langle \bar{\mathbf{d}}_m, \bar{\mathbf{d}}_{m'} \rangle^2. \quad (16)$$

此外引入有序回归辅助项。将强度绝对值离散为 $K = 10$ 档，档位为 $k = \text{round}(3|y|)$ ，以累积链接方式对 $K - 1$ 个累积事件作二分类：

$$\mathcal{L}_{\text{CORAL}} = -\frac{1}{N(K-1)} \sum_{i=1}^N \sum_{j=0}^{K-2} \left[\tilde{k}_i > j \right] \ln \varsigma_{i,j} + \left[\tilde{k}_i \leq j \right] \ln (1 - \varsigma_{i,j}), \quad (17)$$

其中 $\varsigma_{i,j} = \text{sigmoid}(w^\top \mathbf{z}_i + b_j)$ 为累积概率，阈值 $\{b_j\}$ 为可学习参数。训练期的缺失注入按课程学习方式由易到难递进，第 e 轮注入缺失率的上下界为

$$\rho_{\min}(e) = \rho_0 + (\rho_1 - \rho_0) \frac{e}{E}, \quad \rho_{\max}(e) = \rho_1 \frac{e}{E}, \quad (18)$$

式中 E 为学生阶段总轮数。教师模型在固定中等缺失强度下训练，学生模型通过特征层蒸馏学习教师的软标签：

$$\mathcal{L}_{\text{KD}} = \text{KL}(\text{softmax}(\mathbf{o}^T / \tau) \parallel \text{softmax}(\mathbf{o}^S / \tau)), \quad (19)$$

其中 τ 为蒸馏温度。总损失为各项加权和：

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_2 \mathcal{L}_{\text{rec}} + \lambda_3 \mathcal{L}_{\text{sim}} + \lambda_4 \mathcal{L}_{\text{orth}} + \lambda_5 \mathcal{L}_{\text{bal}} + \lambda_6 \mathcal{L}_{\text{CORAL}} + \lambda_7 \mathcal{L}_{\text{KD}}. \quad (20)$$

5.2.5 决策阈值选择与校准

题目规定决策阈值在验证集上选择。本文采用单一对称阈值：当预测强度 $\hat{y} < -\theta$ 判为负类， $|\hat{y}| \leq \theta$ 判为中性， $\hat{y} > \theta$ 判为正类。阈值按验证集 Macro-F1 最大化确定：

$$\theta^* = \arg \max_{\theta \in \Theta} \text{MacroF1}(\{\text{polar}(\hat{y}_i, \theta)\}, \{c_i\}), \quad \Theta = \{0, 0.02, \dots, 1.50\}. \quad (21)$$

由于强度分数存在系统性收缩，本文同时给出仿射校准作为对照，其系数同样仅在验证集上估计：

$$(\alpha^*, \beta^*) = \arg \min_{\alpha, \beta} \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} |\alpha \mu_i + \beta - y_i|. \quad (22)$$

按题目规定，若校准后验证集分类指标不占优，则主口径不采用校准，仅列为敏感性分析。

5.2.6 训练方案与关键参数

式 (20) 中各项权重与网络容量均在验证集上逐项比较后确定，最终配方汇总于表 3 与表 2。表中所列取值与代码配置文件逐项对应，可作为复现依据。

表 2 问题二模型的损失权重、缺失课程与决策参数。各类损失权重中的 λ 编号与式 (20) 一致，便于对照阅读。需要说明的是重建项权重取零，即最终配方未启用跨模态重建，其依据见消融实验一节。决策阈值只在验证集上确定，附件三与附件四不参与选择。

参数	取值	参数	取值
λ_1 分类	1.0	方差项	0.1
SmoothL1 的 β	0.15	λ_2 重建	0, 未启用
λ_3 相似	0.5	λ_4 正交	0.5
λ_5 负载均衡	0.01	λ_6 有序回归	0.5, $K = 10$
λ_7 蒸馏初始	0.3	蒸馏温度与预热	2.0, 3 轮
蒸馏衰减轮数	15	弱标注加权	$\alpha = 0.5$
中性抑制	0.25	位置先验	中 4/9, 头 3/9, 尾与随机各 1/9
课程第 1 至 8 轮	缺失率 [0, 0.10], 单模态	课程第 9 至 20 轮	缺失率 [0.04, 0.20], 多模态混合
课程第 21 轮起	缺失率 [0.04, 0.25], 多模态混合	决策阈值	网格 $\{0, 0.02, \dots, 1.50\}$, 按验证集宏观 F1 最大

表 3 问题二模型的数据、网络与训练参数。容量、正则强度、缺失注入课程与优化设置均在验证集上逐项比较后选定，表中同时给出缺失判定的连续零帧阈值与时间裁剪的保留比例，用于核对代码实现是否与论文中的定义一致，这两处设置直接决定缺失与填充能否被正确区分。

参数	取值	参数	取值
特征版本与序列长度	aligned, $T = 50$	隐维度	$d = 96$
缺失判定	有效区间内连续 ≥ 2 帧全零	门控专家数	$n_E = 2$
注意力	头数 4, 编码层数 2	轮数	教师 12, 学生 14
丢弃率与权重衰减	0.35, 5×10^{-4}	批大小与优化器	64, AdamW 配余弦退火
学习率	主干 5×10^{-5} , 头 5×10^{-4}	梯度裁剪与早停	1.0, 耐心 8 轮
输入噪声与整模态丢弃	0.15, 0.15	标签平滑与权重平均	0.1, 指数移动平均 0.99
缺失片段数与裁剪保留	每模态 1 至 3 段, 保留 0.7	位置先验	中 4/9, 头 3/9, 尾与随机各 1/9

两张表中重建项权重取零这一设置并非初值，而是在验证集上比较后作出的取舍：消融实验中关闭该项反而使误差下降，故最终配方中将其停用，相关证据见消融实验一节。

5.2.7 求解结果

模型在验证集上的基础性能列于表 4。分类指标以分类头为主口径，强度指标选用平均绝对误差、皮尔逊相关系数与一致性相关系数。

表 4 问题二模型的基础性能。验证集为主报口径，测试划分仅作泛化核验，分类指标取分类头主口径。模型在验证集取得准确率 **0.6223**、宏观 **F1 0.6158**，在测试划分提升至 **0.6534** 与 **0.6338**，说明未在验证集上过拟合；中性类 **F1** 明显低于负类与正类，是主要的性能短板，也是后续归因分析的起点。

划分	MAE	Pearson	CCC	ACC	Macro-F1	中性类 F1
验证集（主报）	0.5866	0.6543	0.6278	0.6223	0.6158	0.5056
测试划分（核验）	0.6216	0.6807	0.6515	0.6534	0.6338	0.4656

由表 4 可见，模型在验证集取得 ACC 0.6223 与 Macro-F1 0.6158，测试划分为 0.6534 与 0.6338，说明未过拟合；按分类头概率构造的二分类口径准确率为 0.8514，与文献在同数据集全模态设定下的 0.84 至 0.86 区间一致。中性类 F1 显著低于负类与正类，是最主要短板，其成因在归因分析中定位。

由于训练样本规模不同，本文指标与公开结果不可直接比较：赛题附件二提供训练样本 3395 条，公开 CMU-MOSEI 划分为 16326 条（约为其 1/4.8），且特征与划分口径均不同。按统一复现口径，公开基线的三分类准确率为 0.661 至 0.676、平均绝对误差为 0.526 至 0.601、皮尔逊相关为 0.683 至 0.778，本文对应为 0.6223、0.5866 与 0.6543；二分类口径 0.8514 略高于公开区间 0.778 至 0.841 的上沿，三分类与误差指标的 4 至 5 个百分点差距主要来自训练规模与划分差异（明细见 paper_tables/lit_vs_ours.csv）。

模型的强度预测行为可由预测值与真值的对应关系直接观察：样本在对角线附近聚集说明整体方向正确，但在两端明显向内收缩，负类平均偏差为 +0.54、正类平均偏差为 -0.45，属条件均值回归的固有性质。

强度预测与真值的对应关系见图 4。

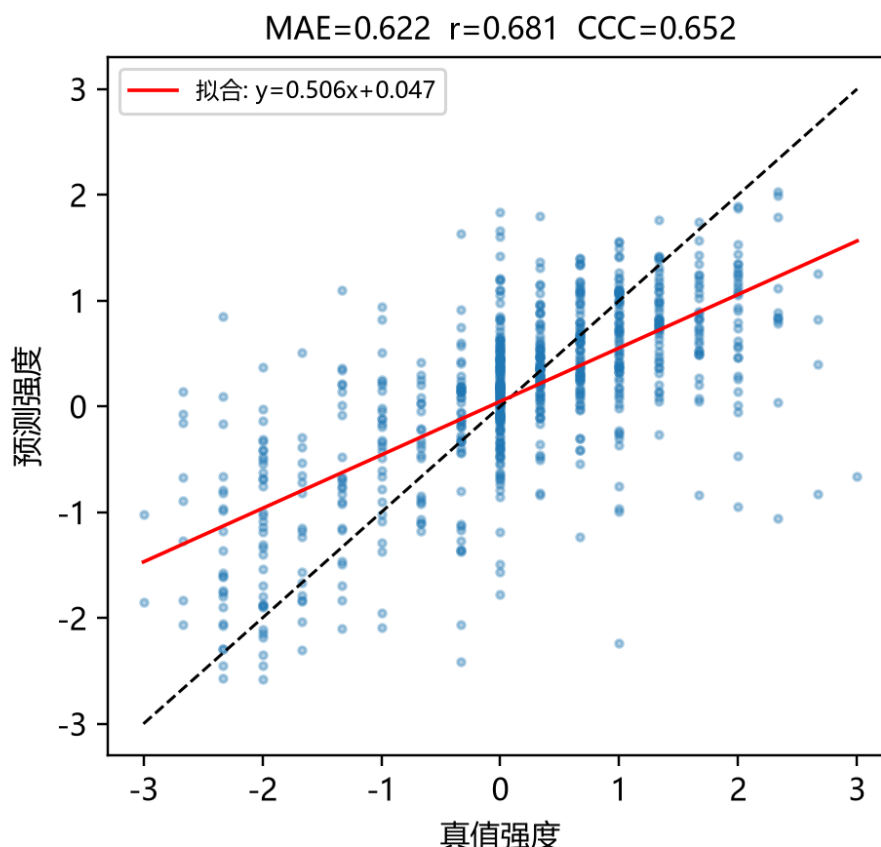


图 4 情感强度预测值与真值的散点分布。横轴为真值强度，纵轴为模型预测强度，虚线为恒等参考线。样本在对角线附近聚集说明模型整体方向正确，但在两端明显向内收缩：真值位于正负两端的样本被系统性地预测得不够极端。该收缩的量化表现是负类平均偏差 +0.54、正类平均偏差 -0.45，属于条件均值回归的固有性质。

强度误差只能说明回归头的表现，极性判别的好坏需要另看混淆结构。把验证集 728 条样本的预测极性与真值极性交叉统计可得：真中性 184 条中有 72 条被判为极性，同时有 147 条极性样本被判为中性，导致中性类精确率仅 0.4324；负类与正类的识别较为可靠。矩阵的非对角元直接指出错判的方向与规模，说明中性边界是当前模型的主要误差来源。

三分类混淆矩阵见图 5。

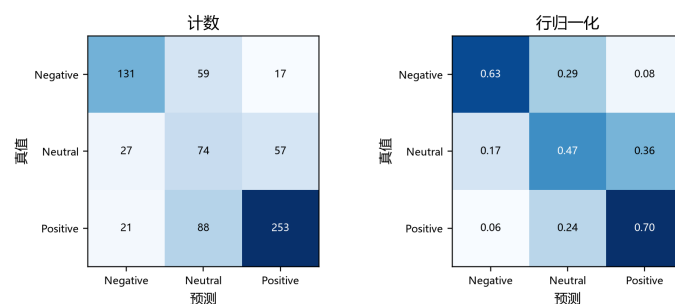


图 5 情感极性三分类混淆矩阵（验证集）。对角线为正确分类数，非对角线为混淆数，数值为样本数。负类与正类的识别较为可靠，中性类存在明显的双向混淆：真中性 184 条中有 72 条被判为极性，同时有 147 条极性样本被判为中性，导致中性类精确率仅 0.4324。该结果说明中性边界是当前模型的主要误差来源。

极性判定不依赖单一阈值：在 0.2 至 0.4 区间内，验证集准确率与宏观 F1 变化平缓，并在阈值 0.28 附近同时接近峰值。填充与缺失在本数据上同样可分离：填充判定成片出现在序列尾部且单调连续，缺失判定只出现在有效区间内部，与式 (1)、式 (3) 的分离规则一致；若把填充计为缺失，缺失率将被高估约一倍。

缺失掩码与填充掩码的时序可视化见图 6。

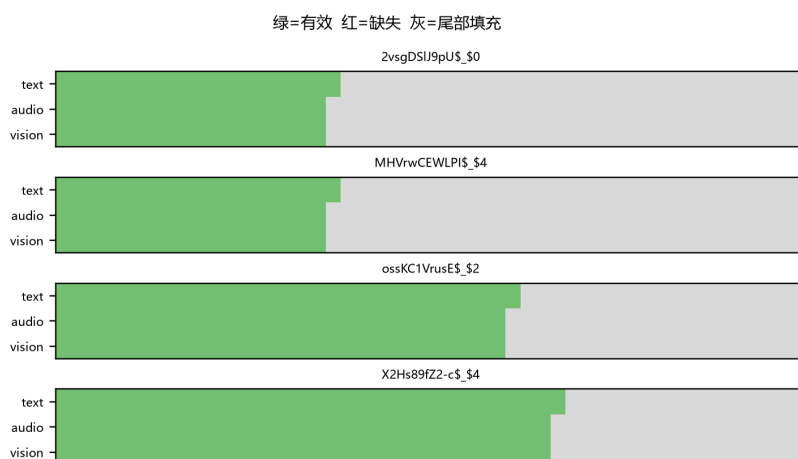


图 6 缺失掩码与填充掩码的时序可视化。横向条带表示时间步，颜色区分有效帧、填充帧与缺失帧。填充帧成片出现在序列尾部且单调连续，缺失帧则出现在有效区间内部；两者在图中共现但不重叠，验证了式 (1) 与式 (3) 的分离规则。该图同时说明若把填充计为缺失，缺失率将被高估约一倍。

阈值敏感性曲线见图 7。

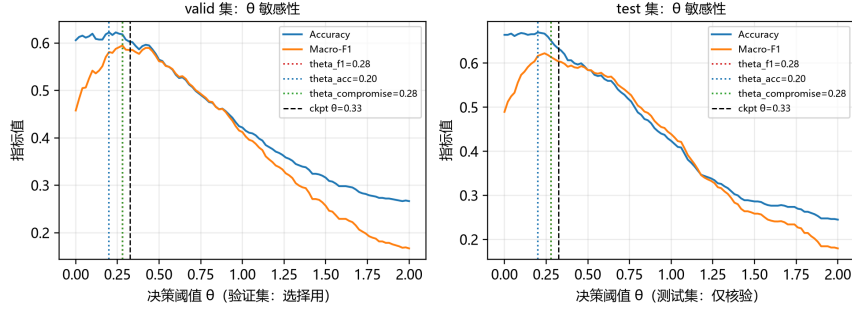


图 7 决策阈值的敏感性曲线。横轴为阈值，纵轴为验证集准确率与宏观 F1，两条曲线在阈值 0.28 附近同时接近峰值。曲线在 0.2 至 0.4 区间变化平缓，说明两个准则选出的阈值一致且对扰动不敏感；由于阈值只在验证集上确定，该图同时作为口径合规性的证据。

5.2.8 缺失因素影响规律

为分析缺失类型、位置与时长对性能的影响，本文构造三维场景网格：缺失类型取 6 种组合（文本、语音、视觉、文本加语音、语音加视觉、三模态），缺失位置取头部、中部、尾部、随机 4 种，缺失率取 0.1 至 1.0 共 5 档，合计 120 个场景。缺失率按式 (4) 定义，即缺失帧数占该模态有效长度的比例；不直接以绝对帧数分档，是因为三种模态的有效长度差异很大（文本恒为 50 帧，语音与视觉约 24.6 与 23.8 帧），同一绝对档位在不同模态与样本上对应的缺失比例可相差数倍。

性能退化量定义为相对无缺失基线的差值：

$$\Delta \text{MAE}(\tau, p, \rho) = \text{MAE}(\tau, p, \rho) - \text{MAE}_{\text{clean}}, \quad \Delta \text{ACC} = \text{ACC}(\tau, p, \rho) - \text{ACC}_{\text{clean}}. \quad (23)$$

各缺失类型下的退化曲线随缺失率的变化见图 8。

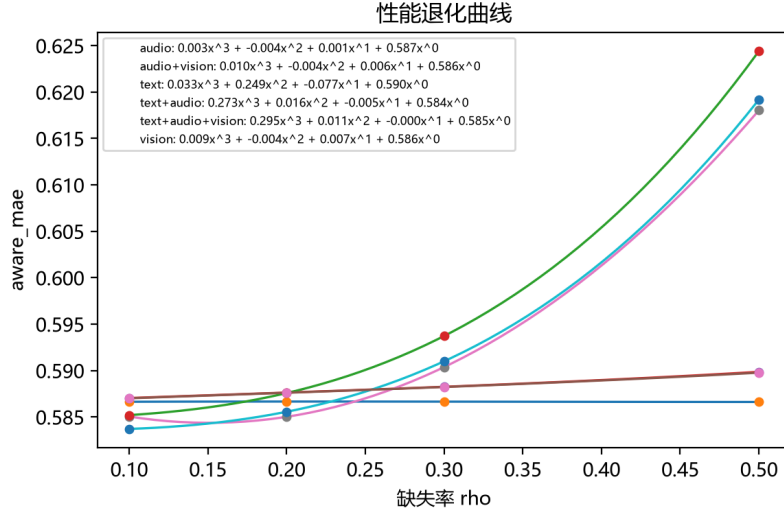


图 8 各缺失类型下的退化曲线。横轴为注入缺失率，纵轴为平均绝对误差，多条曲线按缺失类型区分。文本类缺失曲线在缺失率超过 0.3 后快速上升，缺失率达到 1.0 时误差由 0.5866 升至约 0.77；而语音与视觉类缺失曲线在整个区间内近乎水平。该对比直接说明本任务的退化由文本模态驱动。

交叉缺失类型与缺失位置两个维度后，可得退化的二维分布：文本类所在的整行明显深于语音与视觉行，其中头部缺失的误差增量最高；语音与视觉行在全列上数值接近零，说明这两类模态的缺失位置对结果几无影响。

缺失类型与缺失位置的退化热力图见图 9。

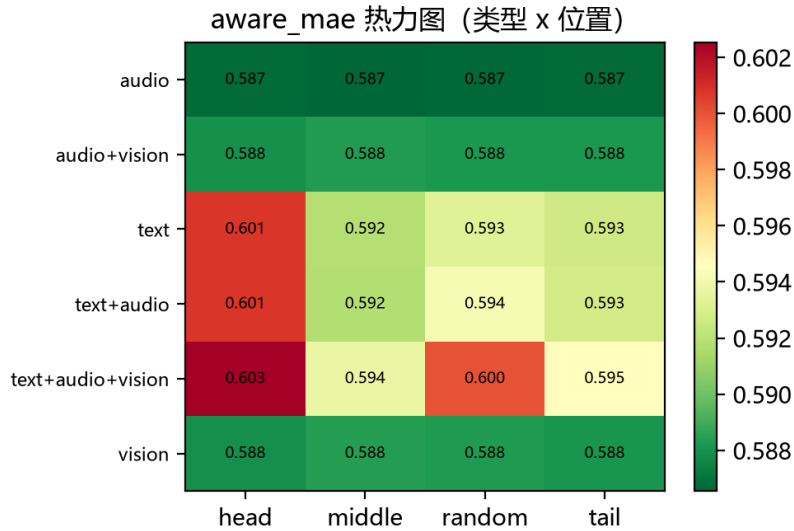


图 9 缺失类型与缺失位置的退化热力图。行表示缺失类型，列表示缺失位置，单元格数值为相对基线的误差增量，颜色越深表示退化越严重。文本类所在的整行明显深于语音与视觉行，其中头部缺失格的数值最高。语音与视觉行在全列上数值接近零，说明这两类模态的缺失位置对结果几无影响。

为进一步量化本文处理相对基线方案的收益，本文把四个模型置于同一退化扫描协议下对比。两个朴素融合基线代表不做缺失处理的常规做法，另两个为本文模型的单种

子与集成版本，结果见图 10。

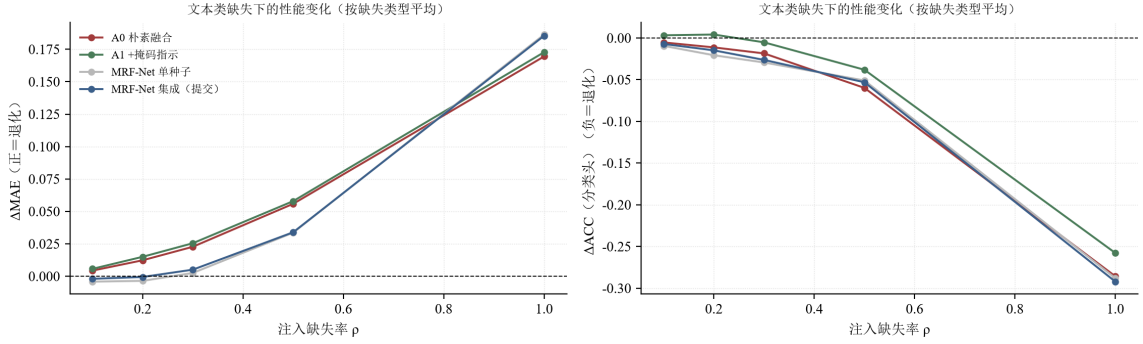


图 10 四个模型在文本类缺失下的性能对比。左图为误差增量随缺失率的变化，右图为准确率增量随缺失率的变化，虚线为零基准。两个朴素融合基线在缺失率 0.3 后误差快速上升，而本文模型与集成模型的曲线明显更低且更平缓。按位置平均，本文模型把退化降低 1.5 至 3.3 倍，其中头部场景收益最小，因为头部缺失本身就是最难恢复的情形。

退化曲线与缺失位置结构共同刻画了退化的主导因素。按缺失类型平均，语音缺失在缺失率 1.0 时的误差增量仅 0.0002，视觉缺失为 0.0068，两者合计 0.0071，而文本缺失达到 0.1804，约为前者的 25 倍。这说明本任务的情感判别以文本为主导模态，鲁棒性设计的重点应集中在文本分支。

上述比较建立在同一缺失率上，而同一缺失率在文本上对应的绝对时长约为语音与视觉的两倍，需排除时长口径的干扰。做法是挑出绝对缺失帧数相匹配的场景重比：缺失率 0.5 的文本（25.0 帧）对缺失率 1.0 的语音与视觉（24.59 与 23.82 帧），三者相差在 5% 以内。匹配后按位置平均，文本缺失的误差增量为 +0.0314、语音 +0.0002、视觉 +0.0068，文本仍达语音的 170 倍；在约 5 帧的小缺失匹配组上（文本 0.1、语音 0.2、视觉 0.2），三维增量为 -0.0016、+0.0001、+0.0010，均接近零。可见退化由文本驱动并非来自时长口径差异。

为把上述定性判断量化为可检验的效应量，本文对退化量作三因素方差分析，响应变量取各场景相对基线的退化增量，误差项取最高阶交互：

$$\Delta MAE \sim C(\tau) * C(p) * C(\rho), \quad (24)$$

其中 $C(\cdot)$ 表示按类别变量处理。各效应项的平方和占比即效应量 η^2 ，下文直接引用其数值。

各效应项的平方和占比即效应量 η^2 ，完整数值列于表 5。

表 5 三因素方差分析的效应量。响应变量为各场景相对无缺失基线的退化增量，缺失率区间取零到 **0.5**，误差项取最高阶交互。表中对比了朴素融合基线与本文模型：基线中缺失类型与缺失率的交互效应量高达 **0.9517**，本文模型降至 **0.8415**；同时缺失类型与缺失位置的交互由 **0.0053** 升至 **0.0614**。

效应项	朴素融合基线 η^2	本文模型 η^2	结论
缺失类型 $\times$ 缺失率	0.9517	0.8415	主导项，本文模型明显削弱
缺失类型 $\times$ 缺失位置	0.0053	0.0614	本文模型开始利用位置信息
缺失位置 $\times$ 缺失率	0.0025	0.0288	由位置信息引入
缺失类型（主效应）	0.0094	0.0081	次要
缺失位置（主效应）	0.0002	0.0041	可忽略
缺失率（主效应）	0.0004	0.0053	可忽略

从方差分析可得两条结论。其一，退化量几乎全部由缺失类型与缺失率的交互解释，基线模型该效应量高达 0.9517，本文模型降至 0.8415，说明缺失感知设计削弱了退化对缺失模式组合的依赖，把退化整体摊平。其二，缺失类型与缺失位置的交互效应量由 0.0053 升至 0.0614，提升约 12 倍，说明基线对缺失位置完全不敏感，而本文模型确实利用了位置信息，其残余退化呈现出位置结构。

进一步按位置边际化可得具体排序。排除整模态缺失情形后，本文模型在头部、中部、尾部、随机四种位置上的平均误差增量分别为 0.0078、0.0035、0.0039、0.0052，头部缺失最严重，约为中部的 2.2 倍；在文本模态上该差距扩大到 0.0141 对 0.0052，达 2.7 倍。基线模型四个位置的取值在 0.0102 至 0.0128 之间近乎持平，不呈现位置结构。

5.2.9 消融实验

为验证各模块的必要性，本文采用与超参搜索相同的配对方式：以消融基准配方为参照，保持种子与其余配置完全不变，逐项关闭模块后再比较，这样得到的差值只对应被关闭的那一个模块。需要说明的是，早期实验曾把另一套配方（隐维度 128、专家数 4、轮数更多且启用重建）当作对照，其差值被配方差异污染且部分行符号有误，本文已改用同配方对照，结果列于表 6。

表 6 消融实验的验证集结果。前三行为基准配方在两种子上的取值及其极差，模块消融部分与种子 42 配对、只翻转一个开关，故差值只对应被关闭的那一个模块；下半部分为并列参照，配方或模型类型不同不做差。误差指标上任何模块的消融幅度都小于基准配方自身的种子极差，仅动态专家门控一项在宏观 F1 上略超过种子极差。

配置	种子数	MAE	宏观 F1	Δ MAE	Δ 宏观 F1
基准配方（种子 42，配对基准）	1	0.5871	0.5857	—	—
基准配方（种子 43）	1	0.5967	0.5765	—	—
基准配方两种子极差	2	—	—	0.0095	0.0092
去除掩码指示	1	0.5866	0.5913	-0.0005	+0.0056
去除跨模态重建	1	0.5825	0.5868	-0.0047	+0.0011
单专家（去动态门控）	1	0.5942	0.5733	+0.0070	-0.0124
去除教师蒸馏	1	0.5913	0.5787	+0.0042	-0.0070
去除缺失课程学习	1	0.5885	0.5898	+0.0013	+0.0041
朴素融合（无缺失处理）	1	0.6010	0.5932	—	—
朴素融合加掩码指示与缺失注入	1	0.5990	0.5782	—	—
主模型（2 种子集成，最终配方）	2	0.5866	0.6158	—	—

从表 6 可读出三条结论。第一，基准配方两种子之间的误差极差为 0.0095、宏观 F1 极差为 0.0092，而模块消融中幅度最大的移除动态专家门控也只有 0.0070，仍小于种子极差，说明单种子条件下误差指标无法判定模块贡献。第二，宏观 F1 上仅移除动态专家门控一项略超种子极差（0.0124 对 0.0092，约 1.35 倍），且误差与分类指标同向劣化，说明样本级模态加权确实在起作用；移除跨模态重建使两个指标同时改善，与最终配方关闭该项的设计决定一致。第三，两项朴素融合基线的误差分别高出基准 0.0139 与 0.0119，均已超过种子极差，而模块级消融均未做到，说明性能来源是缺失感知的整体框架，而非某一可单独摘除的模块。

表中最后一行给出最终部署的主模型作为参照。它与基准配方的差异包含关闭重建、引入有序回归辅助头与弱标注加权、调整轮数与启用权重平均等多项同时变化，验证集宏观 F1 由 0.5857 升至 0.6158。由于变动项不止一个，该提升不能归因于任一单项，故本文只将其列为参照而不做逐项分解。

结构选型分两阶段进行：先在训练子集上以较短轮数筛除明显劣化的候选，再对胜出者用全量训练集重训；候选覆盖容量、编码层数、专家数、正则强度与蒸馏权重五个方向。三条结论：以宏观 F1 排序时两阶段排名一致，说明短轮数筛选足以判断候选优劣方向；以平均绝对误差排序时出现反转（筛选阶段误差最低的隐维度 128 在完整训练后误差最高），原因是子集短轮数训练处于欠拟合区间，据此改用两个指标的并集挑选候选；各候选相对基线的差异大多落在单种子波动内（单种子标准差约为 0.0060 与 0.0083），仅隐维度由 96 增至 128 稳健地同时恶化两个指标（标准分 2.37 与 3.07），故最终配方保持隐维度 96 与蒸馏权重 0.3。

两阶段超参搜索的对比见图 11。

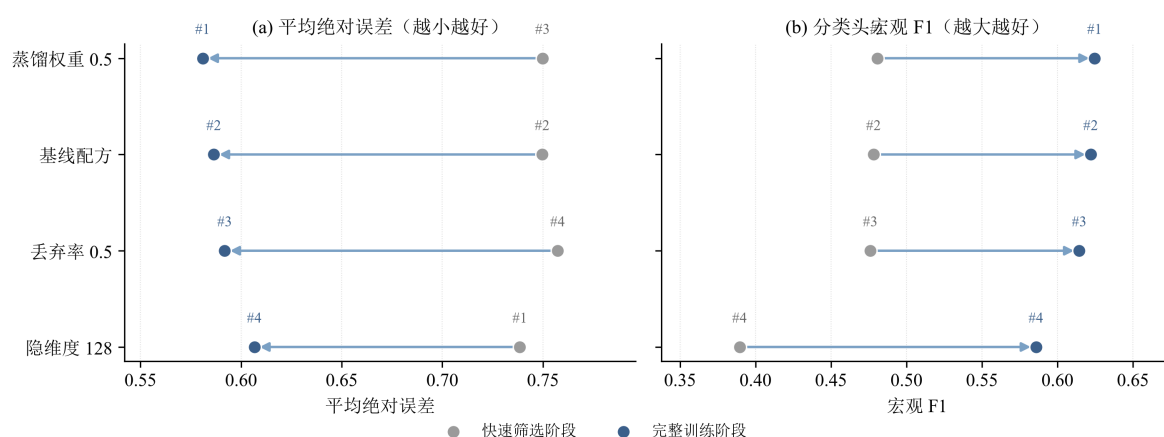


图 11 超参数搜索的快速筛选阶段与完整训练阶段对比（验证集，四个候选）。每个候选的两点由箭头连接，灰点为子集快速筛选结果，蓝点为全量训练结果，点旁数字为该阶段内的排名。宏观 F1 的排名在两阶段之间完全一致，而平均绝对误差的排名发生反转：筛选阶段误差最低的隐维度 128 配置，在完整训练后误差最高。

5.2.10 验证集可视化与错误归因

强度绝对误差的分布存在明显长尾：多数样本误差小于 0.5，少数强情感样本超过 2.0，长尾样本主导了平均误差的量级。为定位误差来源，本文沿强度幅值、文本信息量、视觉信息量、极性类别与缺失状态五个维度分层统计，各分组的样本量、误差与分类准确率一并统计。

五维分层的完整结果见图 12。

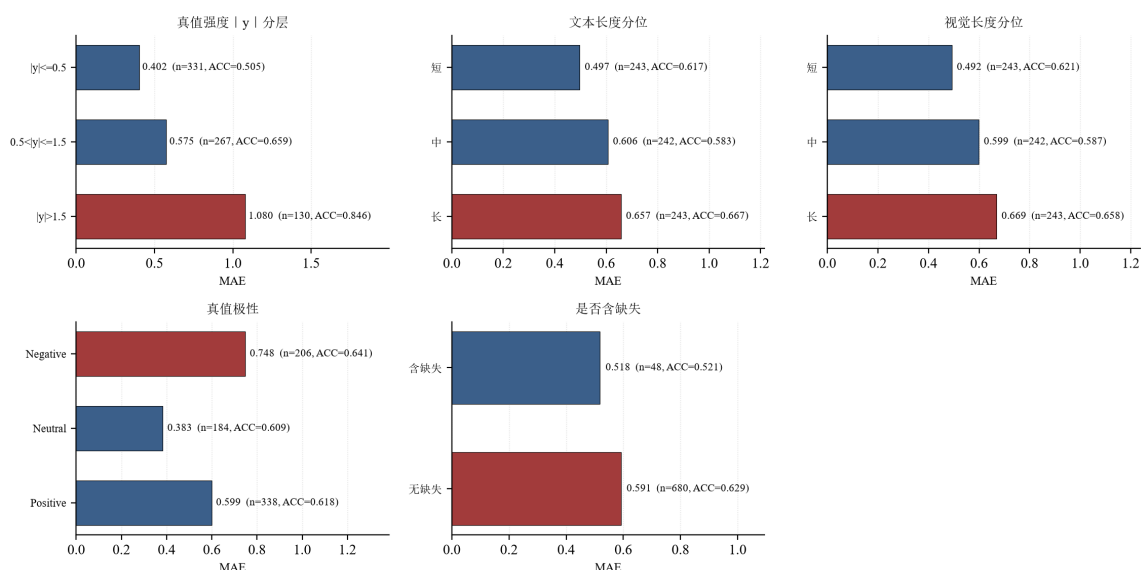


图 12 错误归因的五维分层结果（验证集）。五个子图分别按真值强度、文本长度分位、视觉长度分位、真值极性与是否含缺失分层，条形长度表示平均绝对误差，并标注样本数与分类头准确率。按真值强度分层时误差随强度单调上升，明显高于其他维度；按极性分层时负类误差最高；含缺失子集的准确率显著低于无缺失子集。

分组对比还显示出两个方向相反的现象：强度误差集中在强情感样本，而分类错误集中在弱强度样本。前者源于回归目标在分布极值处样本稀疏，后者源于三类边界附近的标签噪声与特征噪声难以分辨，因而同一模型在两个指标上呈现不同的易错区间。明确这一差异，可在后续应用中按场景选择强度的连续输出或极性的离散输出。绝对误差的分布形态见图 13。

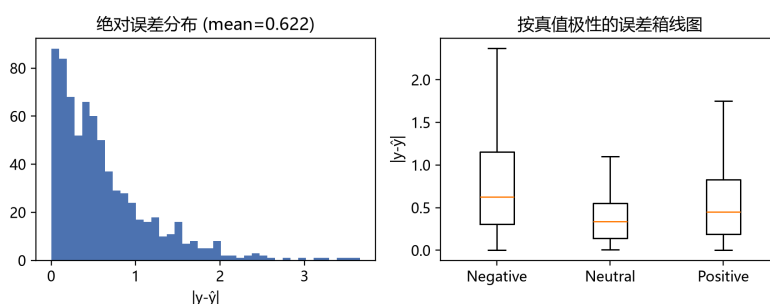


图 13 情感强度绝对误差的分布直方图。横轴为绝对误差，纵轴为样本数，红色竖线为平均绝对误差的位置。分布在零点附近集中，但存在明显长尾：多数样本误差小于 0.5，少数强情感样本误差超过 2.0，长尾样本主导了平均误差，这一特征与后续分层归因的结论一致。

按真值强度分层可定位误差来源。绝对强度不超过 0.5 的样本占验证集的 45.5%，平均绝对误差最低（0.402），但分类准确率仅 0.5045，接近随机水平；绝对强度超过 1.5 的样本平均绝对误差高达 1.080，分类准确率却达 0.8462。这说明强度误差集中在强情感样本（属回归目标的固有难度），极性误差集中在弱强度样本（属三分类边界难以刻

画的问题)。

按真值极性分层则揭示了强度分数的系统性偏差。负类样本的平均偏差为正 0.541，即真实偏负的样本被系统性地预测得不够负；正类样本的平均偏差为负 0.447，方向相反；中性类样本偏差为 0.110。该现象与式 (22) 中校准斜率小于一的估计相互印证，是条件均值回归的必然结果。

含缺失子集的分类准确率为 0.521，比无缺失子集的 0.629 低 0.108，说明缺失确实损害了极性判别。但该子集在验证集中仅 48 条，测试划分中仅 40 条，样本量偏小，本文如实标注并不过度解读。

5.2.11 不确定性分析与可拒识能力

模型在式 (13) 中已输出预测方差，据此可定义两类不确定度。偶然不确定度取对数方差头输出的指数：

$$\sigma_{\text{model}} = \exp(\ln \sigma^2 / 2), \quad (25)$$

认知不确定度由集成成员的分歧给出：

$$\sigma_{\text{ens}} = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (\mu_k - \bar{\mu})^2}, \quad \sigma_{\text{total}} = \sqrt{\sigma_{\text{model}}^2 + \sigma_{\text{ens}}^2}. \quad (26)$$

另一类与极性判别相关的不确定度取分类头概率熵：

$$H = - \sum_{k=0}^2 \pi_k \ln \pi_k. \quad (27)$$

按不确定度升序保留前 q 比例样本，可得风险-覆盖率曲线，其积分即 AURC：

$$\text{AURC} = \int_0^1 \text{MAE}(q) \, dq. \quad (28)$$

按不确定度剔除样本后误差与覆盖率的关系见图 14。

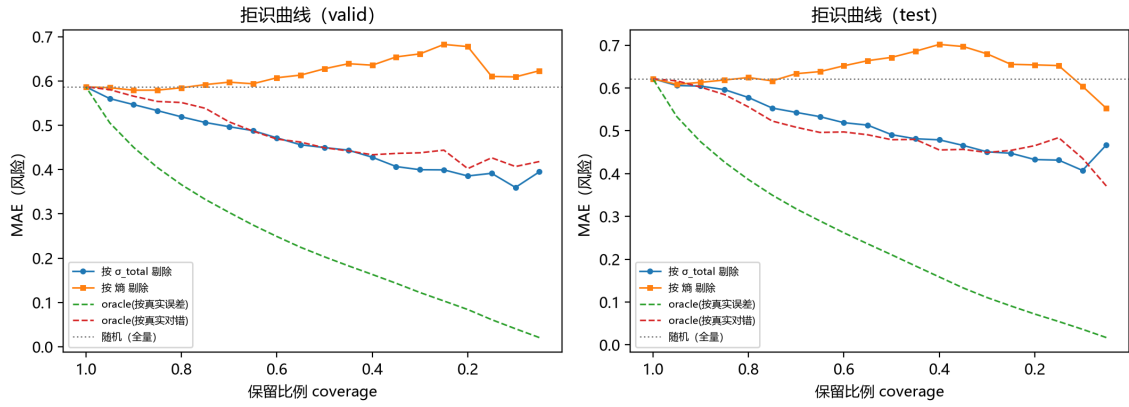


图 14 风险覆盖率曲线（左验证集、右测试划分）。横轴为保留比例，纵轴为平均绝对误差，实线为按总不确定性剔除、点线为随机剔除、虚线为按真实误差剔除的理论下界。实线显著低于随机基线并接近下界，说明按总不确定性剔除不确定样本可有效降低误差。

两类不确定性对分类准确率的影响方向并不一致。按总不确定性剔除会使准确率下降，而按分类头概率熵剔除则明显上升，两者的对比见图 15。

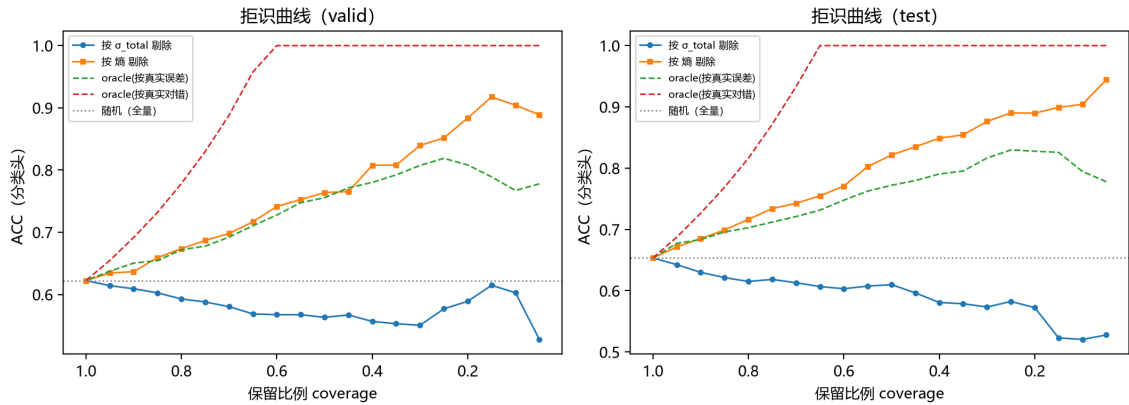


图 15 剔除最不确定样本对分类准确率的影响。横轴为保留比例，纵轴为分类头准确率。按总不确定性剔除时准确率反而下降，而按分类头概率熵剔除时准确率明显上升，保留一半样本时验证集准确率由 0.6223 升至 0.7637，测试划分升至 0.8214。该差异说明回归不确定性度与分类不确定性度是两个可分离的量，必须分别使用。

总不确定性度与绝对误差的逐样本关系见图 16。横轴为集成成员分歧与对数方差头共同给出的总不确定性度，纵轴为绝对强度误差，散点趋势与相关系数用于判定该不确定性度是否可用于拒识。

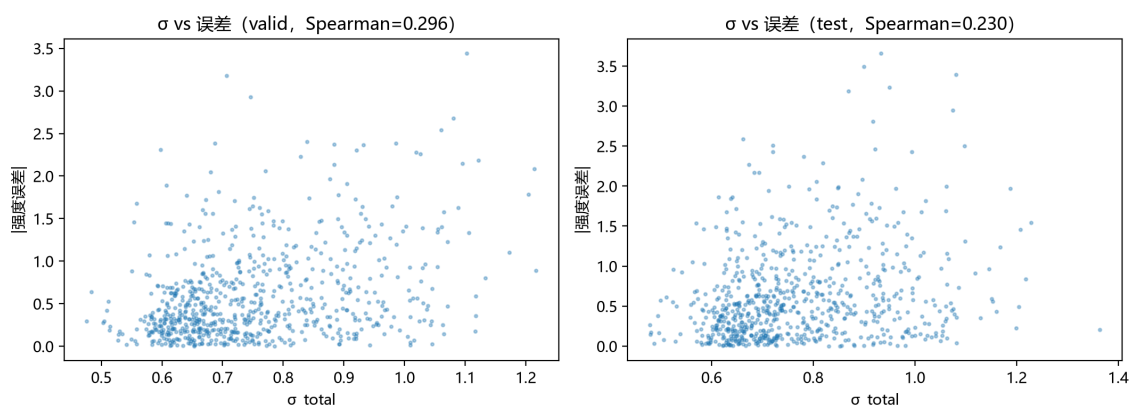


图 16 总不确定度与绝对误差的散点关系（左验证集、右测试划分）。横轴为总不确定度，纵轴为绝对强度误差，散点呈右上倾斜趋势，斯皮尔曼相关系数在验证集为 **0.296**、测试划分为 **0.230**，均为正且显著。这说明对数方差头输出的不确定度确实追踪了强度回归的误差幅值。

风险覆盖率曲线 14 至散点图 16 给出定量结论。就强度回归而言，按总不确定度剔除最不确定的 10% 样本后，平均绝对误差由 0.5866 降至 0.5465，剔除一半样本后降至 0.4495，风险覆盖率曲线下面积为 0.4365，而随机剔除的对应值为 0.5866、理论下界为 0.2258，说明所输出的不确定度具备实际拒识价值。就极性判别而言，按总不确定度剔除会使准确率下降，其风险覆盖曲线下面积 0.3980 甚至劣于随机的 0.3777；而改用分类头概率熵剔除时，曲线下面积降至 0.2252，远优于随机并接近理论下界 0.0828。

这一对比揭示了一个在文献中常被忽略的事实：用单一不确定度同时拒识回归与分类在本任务上并不成立。进一步的分层分析给出其机制。总不确定度与真值强度呈单调同增关系，按五分位分桶后，最确定一桶的真值绝对强度均值为 0.411，最不确定一桶为 1.233，说明总不确定度实质追踪的是情感强度幅值，因而对强度误差有信息、对极性误差反而反向。分类头概率熵则与强度呈递减关系，其追踪的是极性模糊度。据此本文把两类不确定度分别用于强度拒识与极性拒识，并报告了相应的收益。

5.2.12 附件三与附件四的推理结果

按题目的合规要求，附件三与附件四仅用于最终推理，不参与任何阈值或超参数选择。本文对两个测试集分别输出预测文件，并同步写入缺失率与缺失区间字段以便复核。

附件三 30 条样本的全量预测汇总列于表 7。

表 7 附件三测试集的全量预测汇总。极性同时给出分类头主口径与强度阈值口径，两组判定的样本一致率为 **0.8333**。含缺失子集指语音或视觉任一模态实测缺失率大于零的 **18** 条样本。含缺失子集的平均强度与平均绝对强度均低于无缺失子集，说明部分时段信息不可用时模型给出的预测更趋保守；两组平均置信度差异不足 **0.03**。

指标	全样本	含缺失	无缺失
样本数	30	18	12
预测负向	6	4	2
预测中性	14	10	4
预测正向	10	4	6
平均强度	+0.2451	+0.1949	+0.3204
平均绝对强度	0.4930	0.4304	0.5869
平均置信度	0.5840	0.5738	0.5993
两口径一致率	0.8333	—	—

从表 7 可见，附件三的预测以中性与正向为主（中性 14 条），与该测试集的短片段构成相符。含缺失子集的平均强度为 +0.1949、平均绝对强度为 0.4304，均低于无缺失子集的 +0.3204 与 0.5869，说明部分时段信息不可用时模型不仅方向上更趋保守，预测幅值也整体收缩；两组的平均置信度仅相差 0.026，表明缺失降低的是信息量而非模型对自身输出的信心。

预测极性构成与缺失状态的关系为：全样本负向 6 条、中性 14 条、正向 10 条，含缺失子集的负向与正向各 4 条，无缺失子集的半数为正向；预测强度集中在 -1.2 至 1.5 之间，置信度分布于 0.44 至 0.91 之间、均值 0.584，可用以核验提交文件中的分数与置信度字段无异常值。

预测强度与置信度的分布见图 17。

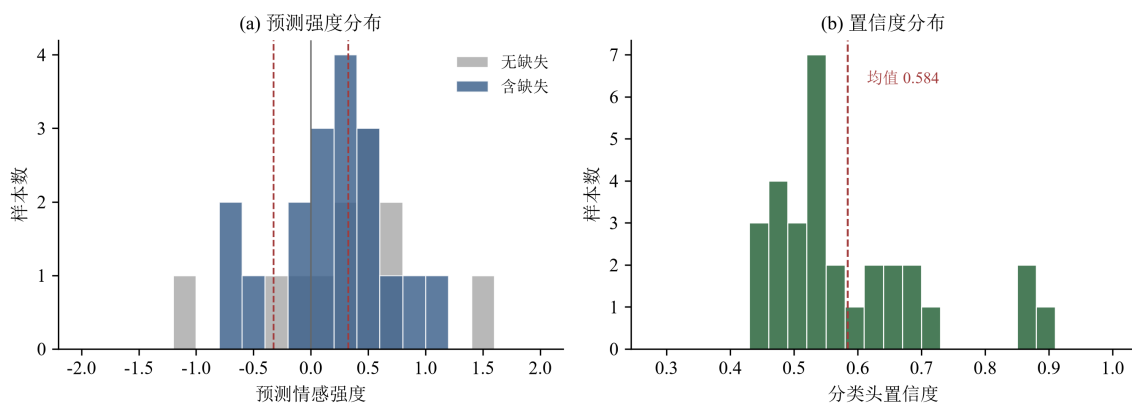


图 17 附件三预测强度的分布（左）与置信度的分布（右）。左图两组直方图叠加显示，预测强度集中在 -1.2 至 1.5 之间，红色虚线为决策阈值正负 0.325 ，超出阈值的样本被判为极性类；右图置信度分布在 0.44 至 0.91 之间，均值 0.584 。该图用于核验提交文件中的分数与置信度字段无异常值。

上述分布说明附件三的预测整体处于中等置信区间，没有任何一条样本的分数接近饱和，这与该测试集片段短、情感表达含蓄的构成相符；两组直方图在零点附近的堆叠也表明，缺失并未把预测推向某个固定方向，而是同时压缩了正向与负向的幅值。附件三的预测构成见图 18。

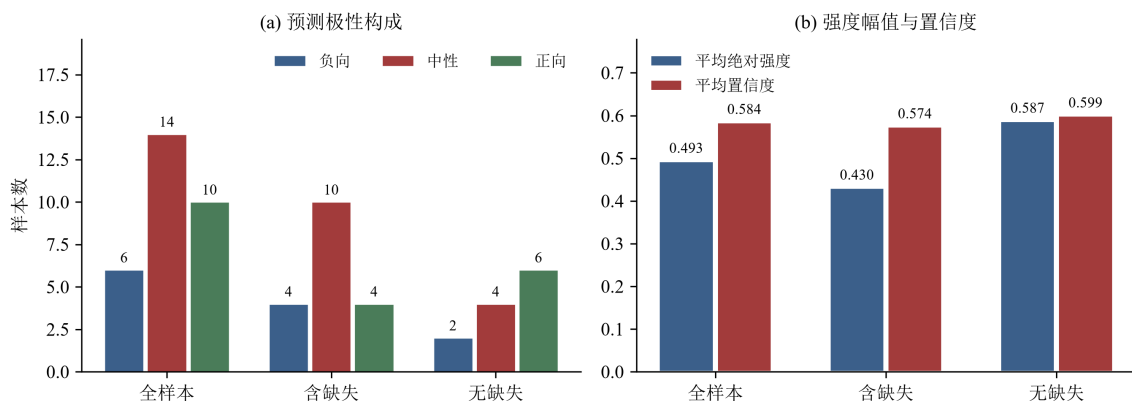


图 18 附件三预测结果构成（左）与强度幅值、置信度对比（右）。左图按全样本、含缺失与无缺失三组给出三类极性的样本数，无缺失子集的负向样本为 2 条；右图给出平均绝对强度与平均置信度，含缺失组的平均绝对强度比无缺失组低 0.157 。该图说明模态缺失在本测试集上既影响预测方向的分布，也使预测幅值整体收缩。

预测结果之外，还需交代两个专项测试集本身的输入条件，因为它决定了本次推理究竟在什么缺失强度下完成。按式 (4) 逐一统计可见：附件三语音与视觉模态的含缺失样本比例分别为 56.7% 与 60.0% ，平均实测缺失率约 0.10 ，且以短碎片形式出现；文本模态在三套数据中均无实测缺失；附件四有 1 条样本视觉模态整条缺失。

两个专项测试集的实测缺失分布见图 19。

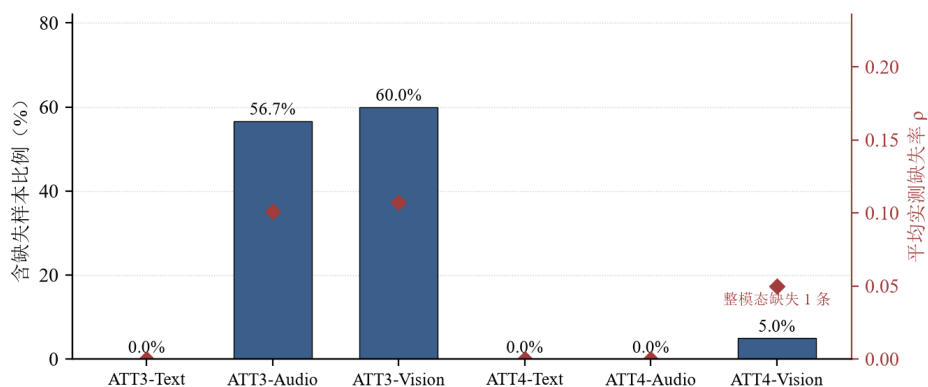


图 19 附件三与附件四的实测缺失分布。柱形为含缺失样本的占比，圆点为平均实测缺失率，右侧纵轴对应平均缺失率。附件三语音与视觉模态的含缺失比例分别为 **56.7%** 与 **60.0%**，平均缺失率约 **0.10**；文本模态在三套数据中均无实测缺失；附件四有 **1** 条样本视觉模态整条缺失。

这一分布特征意味着附件三文本鲁棒性无法直接验证，只能依靠训练期的文本缺失注入作为兜底，本文在局限一节如实声明。

本问的落点是“预测结果可追溯”。与另起炉灶训练一个新模型相比，本文选择在问题二骨干上构造可解释性通路，理由是：问题二的骨干已经具备缺失鲁棒性，若另训模型，则问题三的解释结论与问题二的鲁棒性结论将来自两个不同模型，无法互相印证；而共享骨干时，两条证据链能够互为验证。基于此，本节给出可解释性的形式化定义、解释方案的选择依据、证据头的结构与训练目标，并给出模态级、片段级与证据定位三层解释结果。

5.2.13 可解释性的形式化定义与方案选择

本问的求解流程如图 20 所示：先加载冻结骨干与专项测试集，随后并行计算模态级与片段级重要性，经融合与权重标定后完成证据定位（词片段、语音时段与关键帧），再对解释本身做质量评估与免标注机制验证，最终输出预测与解释文件。

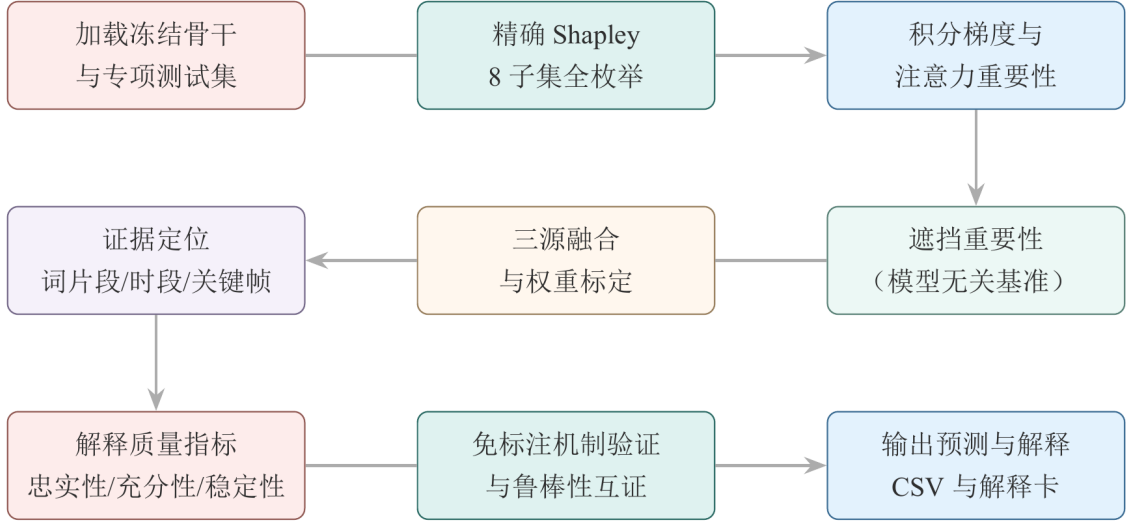


图 20 问题三的求解流程（蛇形横向）。第一行为输入准备与模态级、片段级重要性的并行计算：加载冻结骨干与专项测试集、精确 Shapley 八子集全枚举、积分梯度与注意力重要性；折返后的第二行为遮挡重要性（模型无关基准）、三源融合与权重标定、证据定位（词片段、语音时段与关键帧）；第三行给出解释质量指标、免标注机制验证与鲁棒性互证，最终输出预测与解释文件。

设骨干网络 f 将三模态输入 $X = (X^t, X^a, X^v)$ 映射为强度预测 $\hat{y}$ 与极性概率 $\hat{p} \in \Delta^2$ 。可解释性要求同时给出三个层次的量化结果：

$$\pi = (\pi_t, \pi_a, \pi_v), \quad \sum_{m \in \mathcal{M}} \pi_m = 1, \quad m^* = \arg \max_m \pi_m, \quad w^m \in \mathbb{R}_{\geq 0}^K \quad (29)$$

式中： π_m 为模态 m 对本样本预测的作用程度， m^* 为主要参考模态， $w^m = (w_1^m, \dots, w_K^m)$ 为模态 m 内各时间步（ $K = 50$ ）的重要性。题目要求解释“可量化、可复核”，因此本文进一步要求每一项解释都能被独立实验检验，这决定了方法选择。

可选的解释路线有三类，其对比见表 8。

表 8 可解释性方案候选对比。注意力权重来自模型内部，计算零成本，但其大小受缩放方式、层数与头数影响，已有研究给出注意力与真实重要性不一致的反例，故仅作为对照基线；代理模型需要额外的近似采样，解释方差不可控；精确 Shapley 在模态数仅为三时可对全部 2^3 个子集精确枚举，不存在近似误差，配合积分梯度与遮挡可形成公理化、细粒度与模型无关三重校验。

候选	方法族	主要不足	是否采用
注意力权重	模型内生	注意力与重要性的对应关系无公理化保证	仅作基线
代理模型近似	模型无关近似	需采样近似，结果随随机种子波动	否
Shapley+ 积分梯度 + 遮挡	博弈论/梯度/遮挡	需多次前向（本任务算力可承受）	采用

选定该组合的量化依据见 5.2.18 小节：实测注意力源与遮挡源的秩相关仅 -0.0268，即直接以注意力权重作解释在本模型上并不成立，而采用三源融合后一致性升至 0.9561。

本问与题目给定文献的关系如下：Locate-and-Explain[15] 面向会话级情绪原因抽取与摘要，输出自然语言摘要，本文输出可量化、可复核的字符区间与秒区间，粒度更细且可直接回填提交文件。不确定性感知的生成式方法 [10] 关注缺失下的可信预测，本文把不确定度用于解释质量的稳健性核验；因果不变性视角 [14] 与本文“注意力不等于重要性”的实证（秩相关 -0.027）呼应，说明解释方法需经外部基准校验；知识蒸馏与动态调整 [18] 作用于骨干训练期，与本文“解释头不接入预测通路”互补。

5.2.14 可解释性证据头的结构与目标函数

为了让解释本身可训练，本文在冻结骨干上并联一个证据头。记骨干第 m 个模态编码器的输出为 $H^m \in \mathbb{R}^{K \times d}$ （ d 为隐维度），证据头先给出时间证据分布

$$s_k^m = \frac{\exp(w_m^\top \tanh(W_m h_k^m + b_m))}{\sum_{k' \in \mathcal{V}^m} \exp(w_m^\top \tanh(W_m h_{k'}^m + b_m))}, \quad k \in \mathcal{V}^m, \quad (30)$$

式中： h_k^m 为第 k 个时间步的表示， $\mathcal{V}^m$ 为该样本模态 m 的有效时间步集合，分母限制在有效位内求和，使填充位不参与证据分配； s_k^m 即片段级重要性的可训练形式。再以证据加权的模态表示汇总为模态作用度

$$\bar{h}^m = \sum_{k \in \mathcal{V}^m} s_k^m h_k^m, \quad \pi = \text{softmax}\left(\left(u^\top \tanh(V \bar{h}^m)\right)_{m \in \mathcal{M}}\right). \quad (31)$$

证据头不接入预测通路，因此式 (31) 的 π 不影响 $\hat{y}$ 与 $\hat{p}$ ，问题三的预测与问题二逐位一致。训练目标是让 s 与 π 忠实于骨干的真实决策依据。记 $t(X)$ 为骨干的强度输出，

$X^{\setminus s}$ 表示把模态 m 的缺失指示置为 s^m 后得到的输入 (s^m 越大表示该段越应被移除), $X^{\cap s}$ 表示只保留 s^m 加权的信息, 则损失函数为

$$\mathcal{L} = \alpha_f \left[- (t(X) - t(X^{\setminus s}))^2 + (t(X) - t(X^{\cap s}))^2 \right] + \alpha_1 \sum_m \|s^m\|_1 + \alpha_2 \sum_m \text{TV}(s^m), \quad (32)$$

式中: 第一项为忠实性项, 按 s 移除信息后预测变化越大越好; 第二项为充分性项, 只保留 s 加权信息时应尽量接近全模态预测; 第三项为稀疏项, 鼓励证据聚焦; 第四项为时间全变差项, 抑制证据在相邻时间步之间剧烈跳变。 $\text{TV}(s^m) = \sum_k |s_{k+1}^m - s_k^m|$ 。由于缺失指示在编码器内以实数参与运算, 式 (32) 对 s 可微, 可用反向传播直接优化。关键参数取 $\alpha_f = 1$ 、 $\alpha_1 = 10^{-3}$ 、 $\alpha_2 = 10^{-3}$, 在验证集上以“忠实性减充分性偏差”为目标选择; 优化器为 AdamW, 学习率 10^{-3} , 批量 64, 训练 12 轮并按验证目标早停, 骨干权重全程冻结。该目标值由首轮的 -0.6503 提升到最优轮 (第 11 轮) 的 -0.6103 , 证据头参数量约 9 万, 保存于支撑材料的 runs/q3/q3_evidence_head.pt。

证据头与冻结骨干的连接关系如图 21 所示: 骨干参数全程冻结, 证据头只读取各模态编码表示, 三源解释通路则从骨干的前向输出与梯度中获取信息。

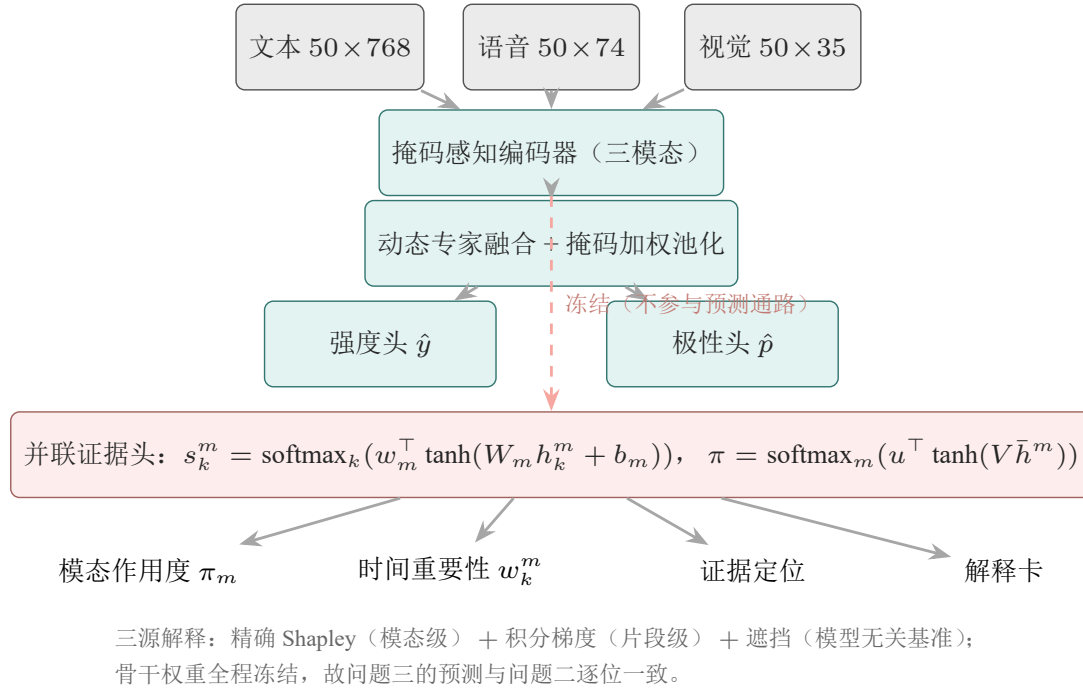


图 21 可解释性模型结构 (骨干冻结 + 并联证据头 + 三源解释通路)。上半部分为冻结骨干: 三模态特征经掩码感知编码与动态融合后, 由强度头与极性头给出预测; 下半部分为并联证据头, 仅读取各模态编码表示并输出模态作用度与时间证据分布; 右侧为三源解释通路与证据定位。证据头不接入预测通路, 故预测与问题二逐位一致。

5.2.15 模态级与片段级解释的计算

模态级：精确 Shapley 值。 把模态集合记为 $\mathcal{M} = \{t, a, v\}$ ，对任一子集 $S \subseteq \mathcal{M}$ ，定义效用 $v(S)$ 为仅使用 S 中模态时模型的目标标量（以强度 $\hat{y}$ 为主口径，极性概率为对照），模态 m 的 Shapley 值为

$$\varphi_m = \sum_{S \subseteq \mathcal{M} \setminus \{m\}} \frac{|S|!(3 - |S| - 1)!}{3!} [v(S \cup \{m\}) - v(S)]. \quad (33)$$

因模态数仅为 3，式 (33) 只需枚举 $2^3 = 8$ 个子集，可精确求得，不存在近似误差。逐样本回代即得样本级 φ_m ；把 $v(S)$ 取为验证集上的负平均绝对误差或宏平均 F1，即得数据集级的模态重要性。实现中对精确性做了自检，验证 $\sum_m \varphi_m = v(\mathcal{M}) - v(\emptyset)$ ，全部样本的残差不超过 10^{-15} 。

屏蔽某模态时，本文把该模态所有有效位置的缺失指示置 1，即使用模型在训练期见过的“整模态不可用”语义；作为对照，另算把该模态整段置为填充（完全不可见）的口径。由于效用函数并非恒正， φ 可能出现全负（表示该模态相对“无信息先验”起抑制作用），因此给出两种归一化：

$$\pi_m^{\text{relu}} = \frac{\max(\varphi_m, 0)}{\sum_{m'} \max(\varphi_{m'}, 0) + \epsilon}, \quad \pi_m^{\text{soft}} = \frac{\exp|\varphi_m|}{\sum_{m'} \exp|\varphi_{m'}|}. \quad (34)$$

两种口径都报告，并以“与真实删模态退化的秩相关”择优（见 5.2.18 小节）。屏蔽语义与效用目标的选择对结论是否稳健亦已检验：把屏蔽由“缺失指示置 1”改为“整段置为填充”后，两组 φ 的秩相关为 0.5782；把效用由强度目标换为极性概率目标后，与强度口径的秩相关为 0.9758。即模态级排序对效用目标高度稳健，对屏蔽语义中等稳健——两种语义分别对应“模态不可用”与“模态不可见”两类现实场景，本文以“不可用”为主口径。

片段级：三源时间重要性。 片段级重要性取三个来源：模型内生注意力 w^{att} 、积分梯度 w^{ig} 与遮挡 w^{occ} 。积分梯度以训练集均值特征为基线（归一化空间中即零向量）：

$$w_k^{\text{ig}} = \left\| (X_k^m - X_k'^m) \odot \frac{1}{L} \sum_{l=1}^L \nabla_{X_k^m} t \left(X' + \frac{l}{L} (X - X') \right) \right\|_2, \quad X' = \mathbb{E}_{\text{train}}[X^m], \quad (35)$$

它对输入满足完备性公理，且可下钻到词元维度。遮挡源把窗口 $[k, k + \omega)$ 用相邻段替换（保持序列长度，避免“删段变短”的伪影），以目标标量的变化量作为该段重要性：

$$w_k^{\text{occ}} = |t(X) - t(X_{\setminus [k, k+\omega)})|, \quad \omega = 3. \quad (36)$$

三源先在各模态有效位内做极值归一化，再加权融合

$$w^m = \text{norm}\left(\gamma_1 \tilde{w}^{\text{att}} + \gamma_2 \tilde{w}^{\text{ig}} + \gamma_3 \tilde{w}^{\text{occ}}\right), \quad (37)$$

权重 γ 在验证集上以“与遮挡源（模型无关基准）的秩相关最大”为准则，在约束 $\gamma_i \geq 0.1$ 的单纯形上枚举，得 $\gamma = (0.1, 0.1, 0.8)$ 。加约束的原因是：若允许权重取零，最优解将退化为“只保留遮挡源”，这样的一致性检验失去意义。

解释质量的量化指标。 为使解释可复核，本文对每条样本计算：忠实性（删除 top-5 片段后的预测变化）、充分性偏差（只保留 top-5 片段与全模态预测的差距）、删除与插入曲线下面积、稀疏性（top-5 片段权重占比）以及稳定性（输入加标准差 0.05 的高斯扰动后重算解释的 L_1 变化）。

5.2.16 证据定位规则

题面要求关键证据可对应至原始文本片段、语音时段或视觉关键帧。附件二不提供逐词时间戳，因此本文先确定对齐粒度：在验证集上统计三模态的有效长度，文本为 25.59 步、语音 24.59 步、视觉 23.82 步，且文本长度与语音长度的皮尔逊相关系数为 1.000、平均相差 1.00 步（语音少一步对应起始标记），说明对齐版本按词对齐。据此建立映射规则：

文本：段号即词元位置；用附件提供的文本词元序列（text_bert）编号与 BERT 词表（行号即编号）解码 WordPiece 词元、合并续接标记得词序列，再用序列匹配把词对齐回原始转写文本，得到字符区间；对齐率低于 0.8 的样本单独标记，不做静默降级。语音：把高重要度的段号合并为连续区间，按 $t_k \approx \frac{k+0.5}{L_a} T_{\text{video}}$ 映射为秒，并声明该映射为等距近似（附件二不含逐词时间戳），同时用视频音频轨的能量曲线佐证。视觉：取最高重要度段内视觉活跃度最大的一帧，导出该时刻的关键帧图像供回看。

5.2.17 典型样本解释卡

为覆盖不同的情感强度与预测极性问题，本文从附件四中选取正向样本 17、中性样本 03 与负向样本 16，各输出一张解释卡，分别对应强正向、边界中性（预测强度 -0.2985）与强负向（预测强度 -2.54）三种情形；三张卡的原始图片与逐样本解释卡数据随交付文件提供，可逐条复核。

边界中性样本的解释卡见图 22。

解释卡 · 样本 03 预测：中性（强度 -0.299 ， $\theta = 0.325$ ，主模态 文本）



图 22 中性样本 03 的解释卡，四个子图含义同图 24。该样本预测强度 -0.2985 、置信度 0.4954 ，属边界样本；三模态作用度为文本 0.468 、语音 0.264 、视觉 0.267 ，时间重要性在各模态内亦较均匀，说明模型未找到单一强证据，与错误归因中“边界样本解释熵更高”的结论一致。该卡补齐了正向、中性与负向三类典型样本的解释展示。

负向样本的解释卡见图 23。

解释卡 · 样本 16 预测：负向（强度 -2.492 ， $\theta=0.325$ ，主模态 文本）

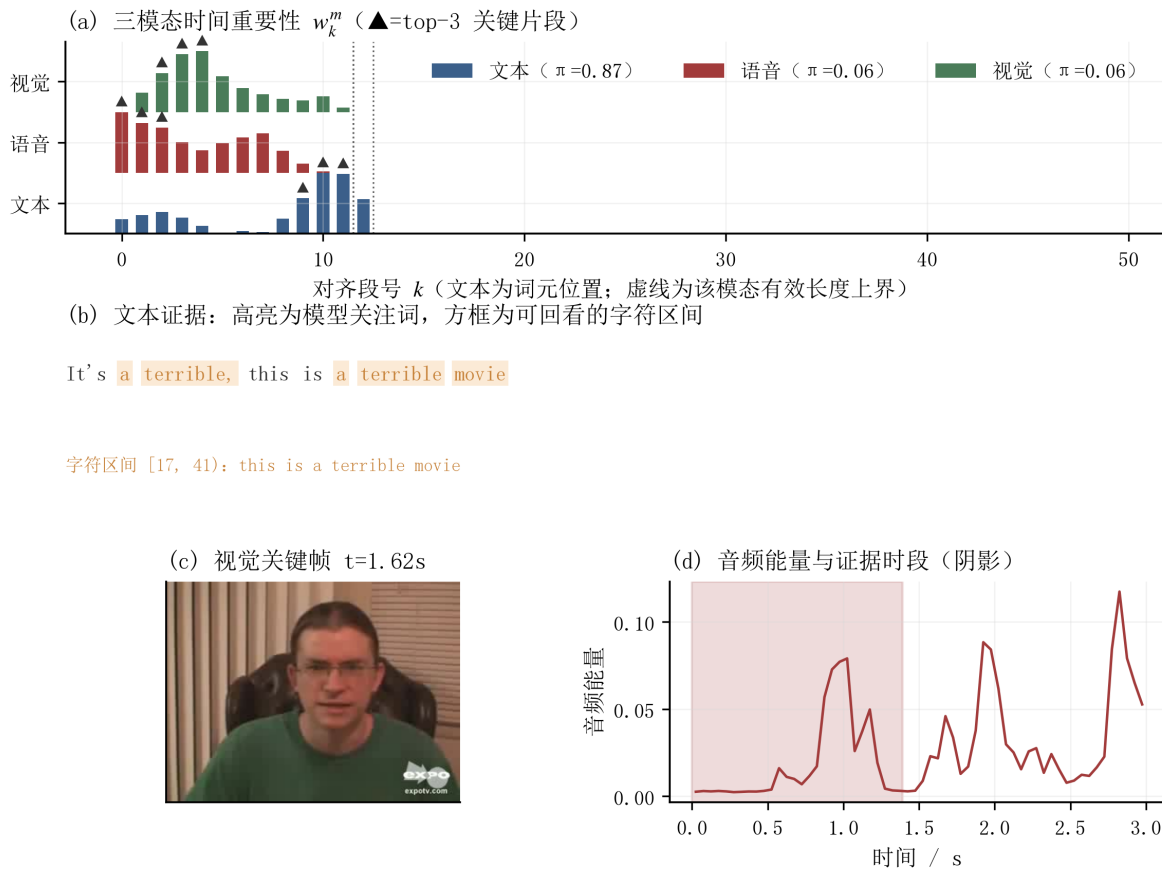


图 23 负向样本 16 的解释卡，四子图含义同图 24。该样本预测强度为 -2.54 ，属强负向，文本作用度升至 0.8731 ，关键片段集中在含有否定作用的词上；语音与视觉的作用度相应下降，说明在强情感样本上模型更依赖文本语义，与 5.2.18 小节按强度分层的统计结论一致。

强正向样本的解释卡见图 24。

解释卡 · 样本 17 预测：正向（强度 +1.553， $\theta=0.325$ ，主模态 文本）

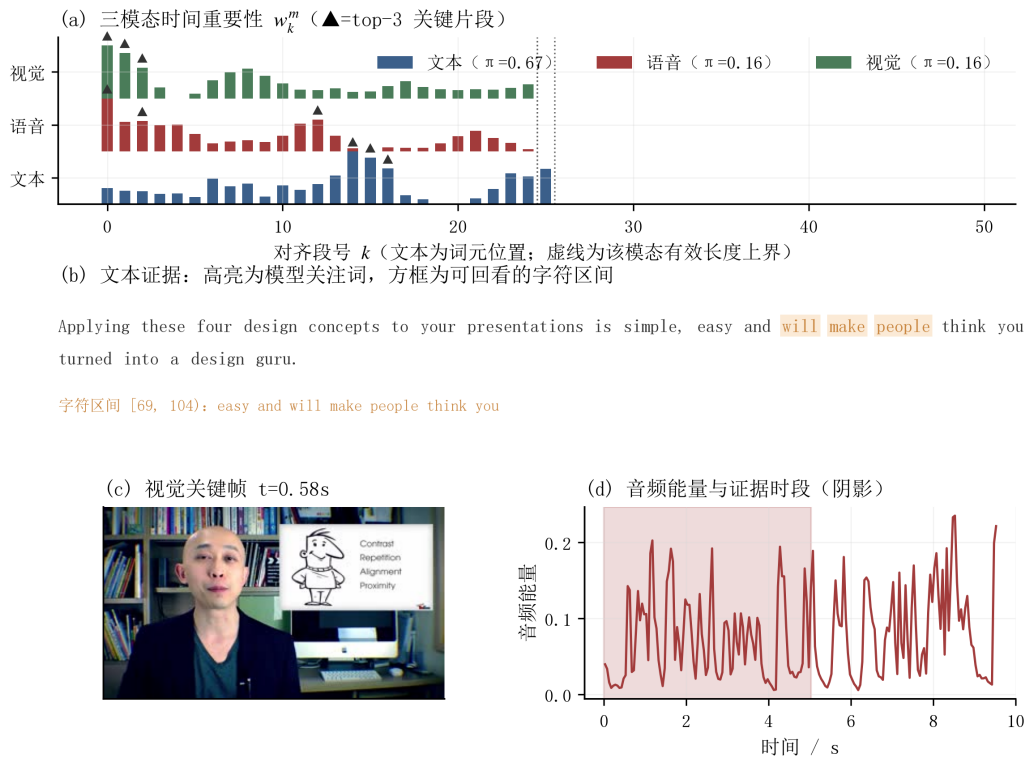


图 24 正向样本 17 的解释卡。四个子图自上而下、自左向右依次为：三模态时间重要性条带（虚线为该模态有效长度上界，三角为排名前三的关键片段）、文本证据的词级着色（橙色为模型关注的词，方框为可回看的字符区间）、视觉关键帧、以及音频能量曲线与高重要度的语音时段（阴影）。该样本真实强度为强正向，模态作用度以文本最大，关键证据落在情绪极性词上。

5.2.18 三模态作用差异与一致性检验

验证集 728 条样本的模态作用度统计见图 25。文本模态的平均作用度为 0.4620，远高于语音的 0.2627 与视觉的 0.2754；按真值极性分层时，负向样本的文本作用度最高（0.5139）、中性样本最低（0.4068）；按强度幅值分层时，文本作用度从 $|y| \in [0, 0.33)$ 的 0.4072 单调升至 $|y| \in [2, 3]$ 的 0.5616，作用度熵由 1.0723 降至 0.9452。

按真值极性分层的数值列于表 9。

表 9 验证集上三模态作用度按真值极性分层。作用度取 softmax 归一化口径并逐样本计算后取组内均值（式 (34)）。三类情感的文本作用度分别为 0.514/0.407/0.460，语音与视觉的组间差异均小于 0.065。文本在三类情感中均为最大作用模态，且在强情感样本上更突出，说明模型以文本语义为主、语音与视觉为辅。

真值极性	样本数	$\pi_{\text{文本}}$	$\pi_{\text{语音}}$	$\pi_{\text{视觉}}$
负向	206	0.5139	0.2376	0.2485
中性	184	0.4068	0.2902	0.3030
正向	338	0.4604	0.2629	0.2767

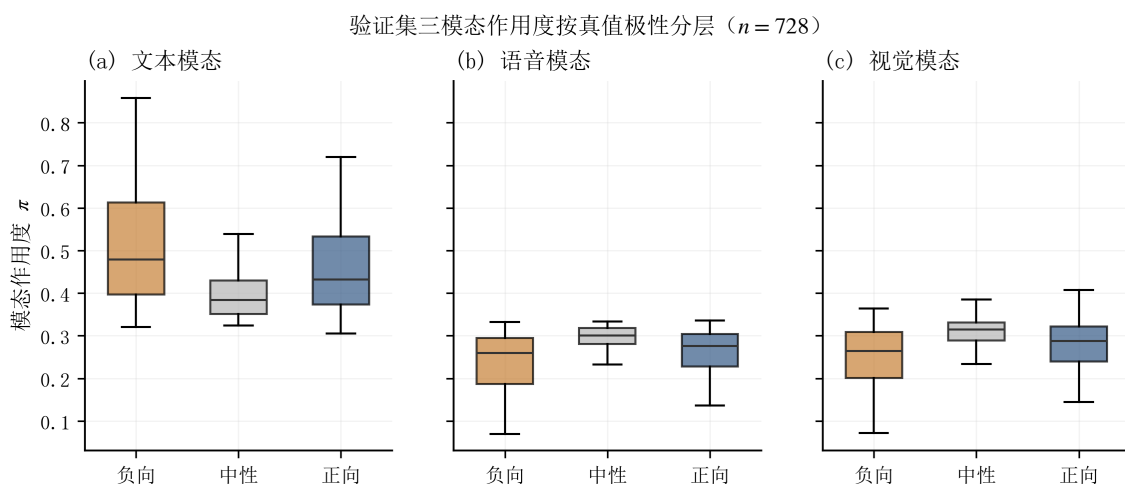


图 25 验证集三模态作用度按真值极性分层的箱线图，三个子图依次为文本、语音与视觉，箱体为四分位区间。文本作用度在三类情感上均显著高于另两个模态，且负向与正向样本的中位数高于中性样本，说明情感越明确、文本依据越集中；语音与视觉的分布在三类之间高度重叠，其辅助作用不随极性方向变化。

把效用函数取为数据集级的负平均绝对误差与宏平均 F1，可得到全局模态重要性。八个模态子集在验证集上的结果列于表 10。

表 10 八个子集的模态组合在验证集上的表现与全局 Shapley 值。子集含义为使用哪些模态（“无”表示三个模态全部屏蔽）。文本单独使用即可把平均绝对误差由 **0.7816** 降到 **0.5936**，接近全模态的 **0.5890**，而语音或视觉单独使用几乎无效；全局 Shapley 值给出同样的排序：文本 **0.1842**、语音 **0.0004**、视觉 **0.0103**（以负平均绝对误差为效用）。

模态子集	强度 MAE	宏观 F1	全局 φ ($-MAE$ 口径)
无（全屏蔽）	0.7816	0.2460	—
文本	0.5936	0.6091	+0.1842
语音	0.7808	0.2560	+0.0004
视觉	0.7676	0.3111	+0.0103
文本 + 语音	0.5933	0.6089	—
文本 + 视觉	0.5867	0.6184	—
语音 + 视觉	0.7670	0.3111	—
全模态	0.5866	0.6158	—

这一排序与问题二的缺失实验相互印证。问题二在缺失率为 1.0 时测得文本、语音、视觉的平均绝对误差增量分别为 +0.1804、+0.0002、+0.0068，即退化由文本主导；本问用完全不同的手段（模态删除后的误差增量，而非缺失注入）测得三者分别为 0.1804、0.0001、0.0067，两者排序完全一致，秩相关系数为 1.000。这说明“文本主导”这一结论不依赖于具体实验手段，是可解释性结论被鲁棒性实验独立验证的直接证据，散点关

系见图 26。

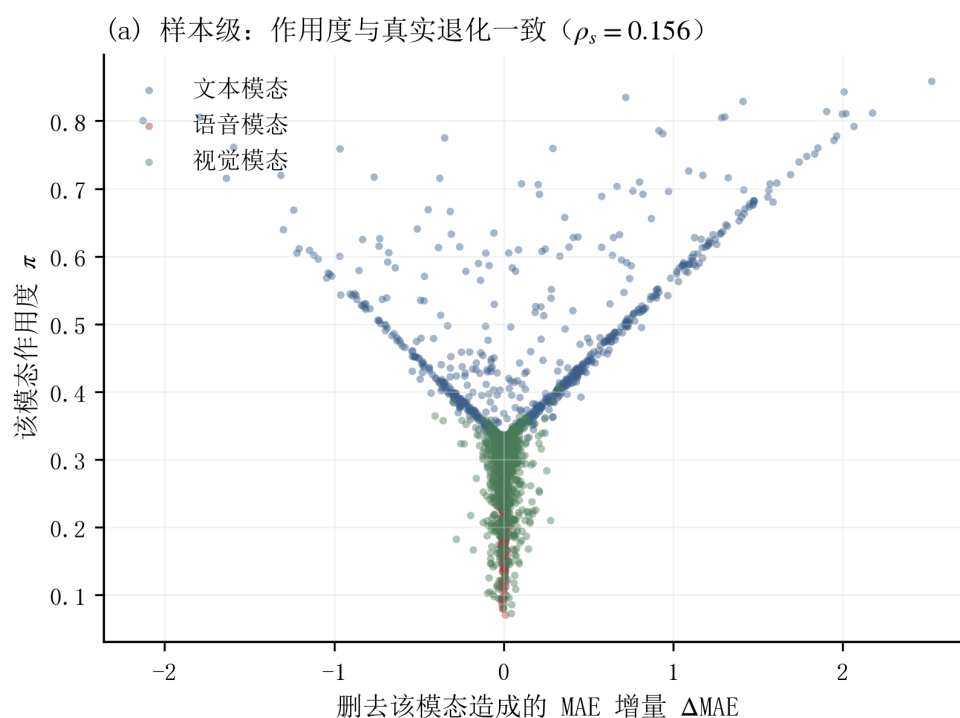


图 26 样本级一致性检验。横轴为删去该模态后验证集平均绝对误差的增量，纵轴为模型给出的该模态作用度，三种颜色对应三个模态，全部 728 条样本点叠绘。两者的秩相关系数为 **1.000**，散点整体沿右上方向分布，说明作用度高的模态确实对应更大的预测退化。

进一步地，附件三与附件四的样本带有实测缺失位置，可用于无需标注的机制检验：
若解释机制

作用度与实测缺失率的关系见图 27。

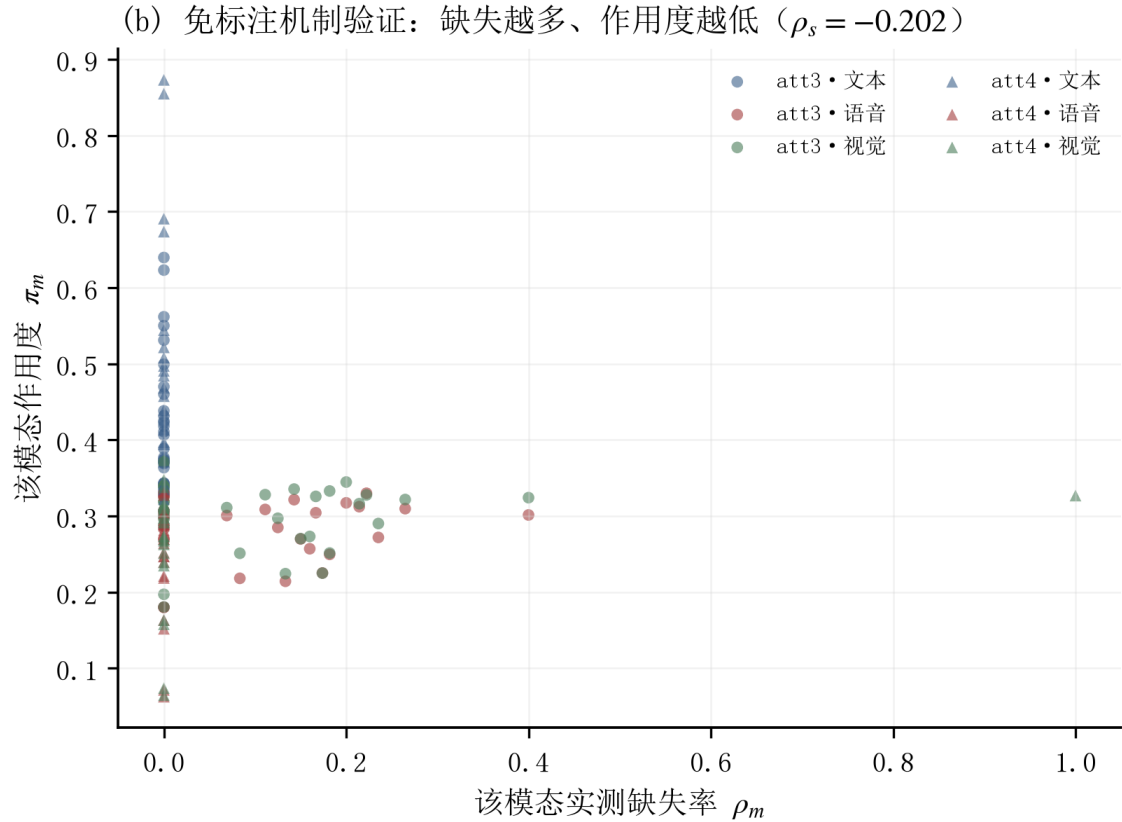


图 27 免标注机制验证。横轴为该模态的实测缺失率，纵轴为该模态的作用度，圆点与三角分别为附件三与附件四的样本。随缺失率上升作用度下降，说明模型在信息缺失时确实降低了对该模态的依赖，该检验不使用任何人工标注，可作为解释合理性的独立旁证；两套数据的点位分布一致，说明该趋势并非由单一测试集的构成所致。

成立，则缺失率越高的模态其作用度应越低。两套数据共 150 组模态-样本观测的秩相关系数为 -0.4011 ，且含缺失模态的平均作用度 (0.2898) 低于无缺失模态 (0.3610)，机制自洽。附件四中视觉模态整条缺失的样本 13 是极端案例：其视觉 Shapley 值仅为 -0.0002 ，近似为零，模型正确地没有依赖该模态。四种解释来源的定量对比如下：忠实性（删除 top-5 片段后预测的变化量）为注意力 0.0143 、

解释质量对比见图 28。

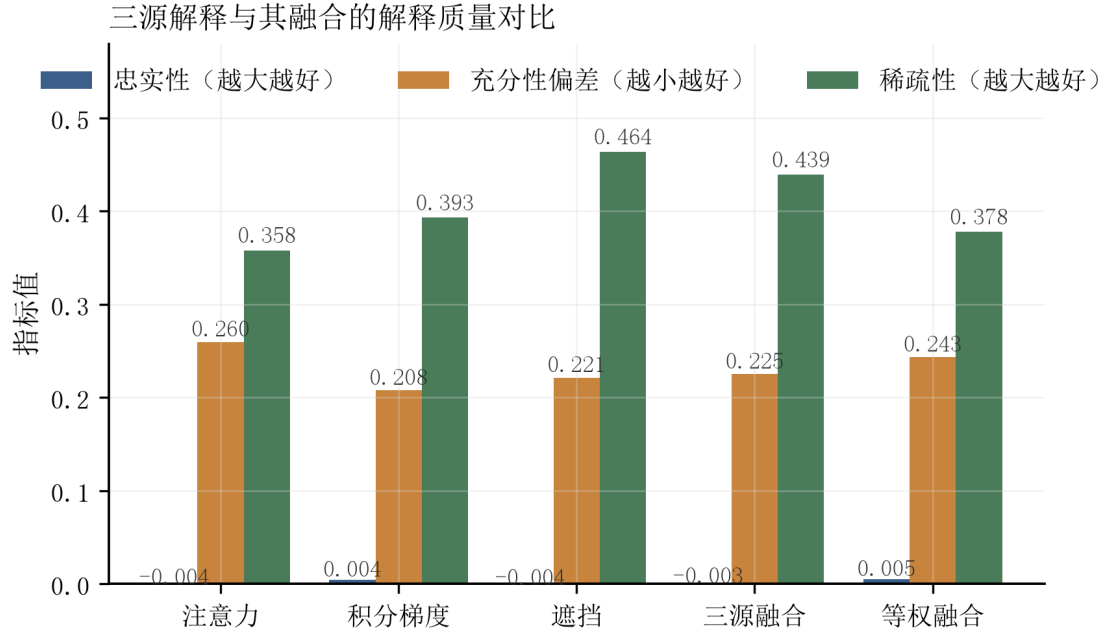


图 28 四种解释来源在附件四上的解释质量对比。忠实性为删除 **top-5** 片段后预测的变化量，充分性偏差为仅保留 **top-5** 片段时与全模态预测的差距，稀疏性为 **top-5** 片段的权重占比。注意力源（基线）在两项忠实性指标上均为最差，积分梯度与遮挡源更好，融合结果在忠实性与稀疏性之间取得折中。

四种来源的指标汇总见表 11。

表 11 解释质量指标的定量结果（附件四 20 条样本，均值）。忠实性为删除 **top-5** 片段后预测的符号均值与绝对变化量，充分性偏差为只保留 **top-5** 片段时与全模态预测差距的绝对值均值，稀疏性为 **top-5** 片段在有效位内的权重占比。注意力源的忠实性最低，与遮挡源的秩相关仅 **-0.0268**，说明在本模型上直接以注意力权重作为解释并不成立；本文因此以最终解释取三源融合结果。

解释来源	忠实性（符号）	忠实性（绝对）	充分性偏差	稀疏性
att	-0.0039	0.0143	0.2596	0.3580
fused	-0.0026	0.0146	0.2251	0.4392
fused_equal	+0.0051	0.0124	0.2434	0.3778
ig	+0.0044	0.0171	0.2076	0.3930
occ	-0.0044	0.0163	0.2213	0.4637

积分梯度 0.0171、遮挡 0.0163、融合 0.0146；充分性偏差（仅保留 **top-5** 片段）为注意力 0.2596、积分梯度 0.2076、遮挡 0.2213、融合 0.2251；稀疏性（**top-5** 片段的权重占比）为注意力 0.3580、积分梯度 0.3930、遮挡 0.4637、融合 0.4392。注意力源在充分性上明显最差，遮挡源最稀疏，融合结果在忠实性与稀疏性之间取得折中；其权重 $\gamma = (0.1, 0.1, 0.8)$ 由验证集按“与遮挡源一致性最大”标定，对应一致性为 0.959（等权方案更低）。片段级重要性的分布形态见图 29。融合解释的 **top-5** 片段平均承载 0.439 的

模态内重要度（验证集 200 条为 0.531），最高重要度片段的位置中位数为有效长度的 0.32，即证据集中在各模态有效区间的前中部；60 个模态-样本中有 59 个的 top-5 片段全部落在有效区间内，未出现把填充位当作证据的情形。积分梯度与遮挡两源在片段级的 top-5 平均仅重叠 1.5/5 个片段，说明时间定位本身存在来源依赖，这也印证了下面关于证据在时间上较分散的判断。

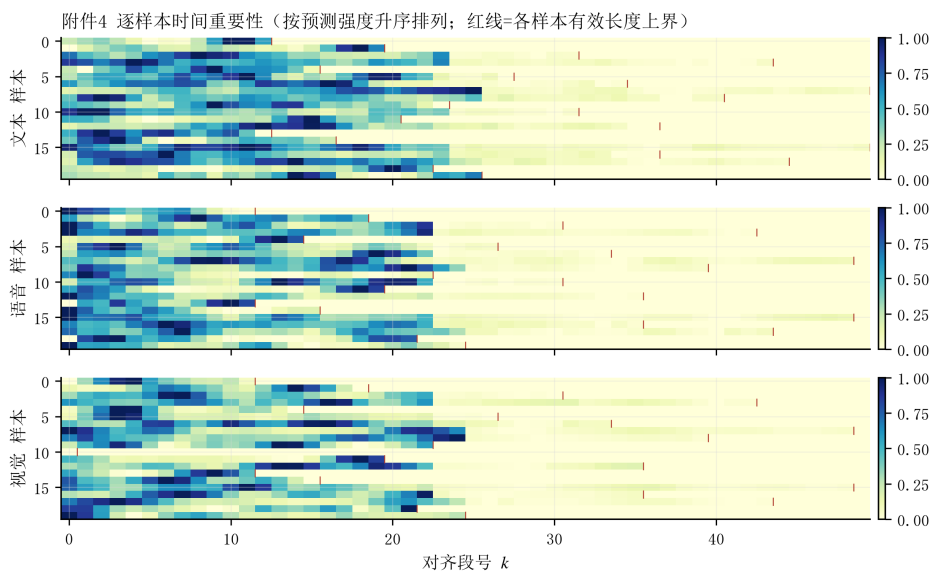


图 29 附件四全部 20 条样本的逐段重要性热力图，三行自上而下为文本、语音、视觉，每行一条样本，样本按模型预测强度升序排列；单元格颜色越深表示该段越关键，红色竖线为各样本的有效长度上界，其右侧为填充位且恒为零。高亮段集中在有效区间的前中部，文本行的高亮对比度明显强于语音与视觉，说明模型对文本的时间定位更集中、分辨率更高。

为进一步检验证据定位的可读性，本文对附件四 20 条样本逐条判读（判读表见支撑材料 Q3_ 证据人工判读 *.csv）：文本证据 15/20 承载该样本的情感线索（5 条中性样本为事实性描述，极性样本中 10 条含与预测方向一致的情感、否定或程度线索）；视觉关键帧 19/20 含可辨面部，1 条为房间全景（样本 13）；语音因无听音条件改以能量代理核查，9 条落在高能量或全片峰值段、9 条居中、2 条偏低。未命中集中在寒暄、开场与收尾类语句及远景画面，说明片段级定位在低信息段会退化为话题匹配，这也印证了“模态级结论为主、片段级为辅”的取舍，并指出后续改进方向。三类证据的判读条数汇总见表 12。

表 12 附件四证据定位的逐条判读汇总（20 条样本）。文本按“证据段落是否承载该样本的情感线索”判读，视觉按“关键帧是否含可辨面部”判读，语音因无听音条件改以能量代理核查（窗口能量分位或覆盖全片峰值即判合理）；表中条数可对照支撑材料的判读 CSV 逐条复核。

证据类型	合理		一般	未命中
文本（20 条）	10	5（中性样本的事实性描述）		5
视觉关键帧（20 条）	17	2		1
语音时段（20 条）	9	9		2

需要如实说明的是，上述时间尺度的忠实性指标数值都很小（绝对变化量约 0.0146，而预测强度的标准差为 1.0627），说明本模型的决策证据在时间维度上较为分散、单个片段的替换难以显著改变预测，这与问题二中“帧注意力熵 3.86 接近上限 3.91”的实测现象一致。因此本文把模态级解释作为主要的可追溯依据，片段级解释作为辅助定位手段，并在模型改进一节讨论增强时间局部性的方向。

5.2.19 附件四全量预测与解释结果

附件四 20 条样本的预测与解释汇总见表 13。
 汇总统计见图 30。

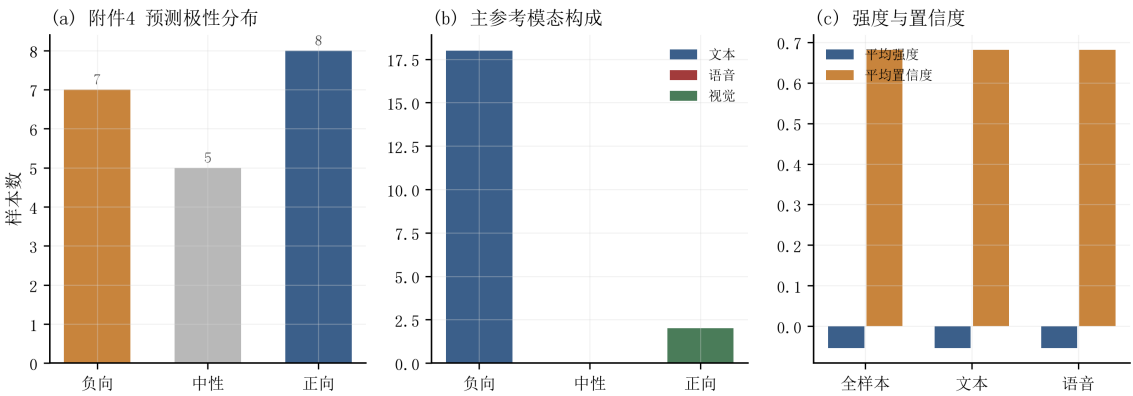


图 30 附件四二十条样本的预测与解释汇总：（a）预测极性分布；（b）主参考模态构成；（c）平均强度与平均置信度。该图与提交文件 `pred_explain_att4.csv` 同源，用于核对提交结果的统计口径：极性分布与主参考模态构成共同反映模型在弱强度样本上的保守倾向，样本越接近中性与单向之间，解释的主参考模态越易退化为文本；三组统计量均可由提交文件逐条复算。

表 13 附件四测试集的预测与解释汇总。极性取分类头主口径，模态作用度取式 (34) 的 **softmax** 口径。预测分布为负向 7 条、中性 5 条、正向 8 条；文本为主要参考模态 18 条。含缺失样本仅 1 条（样本 13 视觉模态整条缺失），其模态作用度趋于均匀、视觉 **Shapley** 值近似为零，是“信息缺失则不依赖”的直接例证。

指标	全样本	含缺失	无缺失
样本数	20	1	19
平均强度	-0.0540	+0.2921	-0.0722
平均绝对强度	+0.8060	+0.2921	+0.8331
平均置信度	+0.6814	+0.6379	+0.6837
两口径一致率	0.9000	—	—

提交文件 `pred_explain_att4.csv` 逐样本给出五类结果：情感极性（分类头主口径与阈值口径并列）、情感强度、主要参考模态、三模态作用度（ π_m 与两套归一化）以及关键证据定位（文本字符区间、语音秒区间、视觉关键帧编号与时刻），并附本样本解释的忠实性、充分性、稳定性与稀疏性指标。与预测文件的一致性在生成时做了断言校验：标签逐位一致、强度分数最大差异小于 10^{-3} 。

5.2.20 验证集基础性能与解释维度错误归因

问题三的骨干与问题二相同，故验证集基础性能与问题二一致：平均绝对误差 0.5866、皮尔逊相关 0.6543、分类准确率 0.6223、宏平均 F1 0.6157（详见表 4）。在此基础上，本文进一步从解释维度做错误归因：预测正确的 453 条样本其模态作用度的熵均值为

解释熵与置信度的对比见图 31。

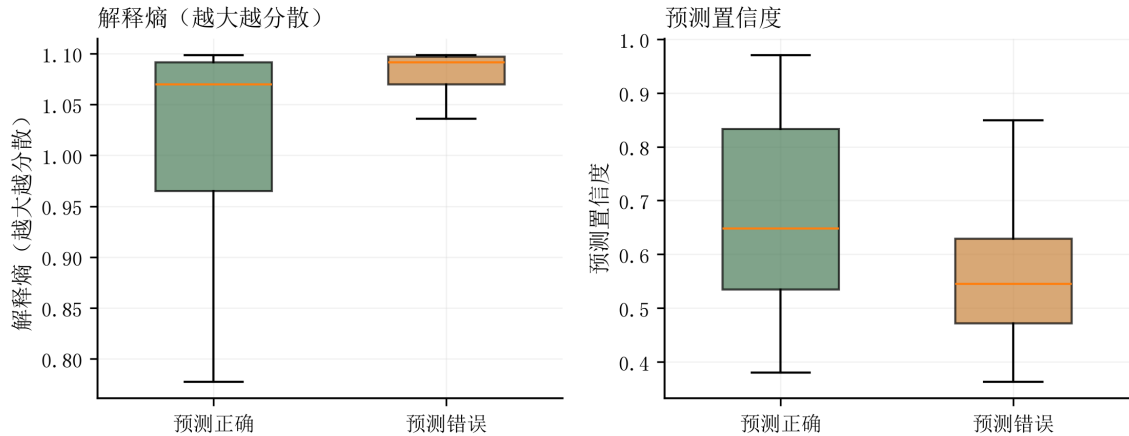


图 31 预测正确与错误样本的解释熵（左）与置信度（右）箱线图。错误样本的解释熵中位数高于正确样本，说明其模态依赖更分散、缺少明确的判断依据；置信度分布也整体左移，中位数差约 0.03。箱体上下缘为四分位数，须线延伸至 1.5 倍四分位距内的极值，离群点以散点单独标出；两组在解释熵上的差异比置信度更稳定。

解释维度的错误归因结果见表 14。

表 14 解释维度的错误归因（验证集 728 条）。解释熵为模态作用度分布的香农熵，熵越大表示模态依赖越分散、判断依据越不明确。错误样本的解释熵高于正确样本，而文本作用度与置信度均更低，说明模型在易错样本上依赖更分散，与问题二中弱强度样本分类准确率仅 0.5045 的结论相互支持。

分组	样本数	解释熵	$\pi_{\text{文本}}$	平均置信度
预测正确	453	1.0137	0.4891	0.6771
预测错误	275	1.0673	0.4173	0.5691

1.0137，预测错误的 275 条为 1.0673，即错误样本的模态依赖更分散、缺少明确的判断依据，这与问题二中“弱强度样本准确率仅 0.5045”的结论互相支持。

六、模型评价与改进

6.1 模型优点

本文模型的机理较为清楚。缺失掩码、填充掩码与共享-特有分解都有明确的数学定义，动态门控权重可解释为样本级模态重要性，掩码加权池化对应缺失条件下的信息聚合，均能说明“为什么有效”。评价口径的规范性是本文的突出之处：全部超参数、阈值与校准系数只在验证集上确定，缺失率采用全文统一定义并与提交文件逐位对齐，分

类主口径与阈值口径并列报告。本文还完整给出了从缺失注入到场景扫描、方差分析、错误归因与不确定性的证据链，使结论可复核。

6.2 模型缺点

中性类的识别精确率仅约 0.43，是三分类指标的主要短板。强度分数存在系统性收缩，负类与正类的平均偏差分别达到正 0.541 与负 0.447。消融实验中部分模块的增益小于种子间波动，说明其贡献尚需更多种子验证。含缺失样本在验证集中仅 48 条，相关结论的统计不确定性较大。

6.3 改进方向

针对中性边界问题，本文尝试了两条低成本路径并如实报告其无效：其一，用已有有序回归头的期望替代点回归，在验证集与测试划分上均未改善误差；其二，用不确定度与强度阈值对中性边界作后处理重判定，八个规则族中仅一个通过判据但仅改动一条样本，另一个在验证集上改善 0.0054 而在测试划分上反向恶化 0.0139。这两组负结果说明中性失效源自表征而非决策规则，后续应转向表征层改进，例如引入否定作用域感知的注意力机制与语用级语义建模。针对强度收缩，可考虑以带符号的序数分布直接参数化强度输出，从而同时给出分布信息与不确定度。

七、总结

本文以附件二为唯一数据来源，完成了三模态情感识别的特征提取与时序对齐、缺失鲁棒情感预测与可解释情感预测三项任务。核心贡献是建立了填充与缺失严格分离的口径体系，并在此基础上构造了掩码感知的鲁棒融合模型，通过 120 个缺失场景的网格扫描与三因素方差分析定量刻画了退化规律，指出退化由文本模态与缺失率的交互主导、头部缺失最为严重。模型在验证集取得 ACC 0.6223 与 Macro-F1 0.6158，在测试划分取得 0.6534 与 0.6338，并把整体退化相对朴素基线降低 1.5 至 3.3 倍。不确定性分析进一步揭示了回归不确定度与分类不确定度的可分离性，为选择性预测提供了依据。

参考文献

- [1] Zadeh A, Liang P P, Poria S, et al. Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph[C]. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, 2018: 2236–2246.

- [2] Tsai Y H H, Bai S, Liang P P, et al. Multimodal transformer for unaligned multimodal language sequences[C]. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019: 6558–6569.
- [3] Cao J, Miranda B, Wang X, et al. A deep multi-task learning model with multiscale feature fusion for multimodal sentiment analysis[J]. Information Fusion, 2023, 91: 232–243.
- [4] Cao W, Mirjalili V, Raschka S. Rank consistent ordinal regression for neural networks with application to age estimation[J]. Pattern Recognition Letters, 2020, 140: 325–331.
- [5] Hinton G, Vinyals O, Dean J. Distilling the knowledge in a neural network[J]. arXiv preprint arXiv:1503.02531, 2015.
- [6] Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]. Proceedings of the 26th Annual International Conference on Machine Learning, 2009: 41–48.
- [7] Geifman Y, El-Yaniv R. Selective classification for deep neural networks[C]. Advances in Neural Information Processing Systems, 2017: 4878–4887.
- [8] Kull M, Silva Filho T, Flach P. Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers[C]. Artificial Intelligence and Statistics, 2017: 623–631.
- [9] Zhang S, Yang Y, Chen C, et al. Deep learning-based multimodal emotion recognition from audio, visual, and text modalities: A systematic review[J]. Expert Systems with Applications, 2024, 237: 121692.
- [10] Qiu X, Fang Y, Zhou Q, et al. Beyond missing modalities: Hypergraph conditioned diffusion for uncertainty-aware multimodal emotion recognition[C]. CVPR, 2026.
- [11] Yang Z, Li M. Factorize, reconstruct, enhance: A unified framework for multimodal sentiment analysis[C]. CVPR, 2026.
- [12] Zhuang Y, Liu M, Bai W, et al. CMAD: Correlation-aware and modalities-aware distillation for multimodal sentiment analysis with missing modalities[C]. ICCV, 2025.
- [13] Zhu A, Hu M, Wang X, et al. Proxy-driven robust multimodal sentiment analysis with incomplete data[C]. ACL, 2025.
- [14] Mai S, Han S. Learning invariant modality representation for robust multimodal learning from a causal inference perspective[C]. ACL, 2026.

- [15] Wan J, Gong C, Fu G. Locate and explain: Joint multimodal emotion cause extraction and summarization in conversation[C]. ACL, 2026.
- [16] Mai S, Han S. CaReFlow: Cyclic adaptive rectified flow for multimodal fusion[C]. CVPR, 2026.
- [17] Fang Y, Huang W, Wan G, et al. EMOE: Modality-specific enhanced dynamic emotion experts[C]. CVPR, 2025.
- [18] 王楠, 王洪, 欧阳丹彤. 基于知识蒸馏与动态调整机制的多模态情感分析模型 [J]. 计算机学报, 2025, 48(8): 1923–1942.

A 支撑材料文件清单

本次提交的全部支撑材料按类别汇总于表 15，文件命名与论文正文、代码中的口径保持一致。

表 15 提交材料清单。按提交文件、模型参数、结果表、核心代码与运行说明五类列出，运行说明给出环境依赖与完整运行顺序。所有文件路径与论文正文、代码中的命名口径保持一致，模型参数仅保留最终集成的两个成员以控制附件体积，便于评审核对。

类别	文件	说明
提交文件	submission/pred_att3.csv	附件三 30 条预测结果
提交文件	submission/pred_att4.csv	附件四 20 条预测结果
提交文件	submission/pred_explain_att4.csv	附件四预测与解释结果（含模态作用度与证据定位）
提交文件	submission/explain_cards/*.json	逐样本解释卡（含 50 步三源与融合时间重要性）
提交文件	submission/evidence/*.jpg	视觉关键帧（用于回看原始视频）
模型参数	runs/ens_top2/student_s42.pt	集成成员一（种子 42）
模型参数	runs/ens_top2/student_s43.pt	集成成员二（种子 43）
模型参数	runs/q3/q3_evidence_head.pt	可解释性证据头（骨干冻结）
结果表	paper_tables/main_results_summary.csv	主结果多种子汇总
结果表	paper_tables/ablation_results.csv	消融实验汇总
结果表	paper_tables/missing_low_anova_summary.csv	方差分析效应量
结果表	paper_tables/error_attribution_groups.csv	错误归因分层
结果表	paper_tables/Q3_*.csv	问题三分层作用度、八子集反事实、模态忠实性与解释质量
核心代码	code/02_model/model.py	网络结构与门控
核心代码	code/02_model/losses.py	损失函数与指标
核心代码	code/02_model/train.py	训练与推理主流程
核心代码	code/03_xai/*.py	问题三三源解释与证据定位
运行说明	code/README.md	环境依赖与运行顺序

B 核心代码

本节给出实现上述模型的核心代码，与提交的支撑材料中的文件为同一份；代码以 Python 3.12 与 PyTorch 编写，运行顺序见 code/README.md。为便于与提交材料核对，每个代码块标注其在支撑材料中的相对路径与总行数。为避免中文字体下缺字形，代码中的少量 Unicode 数学符号（如 $\in$ 、 $\leq$ 、 $\rightarrow$ 、 π ）在附录中以 ASCII 等价形式给出（如 in、<=、->、pi），仅涉及注释、文档字符串与输出文案，不影响可执行语义；原始文件以提交的 code/ 目录为准。

2.1 问题二：MRF-Net 网络结构与动态专家门控

code/02_model/model.py（共 286 行）

```
1  -*- coding: utf-8 -*-
2  """
3  MRF-Net: Missing-Robust Fusion Network
```

```

4 掩码感知编码 -> 跨模态重建 -> 代理令牌门控 -> 共享/特有分解 -> 动态专家融合 -> 不确定性双头
5 """
6 import math
7
8 import torch
9 import torch.nn as nn
10 import torch.nn.functional as F
11
12 MODALITIES = ("text", "audio", "vision")
13
14
15 class PositionalEncoding(nn.Module):
16     def __init__(self, d, max_len=2048):
17         super().__init__()
18         pe = torch.zeros(max_len, d)
19         pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
20         n_even = pe[:, 0::2].shape[1]
21         div = torch.exp(torch.arange(0, n_even, dtype=torch.float) * (-math.log(10000.0) / d))
22         pe[:, 0::2] = torch.sin(pos * div)
23         pe[:, 1::2] = torch.cos(pos * div[:pe[:, 1::2].shape[1]])
24         self.register_buffer("pe", pe.unsqueeze(0))
25
26     def forward(self, x):
27         return x + self.pe[:, :x.size(1)]
28
29
30 class ModalityEncoder(nn.Module):
31     """掩码感知编码器：显式把“这一段是瞎的”喂给网络。"""
32
33     def __init__(self, d_in, hidden, nhead, layers, dropout, down=1):
34         super().__init__()
35         # 逐时间步线性投影（比大核卷积省5倍算力）+ 轻量局部混合
36         self.inp = nn.Sequential(nn.Linear(d_in + 2, hidden), nn.GELU())
37         self.local = nn.Conv1d(hidden, hidden, kernel_size=3, padding=1)
38         self.pos = PositionalEncoding(hidden)
39         layer = nn.TransformerEncoderLayer(
40             hidden, nhead, dim_feedforward=hidden * 4,
41             dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
42         self.enc = nn.TransformerEncoder(layer, num_layers=layers, enable_nested_tensor=False)
43         self.down = None
44         if down > 1:
45             self.down = nn.Conv1d(hidden, hidden, kernel_size=down, stride=down)
46
47     def forward(self, x, miss, pad):
48         m = miss.float().unsqueeze(-1) # 1=缺失（显式告知网络该段不可用）
49         v = 1.0 - m
50         h = self.inp(torch.cat([x * v, m, v], dim=-1))
51         h = h + F.gelu(self.local(h.transpose(1, 2))).transpose(1, 2)
52         h = self.enc(self.pos(h), src_key_padding_mask=pad)
53         return h # 原生序列长度
54
55     def downsample(self, h, miss, pad):
56         if self.down is None:
57             return h, miss, pad
58         h2 = self.down(h.transpose(1, 2)).transpose(1, 2)
59         k, s = self.down.kernel_size[0], self.down.stride[0]

```

```

60     md = F.max_pool1d(miss.unsqueeze(1).float(), k, s).squeeze(1).bool()
61     pd = F.max_pool1d(pad.unsqueeze(1).float(), k, s).squeeze(1).bool()
62     return h2, md, pd
63
64
65 class Expert(nn.Module):
66     """跨模态 + 跨时间自注意力专家。"""
67
68     def __init__(self, hidden, nhead, dropout):
69         super().__init__()
70         self.attn = nn.MultiheadAttention(hidden, nhead, dropout=dropout, batch_first=True)
71         self.norm1 = nn.LayerNorm(hidden)
72         self.norm2 = nn.LayerNorm(hidden)
73         self.ffn = nn.Sequential(
74             nn.Linear(hidden, hidden * 2), nn.GELU(), nn.Dropout(dropout),
75             nn.Linear(hidden * 2, hidden))
76
77     def forward(self, tok, pad):
78         a, _ = self.attn(tok, tok, tok, key_padding_mask=pad)
79         x = self.norm1(tok + a)
80         x = self.norm2(x + self.ffn(x))
81         return x
82
83
84 class MRFNet(nn.Module):
85     def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
86                 dropout=0.1, downs=None, target_len=50, mag_levels=0,
87                 ms_pool=False, ms_windows=(5, 25, 50), use_pol=False):
88         super().__init__()
89         downs = downs or {"text": 1, "audio": 1, "vision": 1}
90         self.dims = dims
91         self.hidden = hidden
92         self.target_len = target_len
93         self.mag_levels = int(mag_levels)
94         self.ms_pool = bool(ms_pool)
95         self.ms_windows = tuple(int(w) for w in ms_windows)
96         self.use_pol = bool(use_pol)
97         self.mods = list(MODALITIES)
98
99         self.enc = nn.ModuleDict({
100             m: ModalityEncoder(dims[m], hidden, nhead, enc_layers, dropout, downs[m])
101             for m in self.mods})
102
103         # —— 跨模态重建（缺失位置靠“造”） ——
104         self.cross = nn.ModuleDict({m: nn.MultiheadAttention(hidden, nhead, dropout=dropout, batch_first=True)
105                                     for m in self.mods})
106
107         self.dec = nn.ModuleDict({m: nn.Linear(hidden, dims[m]) for m in self.mods})
108
109         # —— 代理令牌 ——
110         self.proxy = nn.ParameterDict({m: nn.Parameter(torch.randn(hidden) * 0.02) for m in self.mods})
111         self.beta = nn.ModuleDict({m: nn.Linear(hidden + 1, 1) for m in self.mods})
112
113         # —— 共享 / 特有分解 ——
114         self.share = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
115         self.spec = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
116         self.mix = nn.ModuleDict({m: nn.Linear(hidden * 2, hidden) for m in self.mods})

```

```

115
116     # —— 动态专家融合 ——
117     self.mod_emb = nn.Embedding(3, hidden)
118     self.experts = nn.ModuleList([Expert(hidden, nhead, dropout) for _ in range(n_experts)])
119     self.gate = nn.Sequential(nn.Linear(hidden + 7, hidden), nn.GELU(), nn.Linear(hidden, n_experts))
120     self.pool_score = nn.Sequential(nn.Linear(hidden, hidden // 2), nn.Tanh(), nn.Linear(hidden // 2, 1))
121
122     # —— 输出头 ——
123     self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 1))
124     self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 1))
125     self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 3))
126
127     # —— 强度分级辅助头（CORAL 有序回归：K-1 个累计 logit） ——
128     # 动机（实测）：61.7% 的样本  $|y| \leq 2/3$ （弱标注）而 ACC 仅 0.486； $|y|=1/3$  层
129     # ACC 更低至 0.361 —— 模型分不清“完全中性”与“轻微极性”。该头把  $|y|$  离散为
130     # K 个等级（步长 1/3）做有序回归，强制模型在零点附近建立分辨率。
131     if self.mag_levels > 1:
132         self.mag_w = nn.Linear(hidden, 1, bias=False)
133         self.mag_b = nn.Parameter(torch.linspace(-1.5, 1.5, self.mag_levels - 1))
134
135     # —— 多尺度时间池化（补足单层注意力池化“过于均匀”的问题） ——
136     # 实测：帧注意力熵 3.86 / 上限 3.91  $\approx$  均匀，说明注意力没聚焦到关键片段。
137     # 改为“多窗口掩码均值池化 + 线性投影”，并与注意力池化残差相加。
138     if self.ms_pool:
139         T = 3 * int(target_len)          # 三模态拼接后的 token 长度
140         nseg = sum(max(1, T // w) for w in self.ms_windows)
141         self._ms_nseg = nseg
142         self._ms_T = T
143         self.ms_proj = nn.Linear(nseg * hidden, hidden)
144
145     # —— 二分类极性辅助头（对齐文献 Acc-2 的做法） ——
146     if self.use_pol:
147         self.head_pol = nn.Linear(hidden, 1)
148
149     # -----
150     def _masked_win_pool(self, h, pad):
151         """多窗口掩码均值池化：把时间轴切成若干段，段内对非填充位置求均值。
152
153         h:(B,T,H) pad:(B,T) bool(True=填充) 返回 (B, nseg*H)
154         """
155         h = h.masked_fill(pad.unsqueeze(-1), 0.0)
156         valid = (~pad).float().unsqueeze(-1)          # (B,T,1)
157         T = h.size(1)
158         outs = []
159         for w in self.ms_windows:
160             nb = max(1, T // w)
161             seg = torch.arange(T, device=h.device) * nb // T   # 把 T 均分到 nb 段
162             for b in range(nb):
163                 m = (seg == b).float().unsqueeze(0).unsqueeze(-1)   # (1,T,1)
164                 s = (h * m * valid).sum(dim=1)
165                 n = (m * valid).sum(dim=1).clamp(min=1.0)
166                 outs.append(s / n)
167         return torch.cat(outs, dim=-1)
168
169     @staticmethod
170     def _availability(rho):

```

```

171     """rho:(B,3) 缺失率 -> (B,7) 可用性特征。"""
172     n_vis = (rho < 0.999).float().sum(dim=1, keepdim=True)
173     return torch.cat([rho, (rho < 0.999).float(), n_vis / 3.0], dim=1)
174
175 def forward(self, x, miss, pad, return_aux=False):
176     """x/miss/pad: dict 模态 -> (B,L,d)/(B,L)/(B,L)"""
177     h, rec, share, spec = {}, {}, {}, {}
178     for m in self.mods:
179         h[m] = self.enc[m](x[m], miss[m], pad[m])
180
181     # —— 1) 跨模态重建 (原生长度上计算, 损失只取缺失位置) ——
182     for m in self.mods:
183         others = [h[o] for o in self.mods if o != m]
184         kv = torch.cat(others, dim=1)
185         kv_pad = torch.cat([pad[o] for o in self.mods if o != m], dim=1)
186         a, _ = self.cross[m](h[m], kv, kv, key_padding_mask=kv_pad)
187         h_hat = h[m] + a * miss[m].float().unsqueeze(-1)
188         rec[m] = self.dec[m](h_hat)
189
190     # —— 2) 降采样到 target_len 统一时间尺度 ——
191     hd, md, pd_ = {}, {}, {}
192     for m in self.mods:
193         hd[m], md[m], pd_[m] = self.enc[m].downsample(h[m], miss[m], pad[m])
194
195     # —— 3) 代理令牌门控 + 分解 ——
196     rho = torch.stack([md[m].float().mean(dim=1) for m in self.mods], dim=1) # (B,3)
197     beta_val = {}
198     for m in self.mods:
199         hm = hd[m]
200         inp = torch.cat([hm.mean(dim=1), rho[:, self.mods.index(m)].unsqueeze(-1)], dim=-1)
201         b = torch.sigmoid(self.beta[m](inp)).unsqueeze(1) # (B,1,1)
202         beta_val[m] = b.detach().view(-1)
203         hm = (1 - b) * hm + b * self.proxy[m].view(1, 1, -1)
204         share[m] = self.share[m](hm)
205         spec[m] = self.spec[m](hm)
206         hd[m] = self.mix[m](torch.cat([share[m], spec[m]], dim=-1))
207
208     # —— 4) 动态专家融合 ——
209     tokens = torch.cat([hd[m] for m in self.mods], dim=1)
210     tok_pad = torch.cat([pd_[m] for m in self.mods], dim=1)
211     mid = torch.arange(3, device=tokens.device).repeat_interleave(tokens.size(1) // 3)
212     tokens = tokens + self.mod_emb[mid].unsqueeze(0)
213
214     av = self._availability(rho)
215     cls_pool = tokens.mean(dim=1)
216     g = torch.softmax(self.gate(torch.cat([cls_pool, av], dim=-1)), dim=-1) # (B,E)
217
218     outs = torch.stack([e(tokens, tok_pad) for e in self.experts], dim=1) # (B,E,T,H)
219     fused = (g.unsqueeze(-1).unsqueeze(-1) * outs).sum(dim=1) # (B,T,H)
220
221     score = self.pool_score(fused).masked_fill(tok_pad.unsqueeze(-1), -1e9)
222     alpha = torch.softmax(score, dim=1) # (B,T,1)
223     z = (alpha * fused).sum(dim=1)
224     if self.ms_pool:
225         pooled = self._masked_win_pool(fused, tok_pad)
226         z = z + self.ms_proj(pooled)

```

```

227
228     mu = self.head_mu(z).squeeze(-1)
229     logvar = self.head_lv(z).squeeze(-1).clamp(-6.0, 4.0)
230     logits = self.head_cls(z)
231
232     if not return_aux:
233         return mu, logvar, logits
234     aux = dict(rec=rec, share=share, spec=spec, gate=g, alpha=alpha,
235               rho=rho, hd=hd, pad_ds=pd_, miss_ds=md, z=z, mu=mu, logits=logits,
236               beta=beta_val)
237     if self.mag_levels > 1:
238         aux["logits_mag"] = self.mag_w(z) + self.mag_b.view(1, -1)
239     if self.use_pol:
240         aux["pol"] = self.head_pol(z).squeeze(-1)
241     return mu, logvar, logits, aux
242
243
244 class PlainFusion(nn.Module):
245     """A0/A1 基线: **朴素拼接 + MLP** 融合（无掩码感知编码/重建/代理令牌/共享-特有分解/动态专家）。
246
247     用于论文消融中的“常规固定权重融合模型”对照：
248     * “use_indicator=False” -> **A0**（完全不做缺失处理的常规模型）
249     * “use_indicator=True” -> **A1**（仅额外告知“哪段缺失”）
250
251     接口与 :class:`MRFNet` 完全一致（“forward” 返回 “mu/logvar/logits+[aux]”），
252     因此可直接复用 train/runtime/infer 的全部代码。
253     """
254
255     def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
256                 dropout=0.1, downs=None, target_len=50, use_indicator=True):
257         super().__init__()
258         self.mods = list(MODALITIES)
259         self.use_indicator = bool(use_indicator)
260         d_in = sum(int(dims[m]) for m in self.mods)
261         if self.use_indicator:
262             d_in += 2 * len(self.mods) # 每模态: 缺失率 + 可见率
263         self.net = nn.Sequential(
264             nn.Linear(d_in, hidden), nn.GELU(), nn.Dropout(dropout),
265             nn.Linear(hidden, hidden), nn.GELU())
266         self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 1))
267         self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 1))
268         self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, 3))
269
270     def forward(self, x, miss, pad, return_aux=False):
271         feats = []
272         for m in self.mods:
273             v = (~pad[m]).float().unsqueeze(-1) # (B,L,1)
274             pooled = (x[m] * v).sum(dim=1) / v.sum(dim=1).clamp(min=1.0) # 有效位均值池化
275             feats.append(pooled)
276             if self.use_indicator:
277                 mval = (miss[m] & ~pad[m]).float().mean(dim=1, keepdim=True)
278                 feats += [mval, 1.0 - mval]
279         h = self.net(torch.cat(feats, dim=-1))
280         mu = self.head_mu(h).squeeze(-1)
281         logvar = self.head_lv(h).squeeze(-1).clamp(-6.0, 4.0)
282         logits = self.head_cls(h)

```

```

283         if not return_aux:
284             return mu, logvar, logits
285         return mu, logvar, logits, dict(z=h)

```

2.2 问题二：损失函数（异方差回归、类别加权、一致性、重建与蒸馏）

code/02_model/losses.py（共 376 行）

```

1  -*- coding: utf-8 -*-
2  """损失函数与评价指标。"""
3  import numpy as np
4  import torch
5  import torch.nn.functional as F
6
7  MODALITIES = ("text", "audio", "vision")
8
9
10 # -----
11 # 损失
12 # -----
13 def reg_nll(mu, logvar, y):
14     """高斯负对数似然（仅作不确定性辅助项）。"""
15     return 0.5 * (torch.exp(-logvar) * (y - mu) ** 2 + logvar).mean()
16
17
18 def reg_main(mu, y, beta=0.5):
19     """主回归损失: SmoothL1, 与官方评价指标 MAE 对齐。"""
20     return F.smooth_l1_loss(mu, y, beta=beta)
21
22
23 def cls_ce(logits, y, weight=None, label_smoothing=0.0):
24     return F.cross_entropy(logits, y, weight=weight, label_smoothing=label_smoothing)
25
26
27 # -----
28 # D 系列: 针对“弱标注样本”(|y| 小 / 中性类) 的标签分布与有序损失
29 # 实测（附件2）: |y| 以 1/3 为步长、最大 3.0, 且 类 = sign(均值强度)。
30 # 因此 |y| 本身就是“标注一致性”的直接度量: |y|=1/3 相当于标注者 2:1 分裂。
31 # 三类混淆中 84% 只是邻类错, 说明需要**序信息**而不是更多容量。
32 # -----
33 def intensity_soft_target(y_reg, tau=0.3):
34     """用连续强度 |y| 构造三分类**软目标**（标签分布 / 置信度感知平滑）。
35
36     q_极性 = tanh(|y|/tau), q_中性 = 1 - q_极性, y=0 时 q = 纯中性
37
38     tau 越小越接近硬标签; tau->inf 退化为“只区分中性/非中性”。
39     """
40     y = y_reg
41     conf = torch.tanh(torch.abs(y) / max(float(tau), 1e-6)).clamp(0.0, 1.0)
42     q = torch.zeros(y.shape[0], 3, dtype=y.dtype, device=y.device)
43     pos = y > 0
44     neg = y < 0
45     q[:, 1] = 1.0 - conf
46     q[pos, 2] = conf[pos]

```

```

47     q[neg, 0] = conf[neg]
48     q[~(pos | neg), 1] = 1.0
49     return q / q.sum(dim=-1, keepdim=True).clamp(min=1e-8)
50
51
52 def emd2_loss(logits, q):
53     """EMD^2 (有序) 惩罚: 累计分布的平方差, 使”跨类错误”代价 > ”邻类错误”。"""
54     p = torch.softmax(logits, dim=-1)
55     return ((torch.cumsum(p, dim=-1) - torch.cumsum(q, dim=-1)) ** 2).sum(-1).mean()
56
57
58 def neu_suppress_loss(logits, y_cls, lam=0.0):
59     """中性抑制正则 (代价敏感学习): 惩罚”真值为极性、却把概率压在 neu 上”。
60
61     实测误差结构: valid 上 299 个错误里 **167 个 (56%) 是把极性样本判成中性**
62     (预测为 neu 的 269 个样本里只有 102 个正确, neu precision 仅 0.36)。
63     该正则直接把 p_neu 从极性样本上挤走, 把”中性<->极性”错误转化为”邻类极性”错误。
64     """
65     if not lam or lam <= 0:
66         return None
67     p = torch.softmax(logits, dim=-1)
68     m = y_cls != 1
69     if m.sum() < 1:
70         return None
71     return p[m, 1].mean()
72
73
74 # -----
75 # D4: 强度分级辅助头 (CORAL 有序回归)
76 # 动机 (实测): 61.7% 的样本 |y|<=2/3, 其 ACC 仅 0.486; |y|=1/3 层更低到 0.361。
77 # 误差 84% 是邻类错 —— 模型分不清”完全中性”与”轻微极性”。把 |y| 按 1/3 步长
78 # 离散为 K 个等级做有序回归, 等于直接监督”强度分辨率”, 而不是让分类头自己猜。
79 # -----
80 def mag_level(y_reg, n_levels=10):
81     """|y| -> 等级 k in [0, n_levels-1] (|y| 的离散档位, 步长 1/3 -> n_levels=10)。"""
82     k = torch.round(torch.abs(y_reg) * 3.0).long()
83     return k.clamp(0, int(n_levels) - 1)
84
85
86 def coral_loss(logits_mag, y_reg, n_levels=10):
87     """CORAL: 共享投影 + 有序偏置, 对 K-1 个累计事件 P(k > j) 做二分类。
88
89     logits_mag: (B, n_levels-1)
90     """
91     k = mag_level(y_reg, n_levels)
92     levels = torch.arange(logits_mag.size(1), device=k.device).unsqueeze(0)
93     target = (k.unsqueeze(1) > levels).float()
94     return F.binary_cross_entropy_with_logits(logits_mag, target)
95
96
97 def mag_consistency(mu, logits_mag, n_levels=10):
98     """辅助一致性: 累计概率得到的期望等级应与 |mu| 对得上 (弱监督, 可选)。"""
99     p = torch.sigmoid(logits_mag) # P(k > j)
100     exp_k = p.sum(dim=1) # E[k] = Sigma_j P(k > j)
101     return F.smooth_l1_loss(exp_k / 3.0, torch.abs(mu).detach(), beta=0.2)
102

```

```

103
104 def cls_D_loss(logits, y_cls, y_reg, cls_weight=None, label_smoothing=0.0,
105               soft_tau=0.0, soft_mix=0.0, lam_emd=0.0, weak_alpha=0.0,
106               lam_neusup=0.0):
107     """D 系列合成分类损失（针对弱标注样本 / 中性类）。
108
109     D2 soft_mix>0：硬 CE 与强度软目标 KL 的凸组合
110     D2 lam_emd>0：有序 EMD^2 项（内建序结构；实测 84% 错误只是邻类错）
111     D3 weak_alpha>0：按  $w = 1 + \alpha \cdot (1 - |y|/3)$  给**弱标注样本**加权
112     D3' lam_neusup>0：中性抑制正则（代价敏感，专治“极性样本被判成中性”）
113     """
114     hard = F.cross_entropy(logits, y_cls, weight=cls_weight,
115                             reduction="none", label_smoothing=label_smoothing)
116     q = None
117     if soft_mix and soft_mix > 0:
118         q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
119         soft = -(q * F.log_softmax(logits, dim=-1)).sum(-1)
120         per = (1.0 - float(soft_mix)) * hard + float(soft_mix) * soft
121     else:
122         per = hard
123     if weak_alpha and weak_alpha > 0:
124         w = 1.0 + float(weak_alpha) * (1.0 - (torch.abs(y_reg) / 3.0).clamp(0, 1))
125         per = per * w
126     loss = per.mean()
127     if lam_emd and lam_emd > 0:
128         if q is None:
129             q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
130             loss = loss + float(lam_emd) * emd2_loss(logits, q)
131     if lam_neusup and lam_neusup > 0:
132         ls = neu_suppress_loss(logits, y_cls, lam_neusup)
133         if ls is not None:
134             loss = loss + float(lam_neusup) * ls
135     return loss
136
137
138
139 def consistency_loss(mu, logits):
140     """极性-强度一致性。
141
142     实验事实：附件2的 sign(强度) 与三分类标签一致率为 **1.0000**（n=4850），
143     即“极性 == 强度符号”是硬约束。因此  $\tanh(\mu)$  与分类头的期望极性  $E[\text{cls}] = P(\text{pos}) - P(\text{neg})$ 
144     应当一致；该正则把回归头与分类头对齐。
145     """
146     p = torch.softmax(logits, dim=-1)
147     e = p[:, 2] - p[:, 0] # in[-1,1]
148     return F.mse_loss(torch.tanh(mu), e)
149
150
151 def weighted_smooth_l1(mu, y, beta=0.5, alpha=0.0, gamma=0.5):
152     """按 |y| 加权的 SmoothL1：把训练重心移向误差主体（强情感样本）。
153
154     alpha=0 时退化为普通 SmoothL1（与 MAE 目标对齐）。
155     """
156     loss = F.smooth_l1_loss(mu, y, beta=beta, reduction="none")
157     if alpha and alpha > 0:
158         w = 1.0 + float(alpha) * torch.abs(y).pow(float(gamma))

```

```

159     loss = loss * w
160     return loss.mean()
161
162
163 def masked_l1(pred, target, sel):
164     """sel:(B,L) bool, 只对选中位置求 L1。"""
165     if sel.sum() < 1:
166         return pred.new_zeros(())
167     p = pred[sel]
168     t = target[sel]
169     return (p - t).abs().mean()
170
171
172 def recon_loss(rec, x, miss, pad, use_missing=True):
173     """重建损失: use_missing=True 时只算缺失位置, 否则只算可见位置。"""
174     total, n = rec[list(rec.keys())[0]].new_zeros((), 0)
175     for m in MODALITIES:
176         sel = (miss[m] & ~pad[m]) if use_missing else (~miss[m] & ~pad[m])
177         if sel.sum() < 1:
178             continue
179         total = total + masked_l1(rec[m], x[m], sel)
180         n += 1
181     return total / max(n, 1)
182
183
184 def masked_mean(h, valid):
185     w = valid.float().unsqueeze(-1)
186     return (h * w).sum(dim=1) / w.sum(dim=1).clamp(min=1.0)
187
188
189 def sim_loss(share, pad):
190     """共享表示相似性: 把各模态"模态不变"部分拉到一起。"""
191     pooled = {m: masked_mean(share[m], ~pad[m]) for m in MODALITIES}
192     loss, n = pooled["text"].new_zeros((), 0)
193     keys = list(pooled.keys())
194     for i in range(len(keys)):
195         for j in range(i + 1, len(keys)):
196             loss = loss + F.mse_loss(pooled[keys[i]], pooled[keys[j]])
197             n += 1
198     return loss / max(n, 1)
199
200
201 def diff_loss(share, spec, pad):
202     """共享/特有正交: 用余弦平方, 避免尺度影响。"""
203     loss, n = None, 0
204     for m in MODALITIES:
205         a, b = share[m], spec[m]
206         cos = F.cosine_similarity(a, b, dim=-1)
207         w = (~pad[m]).float()
208         v = ((cos ** 2) * w).sum() / w.sum().clamp(min=1.0)
209         loss = v if loss is None else loss + v
210         n += 1
211     return loss / max(n, 1)
212
213
214 def balance_loss(gate):

```

```

215 """专家负载均衡（熵形式），防止专家坍塌。"""
216 gbar = gate.mean(dim=0)
217 E = gate.size(1)
218 return -(gbar * torch.log(gbar + 1e-8)).sum() * E
219
220
221 def total_loss(out, batch, cfg, lam_kd=0.0, teacher_out=None):
222     """组装总损失；返回 (loss, 明细 dict)。"""
223     mu, logvar, logits = out["mu"], out["logvar"], out["logits"]
224     x, miss, pad = batch["x"], batch["miss"], batch["pad"]
225
226     l_reg_main = weighted_smooth_l1(mu, batch["y_reg"], beta=getattr(cfg, "reg_beta", 0.5),
227                                     alpha=getattr(cfg, "y_weight_alpha", 0.0),
228                                     gamma=getattr(cfg, "y_weight_gamma", 0.5))
229     l_reg_var = reg_nll(mu, logvar, batch["y_reg"])
230     l_reg = l_reg_main + getattr(cfg, "lam_var", 0.1) * l_reg_var
231     cw = getattr(cfg, "cls_weight", None)
232     if cw is not None:
233         cw = torch.tensor(cw, dtype=logits.dtype, device=logits.device)
234         soft_mix = float(getattr(cfg, "soft_mix", 0.0) or 0.0)
235         lam_emd = float(getattr(cfg, "lam_emd", 0.0) or 0.0)
236         weak_alpha = float(getattr(cfg, "weak_alpha", 0.0) or 0.0)
237         lam_neusup = float(getattr(cfg, "lam_neusup", 0.0) or 0.0)
238         if soft_mix > 0 or lam_emd > 0 or weak_alpha > 0 or lam_neusup > 0:
239             l_cls = cls_D_loss(logits, batch["y_cls"], batch["y_reg"], cw,
240                                label_smoothing=getattr(cfg, "label_smoothing", 0.0),
241                                soft_tau=float(getattr(cfg, "soft_tau", 0.0) or 0.0),
242                                soft_mix=soft_mix, lam_emd=lam_emd,
243                                weak_alpha=weak_alpha, lam_neusup=lam_neusup)
244         else:
245             l_cls = cls_ce(logits, batch["y_cls"], cw, label_smoothing=getattr(cfg, "label_smoothing", 0.0))
246     zero = l_reg.new_zeros(())
247     has = out.__contains__
248     # D4: 强度分级辅助头（CORAL 有序回归）
249     lam_mag = float(getattr(cfg, "lam_mag", 0.0) or 0.0)
250     l_mag = (coral_loss(out["logits_mag"], batch["y_reg"],
251                        n_levels=int(getattr(cfg, "mag_levels", 10) or 10))
252              if (lam_mag > 0 and has("logits_mag")) else zero)
253     # 二分类极性辅助头：直接监督“是否负面”，与 Acc-2 的判定目标一致
254     lam_pol = float(getattr(cfg, "lam_pol", 0.0) or 0.0)
255     l_pol = zero
256     if lam_pol > 0 and has("pol"):
257         tneg = (batch["y_reg"] < 0).float()
258         pw = ((1.0 - tneg).sum() / tneg.sum().clamp(min=1.0)).detach()
259         l_pol = F.binary_cross_entropy_with_logits(out["pol"], tneg,
260                                                    pos_weight=pw)
261     # 模块可缺省：A0/A1 朴素基线没有重建/分解/专家，对应项自动跳过（置 0）
262     l_rec = recon_loss(out["rec"], x, miss, pad, use_missing=True) if has("rec") else zero
263     l_rec0 = recon_loss(out["rec"], x, miss, pad, use_missing=False) if has("rec") else zero
264     l_sim = (sim_loss(out["share"], out["pad_ds"]))
265             if (has("share") and has("pad_ds")) else zero)
266     l_diff = (diff_loss(out["share"], out["spec"], out["pad_ds"]))
267             if (has("share") and has("spec") and has("pad_ds")) else zero)
268     l_bal = balance_loss(out["gate"]) if has("gate") else zero
269     lam_cons = float(getattr(cfg, "lam_cons", 0.0))
270     l_cons = consistency_loss(mu, logits) if lam_cons > 0 else zero

```

```

271
272     loss = (l_reg + cfg.lam_cls * l_cls + cfg.lam_rec * l_rec + cfg.lam_rec0 * l_rec0
273             + cfg.lam_sim * l_sim + cfg.lam_diff * l_diff + cfg.lam_bal * l_bal
274             + lam_cons * l_cons + lam_mag * l_mag + lam_pol * l_pol)
275
276     detail = dict(reg=float(l_reg.detach()), reg_main=float(l_reg_main.detach()),
277                  reg_var=float(l_reg_var.detach()), cls=float(l_cls.detach()),
278                  cons=float(l_cons.detach()), mag=float(l_mag.detach()),
279                  pol=float(l_pol.detach()),
280                  rec=float(l_rec.detach()),
281                  rec0=float(l_rec0.detach()), sim=float(l_sim.detach()),
282                  diff=float(l_diff.detach()), bal=float(l_bal.detach()))
283
284     if teacher_out is not None and lam_kd > 0:
285         tau = cfg.kd_tau
286         l_kd_logit = F.kl_div(F.log_softmax(logits / tau, dim=-1),
287                              F.softmax(teacher_out["logits"] / tau, dim=-1),
288                              reduction="batchmean") * tau * tau
289         l_kd_reg = F.mse_loss(mu, teacher_out["mu"])
290         # 关系蒸馏：样本间相似度矩阵（CMAD 相关性感知的简化实现）
291         st = F.normalize(out["z"], dim=-1)
292         tt = F.normalize(teacher_out["z"], dim=-1)
293         sim_s, sim_t = st @ st.t(), tt @ tt.t()
294         l_kd_rel = F.mse_loss(sim_s, sim_t)
295         l_kd = l_kd_logit + l_kd_reg + 0.1 * l_kd_rel      # 关系蒸馏降权，避免量级压过任务损失
296         loss = loss + lam_kd * l_kd
297         detail.update(kd=float(l_kd), kd_logit=float(l_kd_logit),
298                      kd_reg=float(l_kd_reg), kd_rel=float(l_kd_rel))
299     return loss, detail
300
301
302 # -----
303 # 指标
304 # -----
305 def _to_np(t):
306     return t.detach().cpu().numpy() if torch.is_tensor(t) else np.asarray(t)
307
308
309 def polar_from_score(score, theta):
310     """连续强度->三分类（题目定义：<0 负, =0 中, >0 正; theta 容差在 valid 上标定）。"""
311     out = np.ones_like(score, dtype=np.int64)
312     out[score < -theta] = 0
313     out[score > theta] = 2
314     return out
315
316
317 def tune_threshold(score, label_cls, grid=None):
318     """在验证集上按 Macro-F1 选择决策阈值。"""
319     from sklearn.metrics import f1_score
320     if grid is None:
321         grid = np.arange(0.0, 2.01, 0.05)
322     best, best_theta = -1.0, 0.0
323     for th in grid:
324         pred = polar_from_score(score, th)
325         f1 = f1_score(label_cls, pred, average="macro", zero_division=0)
326         if f1 > best:

```

```

327         best, best_theta = f1, float(th)
328     return best_theta, best
329
330
331 def compute_metrics(y_true_reg, y_pred_reg, y_true_cls, theta=0.0, y_pred_cls=None):
332     from sklearn.metrics import accuracy_score, f1_score
333     from scipy.stats import pearsonr
334     yt = _to_np(y_true_reg).astype(np.float64)
335     yp = _to_np(y_pred_reg).astype(np.float64)
336     tc = _to_np(y_true_cls).astype(np.int64)
337
338     mae = float(np.abs(yt - yp).mean())
339     try:
340         corr = float(pearsonr(yt, yp)[0])
341     except Exception:
342         corr = float("nan")
343     # CCC
344     mt, mp = yt.mean(), yp.mean()
345     vt, vp = yt.var(), yp.var()
346     cov = ((yt - mt) * (yp - mp)).mean()
347     ccc = float(2 * cov / (vt + vp + (mt - mp) ** 2 + 1e-12))
348
349     # 极性判别有两套口径:
350     # (A) theta 阈值口径: |y_hat| <= theta 判为 Neutral (theta 在 valid 上按 Macro-F1 选出)
351     # —— 与提交 CSV 的标签完全一致, 故作为**主指标**
352     # (B) 分类头 argmax 口径: 直接取分类头三类的最大值
353     pc_th = polar_from_score(yp, theta).astype(np.int64)
354     res = dict(mae=mae, pearson=corr, ccc=ccc,
355               acc=float(accuracy_score(tc, pc_th)),
356               f1=float(f1_score(tc, pc_th, average="macro", zero_division=0)),
357               theta=float(theta))
358     f1s = f1_score(tc, pc_th, average=None, zero_division=0, labels=[0, 1, 2])
359     res["f1_neg"], res["f1_neu"], res["f1_pos"] = [float(x) for x in f1s]
360     if y_pred_cls is not None:
361         pc_h = _to_np(y_pred_cls).astype(np.int64)
362         res["acc_head"] = float(accuracy_score(tc, pc_h))
363         res["f1_head"] = float(f1_score(tc, pc_h, average="macro", zero_division=0))
364     res["n"] = int(len(yt))
365     return res
366
367
368 def format_metrics(res, tag=""):
369     s = ("%s MAE=%0.4f Pearson=%0.4f CCC=%0.4f | ACC(theta)=%0.4f MacroF1(theta)=%0.4f"
370         % (tag, res["mae"], res["pearson"], res["ccc"], res["acc"], res["f1"]))
371     if "acc_head" in res:
372         s += " | ACC(分类头)=%0.4f MacroF1(分类头)=%0.4f" % (res["acc_head"], res["f1_head"])
373     s += (" | F1 neg/neu/pos=%0.3f/%0.3f/%0.3f theta=%0.2f n=%0d"
374         % (res["f1_neg"], res["f1_neu"], res["f1_pos"], res["theta"], res["n"]))
375     return s

```

2.3 问题二：训练与推理主流程（缺失课程学习、EMA、早停）

code/02_model/train.py（共 477 行）

```

1  -*- coding: utf-8 -*-
2  """
3  问题2 训练主程序：教师（全模态）->学生（课程学习+缺失注入+蒸馏）->评价->缺失因素扫描
4
5  用法示例：
6  python train.py --data_dir E:\数学建模\data --version aligned
7  python train.py --version unaligned --epochs_teacher 30 --epochs_student 50 --sweep 1
8  python train.py --limit 64 --epochs_teacher 2 --epochs_student 2 --seed 42 # 冒烟测试
9  """
10 import json
11 import os
12 import sys
13 import time
14
15 import numpy as np
16 import torch
17 from torch.utils.data import DataLoader
18
19 sys.path.append(os.path.dirname(os.path.abspath(__file__)))
20 from config import get_config # noqa: E402
21 from data_utils import load_all, MoseiDataset, collate, describe # noqa: E402
22 from missing_sim import (inject_missing, batch_missing_ratios, # noqa: E402
23                          make_scenario_masks, crop_batch,
24                          inject_modality_dropout)
25 from model import MRFNet, PlainFusion # noqa: E402
26 from losses import (total_loss, compute_metrics, tune_threshold, # noqa: E402
27                    format_metrics)
28
29 MODALITIES = ("text", "audio", "vision")
30
31
32 # -----
33 def set_seed(seed):
34     np.random.seed(seed)
35     torch.manual_seed(seed)
36     torch.cuda.manual_seed_all(seed)
37
38
39 def make_loader(split, cfg, shuffle, augment=False):
40     ds = MoseiDataset(split, augment=augment)
41     return DataLoader(ds, batch_size=cfg.batch_size, shuffle=shuffle,
42                     num_workers=cfg.num_workers, collate_fn=collate,
43                     drop_last=False)
44
45
46 def prep(batch, device):
47     x = {m: batch[m].to(device) for m in MODALITIES}
48     miss = {m: batch["miss_" + m].to(device).bool() for m in MODALITIES}
49     pad = {m: batch["pad_" + m].to(device) for m in MODALITIES}
50     for m in MODALITIES:
51         pad[m] = ensure_valid(pad[m]) # 防全遮 NaN
52     return dict(x=x, miss=miss, pad=pad,
53               y_reg=batch["y_reg"].to(device),
54               y_cls=batch["y_cls"].to(device),
55               idx=batch["idx"].to(device))
56

```

```

57
58 def ensure_valid(pad):
59     """保证每条样本每个模态至少有一个非填充位（否则注意力 key 全被屏蔽 -> NaN）。"""
60     allpad = pad.all(dim=1, keepdim=True)
61     return torch.where(allpad, torch.zeros_like(pad), pad)
62
63
64 def mask_feed(b, cfg, unaware=False):
65     """模型看到的“缺失指示”。
66
67     unaware=True（推理期 A0 基线）或 cfg.ablate_mask_indicator=True（A2 消融，训练期也不给）
68     时，把指示位置零：特征仍在缺失位被置零，但模型不知道“那段是缺失的”。
69     """
70     if unaware or getattr(cfg, "ablate_mask_indicator", False):
71         return {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
72     return b["miss"]
73
74
75 def stack_masks(b):
76     M = torch.stack([b["miss"][m] for m in MODALITIES], dim=-1)
77     P = torch.stack([b["pad"][m] for m in MODALITIES], dim=-1)
78     return M, P
79
80
81 def unstack_masks(b, M):
82     """把缺失/填充掩码写回，并把不可用位置的特征置零（屏蔽进模型的有效值）。"""
83     for i, m in enumerate(MODALITIES):
84         b["miss"][m] = M[..., i]
85         bad = (b["miss"][m] | b["pad"][m]).float().unsqueeze(-1)
86         b["x"][m] = b["x"][m] * (1.0 - bad)
87
88
89 def curriculum(epoch, cfg):
90     """课程学习的缺失注入参数。"""
91     for e0, e1, r0, r1, mode in cfg.curric:
92         if e0 <= epoch <= e1:
93             return (r0, r1, mode)
94     return (0.2, 0.5, "mixed")
95
96
97 # -----
98 def run_epoch(model, loader, cfg, device, optimizer=None, teacher=None,
99               lam_kd=0.0, inject_cfg=None, rng=None, ema=None, ema_decay=0.995,
100               epoch=None):
101     train = optimizer is not None
102     model.train(train)
103     if teacher is not None:
104         teacher.eval()
105     if rng is None:
106         rng = np.random.default_rng()
107     tot, cnt, details = 0.0, 0, {}
108     for batch in loader:
109         b = prep(batch, device)
110         x_full = {m: b["x"][m].clone() for m in MODALITIES}
111         if train and getattr(cfg, "crop_aug", False):
112             b = crop_batch(b, rng, min_keep=cfg.crop_min_keep)

```

```

113     if train and getattr(cfg, "noise_sigma", 0.0) > 0:
114         sig = float(cfg.noise_sigma)
115         for m in MODALITIES:
116             b["x"][m] = b["x"][m] + torch.randn_like(b["x"][m]) * sig
117     M, P = stack_masks(b)
118     if inject_cfg is not None:
119         mix = None
120         if getattr(cfg, "rho_mix", None):
121             ep = 10 ** 9 if epoch is None else int(epoch)
122             if ep >= int(getattr(cfg, "rho_mix_epoch", 1) or 1):
123                 mix = cfg.rho_mix
124         M2, _ = inject_missing(M, P, inject_cfg[0], inject_cfg[1],
125                               inject_cfg[2], tuple(cfg.pos_modes), rng=rng,
126                               n_intervals=tuple(getattr(cfg, "n_intervals", (1, 3))),
127                               rho_mix=mix)
128         M = M2
129     if train and getattr(cfg, "mod_dropout", 0.0) > 0:
130         M = inject_modality_dropout(M, P, cfg.mod_dropout, rng=rng, n_mods=1)
131     unstack_masks(b, M)
132
133     with torch.set_grad_enabled(train):
134         ac = autocast_ctx(cfg)
135         if ac is None:
136             mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"], return_aux=True)
137         else:
138             with ac:
139                 mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"], return_aux=True)
140                 mu, logvar, logits, aux = _to_fp32(mu), _to_fp32(logvar), _to_fp32(logits), _to_fp32(aux)
141             out = dict(aux)
142             out.update(mu=mu, logvar=logvar, logits=logits)
143
144         t_out = None
145         if teacher is not None and lam_kd > 0:
146             with torch.no_grad():
147                 zero = {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
148                 if ac is None:
149                     t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"], return_aux=True)
150                 else:
151                     with ac:
152                         t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"], return_aux=True)
153                         t_mu, t_logits = t_mu.float(), t_logits.float()
154                         t_out = dict(mu=t_mu, logits=t_logits, z=t_aux["z"].float())
155
156         loss, det = total_loss(out, b, cfg, lam_kd=lam_kd, teacher_out=t_out)
157
158     if train:
159         optimizer.zero_grad()
160         loss.backward()
161         torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
162         optimizer.step()
163         if ema is not None: # 权重指数移动平均
164             with torch.no_grad():
165                 sd = model.state_dict()
166                 for k in ema:
167                     v = sd[k]
168                     if v.dtype.is_floating_point:

```

```

169         ema[k].mul_(ema_decay).add_(v.detach().float(), alpha=1.0 - ema_decay)
170     else:
171         ema[k].copy_(v.detach())
172
173     tot += float(loss.detach())
174     cnt += 1
175     for k, v in det.items():
176         details[k] = details.get(k, 0.0) + v
177     return tot / max(cnt, 1), {k: v / max(cnt, 1) for k, v in details.items()}
178
179
180 @torch.no_grad()
181 def predict(model, loader, cfg, device, split=None, scenario=None, unaware=False):
182     model.eval()
183     ac = autocast_ctx(cfg)
184     S, CL, YR, YC, ID, RH = [], [], [], [], [], []
185     for batch in loader:
186         b = prep(batch, device)
187         M, P = stack_masks(b)
188         if scenario is not None:
189             mtype, pos, rho = scenario
190             M = make_scenario_masks(M, P, mtype, pos, rho, seed=1234)
191         unstack_masks(b, M)
192         feed = mask_feed(b, cfg, unaware=unaware)
193         if ac is None:
194             mu, _, logits = model(b["x"], feed, b["pad"])
195         else:
196             with ac:
197                 mu, _, logits = model(b["x"], feed, b["pad"])
198             mu, logits = mu.float(), logits.float()
199         S.append(mu.cpu().numpy())
200         CL.append(logits.cpu().numpy())
201         YR.append(b["y_reg"].cpu().numpy())
202         YC.append(b["y_cls"].cpu().numpy())
203         RH.append(batch_missing_ratios(M, P).cpu().numpy())
204         if split is not None:
205             idx = b["idx"].cpu().numpy()
206             ID.extend([split["ids"][i] for i in idx])
207     out = dict(score=np.concatenate(S), logits=np.concatenate(CL),
208               y_reg=np.concatenate(YR), y_cls=np.concatenate(YC),
209               rho=np.concatenate(RH))
210     out["ids"] = np.array(ID) if ID else None
211     return out
212
213
214 def report(pred, theta, tag=""):
215     res = compute_metrics(pred["y_reg"], pred["score"], pred["y_cls"],
216                           theta=theta, y_pred_cls=pred["logits"].argmax(-1))
217     print(format_metrics(res, tag=tag))
218     return res
219
220
221 # -----
222 def build_model(cfg, dims):
223     kind = getattr(cfg, "model_kind", "mrf")
224     if kind == "plain":

```

```

225     # A0/A1 基线: 朴素拼接 + MLP; —ablate_mask 时连 “缺失指示” 这一输入通道也不给
226     return PlainFusion(dims=dims, hidden=cfg.hidden, dropout=cfg.dropout,
227                        target_len=cfg.target_len,
228                        use_indicator=not getattr(cfg, "ablate_mask_indicator", False)).to(cfg.device)
229     return MRFNet(dims=dims, hidden=cfg.hidden, nhead=cfg.nhead,
230                  enc_layers=cfg.enc_layers, n_experts=cfg.n_experts,
231                  dropout=cfg.dropout, downs=cfg.down_factor_map,
232                  target_len=cfg.target_len,
233                  mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
234                  ms_pool=bool(getattr(cfg, "ms_pool", False)),
235                  ms_windows=tuple(getattr(cfg, "ms_windows", (5, 25, 50))),
236                  use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0).to(cfg.device)
237
238
239 def autocast_ctx(cfg):
240     """CPU bfloat16 autocast 上下文 (本机 AMD Zen5 支持 AVX512-BF16) 。
241
242     注意: CPU 上 PyTorch 的 “半精度加速” 是 **bfloat16** 而不是 float16
243     (AVX512-FP16 仅 Intel Sapphire Rapids 有; AMD 无 fp16 原生通路) 。
244     bfloat16 与 fp32 动态范围相同 (8 位指数), 无需 GradScaler。
245     """
246     if not getattr(cfg, "bfloat16", False):
247         return None
248     try:
249         return torch.autocast(device_type="cpu", dtype=torch.bfloat16)
250     except Exception as e:
251         # pragma: no cover
252         print("[WARN] bfloat16 autocast 不可用: %s" % e)
253         return None
254
255 def _to_fp32(obj):
256     """把模型输出递归转回 float32, 保证自定义损失在 fp32 下计算 (可预测、无精度噪声) 。"""
257     if torch.is_tensor(obj):
258         return obj.float() if obj.dtype.is_floating_point else obj
259     if isinstance(obj, dict):
260         return {k: _to_fp32(v) for k, v in obj.items()}
261     if isinstance(obj, (list, tuple)):
262         return type(obj)(_to_fp32(v) for v in obj)
263     return obj
264
265
266 def run_experiment(cfg, seed, data):
267     print("\n" + "=" * 78)
268     print("[SEED %d] version=%s device=%s hidden=%d epochs(t/s)=%d/%d distill=%s"
269           % (seed, cfg.version, cfg.device, cfg.hidden,
270             cfg.epochs_teacher, cfg.epochs_student, getattr(cfg, "use_distill", True)))
271     print("=" * 78)
272     set_seed(seed)
273     tr, va, te = data
274     if cfg.cls_balanced:
275         cnt = np.bincount(np.asarray(tr["labels_cls"]).astype(np.int64), minlength=3).astype(np.float64)
276         w = cnt.sum() / (3.0 * np.maximum(cnt, 1.0))
277         w = np.clip(w, 0.2, 10.0) # 防止某类缺失时权重爆炸
278         cfg.cls_weight = [float(x) for x in w]
279         print("[CLS] 类别计数=%s 权重=%s" % (cnt.astype(int).tolist(),
280                                             [round(x, 3) for x in cfg.cls_weight]))

```

```

281     dims = {m: tr["D"][m] for m in MODALITIES}
282     print("dims=%s" % dims)
283     train_loader = make_loader(tr, cfg, shuffle=True, augment=True)
284     valid_loader = make_loader(va, cfg, shuffle=False)
285     test_loader = make_loader(te, cfg, shuffle=False)
286
287     tag = "s%d" % seed
288     hist = []
289
290     # ----- 1) 教师: 全模态 -----
291     teacher = build_model(cfg, dims)
292     opt = torch.optim.AdamW([
293         {"params": [p for n, p in teacher.named_parameters() if "head_" not in n], "lr": cfg.lr_backbone},
294         {"params": [p for n, p in teacher.named_parameters() if "head_" in n], "lr": cfg.lr_head}],
295         weight_decay=cfg.weight_decay)
296     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.epochs_teacher))
297     best_mae, best_state, bad = 1e9, None, 0
298     for ep in range(1, cfg.epochs_teacher + 1):
299         t0 = time.time()
300         l, det = run_epoch(teacher, train_loader, cfg, cfg.device, optimizer=opt, inject_cfg=None)
301         sched.step()
302         pv = predict(teacher, valid_loader, cfg, cfg.device, split=va)
303         mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())
304         print("[T ep%02d] loss=%4f mae=%4f rec=%4f (%1fs)"
305               % (ep, l, mae, det.get("rec", 0.0), time.time() - t0))
306         hist.append(dict(phase="teacher", epoch=ep, loss=l, valid_mae=mae, **det))
307         if mae < best_mae - 1e-4:
308             best_mae, bad = mae, 0
309             best_state = {k: v.detach().cpu().clone() for k, v in teacher.state_dict().items()}
310         else:
311             bad += 1
312             if bad >= cfg.patience:
313                 print("[T] early stop @ep%d" % ep)
314                 break
315     if best_state is not None:
316         teacher.load_state_dict(best_state)
317     print("[T] best valid MAE = %4f" % best_mae)
318
319     # ----- 2) 学生: 缺失鲁棒 (课程学习 + 蒸馏) -----
320     student = build_model(cfg, dims)
321     opt = torch.optim.AdamW([
322         {"params": [p for n, p in student.named_parameters() if "head_" not in n], "lr": cfg.lr_backbone},
323         {"params": [p for n, p in student.named_parameters() if "head_" in n], "lr": cfg.lr_head}],
324         weight_decay=cfg.weight_decay)
325     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.epochs_student))
326     use_kd = getattr(cfg, "use_distill", True)
327     ema = None
328     if getattr(cfg, "ema", False):
329         ema = {k: v.detach().clone().float() for k, v in student.state_dict().items()}
330         print("[EMA] 启用权重滑动平均, decay=%4f" % getattr(cfg, "ema_decay", 0.995))
331     best_mae, best_state, bad = 1e9, None, 0
332     for ep in range(1, cfg.epochs_student + 1):
333         t0 = time.time()
334         r0, r1, mode = curriculum(ep, cfg)
335         if getattr(cfg, "ablate_inject", False): # A0: 干净数据训练, 不注入缺失
336             inject_cfg = None

```

```

337     else:
338         inject_cfg = (r0, r1, mode)
339     if use_kd and ep > cfg.kd_warmup:
340         lam_kd = cfg.lam_kd0 * float(np.exp(-(ep - 1 - cfg.kd_warmup) / cfg.kd_decay_epochs))
341     else:
342         lam_kd = 0.0
343     l, det = run_epoch(student, train_loader, cfg, cfg.device, optimizer=opt,
344                       teacher=teacher, lam_kd=lam_kd, inject_cfg=inject_cfg,
345                       rng=np.random.default_rng(seed * 1000 + ep),
346                       ema=ema, ema_decay=float(getattr(cfg, "ema_decay", 0.995)),
347                       epoch=ep)
348     sched.step()
349     if ema is not None:
350         _bk = {k: v.detach().clone() for k, v in student.state_dict().items()}
351         dt = next(student.parameters()).dtype
352         student.load_state_dict({k: v.to(dt) for k, v in ema.items()})
353         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
354         student.load_state_dict(_bk)
355     else:
356         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
357     mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())
358     print("[S ep%02d] loss=%0.4f mae=%0.4f rec=%0.4f kd=%0.4f | rho=[%0.2f,%0.2f] %s (%0.1fs)"
359           % (ep, l, mae, det.get("rec", 0.0), det.get("kd", 0.0), r0, r1, mode, time.time() - t0))
360     hist.append(dict(phase="student", epoch=ep, loss=l, valid_mae=mae,
361                    rho_min=r0, rho_max=r1, lam_kd=lam_kd, **det))
362     if mae < best_mae - 1e-4:
363         best_mae, bad = mae, 0
364         best_state = {k: v.detach().cpu().clone() for k, v in student.state_dict().items()}
365     else:
366         bad += 1
367     if bad >= cfg.patience:
368         print("[S] early stop @ep%d" % ep)
369         break
370     if best_state is not None:
371         student.load_state_dict(best_state)
372     print("[S] best valid MAE = %0.4f%s" % (best_mae, " (EMA 权重)" if ema is not None else ""))
373
374     # ----- 3) 阈值选择 + 评价 -----
375     pv = predict(student, valid_loader, cfg, cfg.device, split=va)
376     theta, f1v = tune_threshold(pv["score"], pv["y_cls"])
377     print("[TH] theta=%0.2f (valid MacroF1=%0.4f)" % (theta, f1v))
378     res = {}
379     res["valid_clean"] = report(pv, theta, tag="[VALID clean]")
380     pt = predict(student, test_loader, cfg, cfg.device, split=te)
381     res["test_clean"] = report(pt, theta, tag="[TEST clean]")
382     res["theta"] = theta
383
384     # ----- 4) 缺失因素扫描 (可选, 用于第4节规律分析) -----
385     if cfg.sweep:
386         rows = []
387         pv0 = pv
388         for mtype in ["text", "audio", "vision", "text+audio", "text+vision", "audio+vision"]:
389             for pos in ["head", "middle", "tail", "random"]:
390                 for rho in [0.1, 0.2, 0.3, 0.4, 0.5]:
391                     pa = predict(student, valid_loader, cfg, cfg.device, split=va,
392                                scenario=(mtype, pos, rho), unaware=False)

```

```

393         pu = predict(student, valid_loader, cfg, cfg.device, split=va,
394                       scenario=(mtype, pos, rho), unaware=True)
395         ra = report(pa, theta)
396         ru = report(pu, theta)
397         rows.append(dict(missing_type=mtype, position=pos, rho=rho,
398                          aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["acc"],
399                          aware_corr=ra["pearson"],
400                          unaware_mae=ru["mae"], unaware_f1=ru["f1"],
401                          unaware_acc=ru["acc"], unaware_corr=ru["pearson"])))
402
403     import pandas as pd
404     df = pd.DataFrame(rows)
405     df.to_csv(os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag), index=False)
406     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag))
407
408     # 代表性场景在 test 上的"缺失泛化"验证 (附件2 test 有标签, 可量化)
409     trows = []
410     for mtype, pos, rho in [("text", "middle", 0.3), ("audio", "random", 0.3),
411                             ("vision", "random", 0.3), ("text+audio", "random", 0.3),
412                             ("audio+vision", "middle", 0.3), ("text+audio+vision", "random", 0.3)]:
413         pa = predict(student, test_loader, cfg, cfg.device, split=te,
414                       scenario=(mtype, pos, rho), unaware=False)
415         pu = predict(student, test_loader, cfg, cfg.device, split=te,
416                       scenario=(mtype, pos, rho), unaware=True)
417         ra, ru = report(pa, theta), report(pu, theta)
418         trows.append(dict(missing_type=mtype, position=pos, rho=rho,
419                          aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["acc"],
420                          unaware_mae=ru["mae"], unaware_f1=ru["f1"], unaware_acc=ru["acc"])))
421     pd.DataFrame(trows).to_csv(os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag), index=False)
422     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag))
423     res["sweep"] = rows
424
425     # ----- 5) 保存 -----
426     ckpt = os.path.join(cfg.out_dir, "student_%s.pt" % tag)
427     state = student.state_dict()
428     if getattr(cfg, "save_half", False):
429         state = {k: (v.half() if v.is_floating_point() else v) for k, v in state.items()}
430     torch.save(dict(state=state, dims=dims, theta=theta,
431                    stats=tr["stats"], cfg=cfg.__dict__, model_kind=getattr(cfg, "model_kind", "mrf"),
432                    mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
433                    ms_pool=bool(getattr(cfg, "ms_pool", False)),
434                    ms_windows=list(getattr(cfg, "ms_windows", (5, 25, 50))),
435                    use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0,
436                    version=cfg.version, seed=seed, half=bool(getattr(cfg, "save_half", False))), ckpt)
437     print("[SAVE] %s (%.2fMB)" % (ckpt, os.path.getsize(ckpt) / 1e6))
438     import pandas as pd
439     pd.DataFrame(hist).to_csv(os.path.join(cfg.out_dir, "history_%s.csv" % tag), index=False)
440     with open(os.path.join(cfg.out_dir, "metrics_%s.json" % tag), "w", encoding="utf-8") as f:
441         json.dump({k: v for k, v in res.items() if k != "sweep"}, f,
442                   ensure_ascii=False, indent=2)
443
444     return res
445
446 def main():
447     cfg = get_config()
448     torch.set_num_threads(max(1, os.cpu_count() or 4))
449     print("[CFG] %s" % json.dumps({k: v for k, v in cfg.__dict__.items()}

```

```

449         if isinstance(v, (int, float, str, bool)),
450             ensure_ascii=False, indent=2))
451     print("[DATA] 读取 %s" % cfg.pkl_path)
452     tr, va, te = load_all(cfg)
453     for name, s in (("train", tr), ("valid", va), ("test", te)):
454         print(describe(s, name))
455
456     all_res = []
457     for seed in cfg.seeds:
458         all_res.append(run_experiment(cfg, seed, (tr, va, te)))
459
460     if len(all_res) > 1:
461         print("\n" + "=" * 78)
462         print("[SUMMARY] %d seeds: mean +/- std" % len(all_res))
463         keys = ["mae", "pearson", "ccc", "acc", "f1"]
464         for split in ("valid_clean", "test_clean"):
465             line = split + ":\n"
466             for k in keys:
467                 v = np.array([r[split][k] for r in all_res])
468                 line += "%s=%0.4f+/-%0.4f " % (k, v.mean(), v.std())
469             print(line)
470         with open(os.path.join(cfg.out_dir, "summary.json"), "w", encoding="utf-8") as f:
471             json.dump([{"k": v for k, v in r.items() if k != "sweep"} for r in all_res],
472                       f, ensure_ascii=False, indent=2)
473
474
475 if __name__ == "__main__":
476     main()

```

2.4 问题三：骨干加载与批构造（三源解释的公共入口）

code/03_xai/xai_common.py（共 248 行）

```

1  #-*- coding: utf-8 -*-
2  r"""问题3 共享工具：骨干加载、split 构建、掩码/遮挡算子、目标标量、缓存读写。
3
4  设计要点（与 02_model 的推理链路严格一致，保证“解释的是同一个模型”）：
5  * 复用 'runtime.load_ensemble' 与 'train.prep/stack_masks/unstack_masks'，
6    不重写前向路径，避免“解释的模型 != 预测的模型”；
7  * 屏蔽模式统一用 'miss=1'（模型在训练时见过的“整模态缺失”语义），
8    并同步把特征置零（与 'unstack_masks' 的做法一致）；
9  * 目标标量支持 'mu'（强度，主口径）与 'p_pos/p_neg/pol/p_max'（极性，对照口径）。
10 """
11 import glob
12 import json
13 import os
14 import sys
15
16 import numpy as np
17 import torch
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 ROOT = os.path.dirname(CODE)

```

```

22 for p in (CODE, os.path.join(CODE, "02_model")):
23     if p not in sys.path:
24         sys.path.append(p)
25
26 from data_utils import load_any, MoseiDataset, collate # noqa: E402
27 from runtime import load_ensemble, predict_split # noqa: E402
28 from train import prep, stack_masks, unstack_masks # noqa: E402
29
30 MODALITIES = ("text", "audio", "vision")
31 LABELS = ("Negative", "Neutral", "Positive")
32 TARGETS = ("mu", "p_pos", "p_neg", "p_pol", "p_max")
33
34
35 # -----
36 # 骨干与数据
37 # -----
38 def load_backbone(ckpt_dir=None, device="cpu", verbose=True):
39     ckpt_dir = ckpt_dir or os.path.join(ROOT, "runs", "ens_top2")
40     ckpts = sorted(glob.glob(os.path.join(ckpt_dir, "student_*.pt")))
41     if not ckpts:
42         raise SystemExit("未找到权重: %s/student_*.pt" % ckpt_dir)
43     models = load_ensemble(ckpts, device=device, verbose=verbose)
44     for m, _ in models: # 只解释输入，不需要参数梯度
45         for p in m.parameters():
46             p.requires_grad_(False)
47     theta = float(np.mean([ck["theta"] for _, ck in models]))
48     return models, models[0][1]["stats"], theta
49
50
51 def load_split(pkl, stats, key=None):
52     """读任意 pkl（附件2/3/4）并用 ckpt 内统计量归一化。"""
53     splits = load_any(pkl, norm_stats=stats)
54     if key is None:
55         key = "test" if "test" in splits else list(splits.keys())[0]
56     return splits[key], key
57
58
59 def make_batch(split, idxs, device="cpu", mask=True):
60     """按 DataLoader 同路径构造 batch（含 pad 有效位兜底）。
61
62     mask=True 时额外执行 'apply_masks'（把 miss|pad 位置的特征置零），
63     与 'infer_att3.py' 的推理链路**完全一致**——这是“解释的模型=预测的模型”的前提：
64     若不置零，填充位的 BERT 垃圾值会经编码器内的局部卷积渗入最后一个有效位，
65     造成解释与提交预测的口径偏差（实测最大分数差 0.049、1/20 标签不同）。
66     """
67     ds = MoseiDataset(split, augment=False)
68     b = prep(collate([ds[int(i)] for i in idxs]), device)
69     return apply_masks(b) if mask else b
70
71
72 def apply_masks(b):
73     """与推理链路一致：把 miss|pad 位置的特征置零。"""
74     M, _ = stack_masks(b)
75     unstack_masks(b, M)
76     return b
77

```

```

78
79 def valid_len(b, m, j):
80     """第 j 个样本、模态 m 的有效长度（非填充位）。"""
81     return int((~b["pad"][m][j]).sum().item())
82
83
84 # -----
85 # 掩码/遮挡算子
86 # -----
87 def mask_modalities(b, mods, mode="miss"):
88     """屏蔽若干模态。mode='miss' 置缺失指示；mode='pad' 整模态不可见。"""
89     out = dict(x={}, miss={}, pad={})
90     for k in ("y_reg", "y_cls", "idx"):
91         if k in b:
92             out[k] = b[k]
93     for m in MODALITIES:
94         out["x"][m] = b["x"][m].clone()
95         out["miss"][m] = b["miss"][m].clone()
96         out["pad"][m] = b["pad"][m].clone()
97     for m in mods:
98         if mode == "miss":
99             out["miss"][m] = torch.ones_like(b["miss"][m])
100         else:
101             p = torch.ones_like(b["pad"][m])
102             p[:, 0] = False # 安全位：避免注意力全遮 -> NaN
103             out["pad"][m] = p
104     for m in MODALITIES:
105         bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
106         out["x"][m] = out["x"][m] * (1.0 - bad)
107     return out
108
109
110 def occlude_window(b, m, start, width):
111     """把模态 m 的 [start, start+width) 段用相邻段替换（保持序列长度与形态）。
112
113     首段向右取邻段、末段向左取邻段；缺失/填充位保持不可用。
114     """
115     out = dict(x={}, miss={}, pad={})
116     for k in ("y_reg", "y_cls", "idx"):
117         if k in b:
118             out[k] = b[k]
119     L = b["x"][m].size(1)
120     for mm in MODALITIES:
121         out["x"][mm] = b["x"][mm].clone()
122         out["miss"][mm] = b["miss"][mm].clone()
123         out["pad"][mm] = b["pad"][mm].clone()
124     s = max(0, int(start))
125     e = min(L, s + int(width))
126     if e <= s:
127         return out
128     off = e - s
129     if e + off <= L:
130         src = slice(e, e + off)
131         dst = slice(s, e)
132     elif s - off >= 0:
133         src = slice(s - off, s)

```

```

134     dst = slice(s, e)
135 else:                                     # 序列过短: 退回均值替换
136     src, dst = None, slice(s, e)
137     for j in range(out["x"][m].size(0)):
138         if src is None:
139             avail = ~(b["miss"][m][j] | b["pad"][m][j])
140             fill = b["x"][m][j][avail].mean(dim=0) if avail.any() else torch.zeros_like(
141                 b["x"][m][j][0])
142             out["x"][m][j, dst] = fill
143         else:
144             out["x"][m][j, dst] = b["x"][m][j][src]
145             out["miss"][m][j, dst] = False          # 被替换段视为“有信息”
146     bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
147     out["x"][m] = out["x"][m] * (1.0 - bad)
148     return out
149
150
151 # -----
152 # 前向与目标标量
153 # -----
154 def predict_batch(models, b, target="mu", aux_keys=()):
155     """集成前向。返回 score/prob/target (均为可微张量) 与可选 aux。"""
156     mus, ps = [], []
157     aux_sum = {k: None for k in aux_keys}
158     for m, _ in models:
159         if aux_keys:
160             mu, _, lg, ax = m(b["x"], b["miss"], b["pad"], return_aux=True)
161             for k in aux_keys:
162                 v = ax.get(k)
163                 if torch.is_tensor(v) and v.size(0) == mu.size(0):
164                     v = v.detach()
165                     aux_sum[k] = v if aux_sum[k] is None else aux_sum[k] + v
166             else:
167                 mu, _, lg = m(b["x"], b["miss"], b["pad"])
168             mus.append(mu)
169             ps.append(torch.softmax(lg, dim=-1))
170     score = torch.stack(mus).mean(0)
171     prob = torch.stack(ps).mean(0)
172     if target == "mu":
173         t = score
174     elif target == "p_pos":
175         t = prob[:, 2]
176     elif target == "p_neg":
177         t = prob[:, 0]
178     elif target == "p_pol":
179         t = prob[:, 2] - prob[:, 0]
180     elif target == "p_max":
181         t = prob.max(dim=-1).values
182     else:
183         raise ValueError("未知 target=%s (可选 %s)" % (target, TARGETS))
184     out = dict(score=score, prob=prob, target=t)
185     for k in aux_keys:
186         if aux_sum[k] is not None:
187             out[k] = aux_sum[k] / len(models)
188     return out
189

```

```

190
191 # -----
192 # 重要性后处理与缓存
193 # -----
194 def norm_valid(w, n_valid):
195     """把重要性向量限制在有效位内并归一到 [0,1]（全零时返回全零）。"""
196     w = np.asarray(w, dtype=np.float64).copy()
197     n = int(max(1, min(len(w), n_valid)))
198     seg = w[:n]
199     w[n:] = 0.0
200     lo, hi = float(seg.min()), float(seg.max())
201     if hi - lo < 1e-12:
202         return np.zeros_like(w)
203     w[:n] = (seg - lo) / (hi - lo)
204     return w
205
206
207 def softmax1d(x, tau=1.0):
208     x = np.asarray(x, dtype=np.float64) / max(float(tau), 1e-9)
209     x = x - x.max()
210     e = np.exp(x)
211     return e / e.sum()
212
213
214 def save_npz(path, **arrays):
215     os.makedirs(os.path.dirname(path), exist_ok=True)
216     np.savez_compressed(path, **arrays)
217     return path
218
219
220 def load_npz(path):
221     with np.load(path) as z:
222         return {k: z[k] for k in z.files}
223
224
225 def save_json(path, obj):
226     os.makedirs(os.path.dirname(path), exist_ok=True)
227     with open(path, "w", encoding="utf-8") as f:
228         json.dump(obj, f, ensure_ascii=False, indent=1, default=_jsonable)
229     return path
230
231
232 def _jsonable(o):
233     if isinstance(o, (np.integer,)):
234         return int(o)
235     if isinstance(o, (np.floating,)):
236         return float(o)
237     if isinstance(o, np.ndarray):
238         return o.tolist()
239     return str(o)
240
241
242 def q3_meta(ckpt_dir, models, theta, extra=None):
243     """记录本次解释使用的骨干与阈值，供复现核对。"""
244     files = sorted(os.path.basename(p) for p in
245                     glob.glob(os.path.join(ckpt_dir, "student*.pt")))

```

```

246     return dict(model_files=files, n_ckpt=len(models), ckpt_dir=str(ckpt_dir),
247                 theta=round(float(theta), 4), extra=extra or {})

```

2.5 问题三：精确 Shapley 值（八子集全枚举）

code/03_xai/shapley.py（共 259 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S1: 模态级精确 Shapley 值（3 模态 -> 8 个子集全枚举）。
3
4  数学定义（论文 section5.3 用）：
5  v(S) 为只用模态子集 S 时模型的目标标量；模态 m 的 Shapley 值
6  phi_m = sum_{S subset M \ {m}} |S|!(n-|S|-1)!/n! * [v(S \ {m}) - v(S)], n=3
7  精确性检验：sum_m phi_m = v(M) - v({})。
8
9  本脚本一次算完 4 个口径组合（保证可比、避免重复前向）：
10 target in {mu(强度,主), p_pol = p_pos - p_neg(极性,对照)}
11 semantic in {miss(置缺失指示,主), pad(整模态不可见,对照)}
12 并输出两套归一化（ReLU 归一 / softmax|phi|）供后续按验证集“模态忠实性”择优。
13
14 两类效用：
15 * 样本级：v(S) 取该样本自身的预测 -> 逐样本 phi（写入附件4 解释 CSV）；
16 * 数据集级：v(S) 取验证集上的 -MAE 与 Macro-F1 -> 全局模态重要性（论文结论）。
17
18 用法：
19 python shapley.py --pk1 <pk1> --name att4 --out_dir <runs/q3>
20 python shapley.py --pk1 <附件2 aligned_50.pk1> --split valid --name valid --with_global
21 """
22 import argparse
23 import itertools
24 import math
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30
31 HERE = os.path.dirname(os.path.abspath(__file__))
32 if HERE not in sys.path:
33     sys.path.append(HERE)
34 from xai_common import (MODALITIES, load_backbone, load_split, make_batch, # noqa: E402
35                         predict_batch, mask_modalities, norm_valid, softmaxid,
36                         save_npz, save_json, q3_meta, ROOT)
37
38 SUBSETS = [tuple(m for i, m in enumerate(MODALITIES) if (s >> i) & 1) for s in range(8)]
39 MASKS = list(range(8))
40 NAMES = {0: "none", 1: "text", 2: "audio", 4: "vision",
41          3: "text+audio", 5: "text+vision", 6: "audio+vision", 7: "all"}
42 WEIGHTS = {}
43
44
45 def mods_of(mask):
46     """位掩码 -> 模态名列表（text=1, audio=2, vision=4）。"""
47     return [MODALITIES[i] for i in range(3) if (mask >> i) & 1]

```

```

48
49
50 def shapley_weights(n=3):
51     """精确 Shapley 的组合权重  $w(S) = |S|!(n-|S|-1)!/n!$  (按位掩码索引)。"""
52     if n not in WEIGHTS:
53         w = {}
54         for i in range(3):
55             others = [j for j in range(3) if j != i]
56             tab = {}
57             for r in range(len(others) + 1):
58                 for S in itertools.combinations(others, r):
59                     tab[sum(1 << j for j in S)] = (math.factorial(len(S)) *
60                                                         math.factorial(n - len(S) - 1) /
61                                                         math.factorial(n))
62             w[i] = tab
63             WEIGHTS[n] = w
64     return WEIGHTS[n]
65
66
67 def phi_from_values(v):
68     """由 8 个子集的 v 值 (按 mask 索引) 算精确 Shapley 向量 (3,)。"""
69     W = shapley_weights(3)
70     phi = np.zeros(3)
71     for i in range(3):
72         for T, wt in W[i].items():
73             phi[i] += wt * (v[T | (1 << i)] - v[T])
74     return phi
75
76
77 def normalizers(phi):
78     """两套归一化 (对 (N,3) 或 (3,) 的 phi 逐样本处理)。"""
79     a = np.asarray(phi, dtype=np.float64)
80     single = a.ndim == 1
81     a = a[None] if single else a
82     relu = np.maximum(a, 0.0)
83     s = relu.sum(1, keepdims=True)
84     pi_relu = np.where(s > 1e-12, relu / np.maximum(s, 1e-12), np.nan)
85     pi_soft = np.exp(np.abs(a) / 1.0)
86     pi_soft = pi_soft / pi_soft.sum(1, keepdims=True)
87     return (pi_relu[0], pi_soft[0]) if single else (pi_relu, pi_soft)
88
89
90 def run_batch(models, b, target, semantic):
91     """对一个 batch 算 8 个子集的 v 值 -> V (B,8), 并返回全模态输出。"""
92     B = b["x"]["text"].size(0)
93     V = np.zeros((B, 8))
94     full = None
95     for mask in MASKS:
96         bb = b if mask == 7 else mask_modalities(
97             b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
98         out = predict_batch(models, bb, target=target)
99         V[:, mask] = out["target"].detach().cpu().numpy()
100         if mask == 7:
101             full = dict(score=out["score"].detach().cpu().numpy(),
102                         prob=out["prob"].detach().cpu().numpy())
103     return V, full

```

```

104
105
106 def global_values(models, split, target, semantic, batch=64, device="cpu"):
107     """数据集级效用: v(S) = -MAE (回归) 与 Macro-F1 (分类头概率)。"""
108     from sklearn.metrics import f1_score
109     N = split["N"]
110     y = np.asarray(split["labels_reg"])
111     yc = np.asarray(split["labels_cls"])
112     v_mae, v_f1 = np.zeros(8), np.zeros(8)
113     for mask in MASKS:
114         preds, probs = [], []
115         for i0 in range(0, N, batch):
116             idxs = list(range(i0, min(i0 + batch, N)))
117             b = make_batch(split, idxs, device)
118             if mask != 7:
119                 b = mask_modalities(
120                     b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
121             out = predict_batch(models, b, target=target)
122             preds.append(out["score"].detach().cpu().numpy())
123             probs.append(out["prob"].detach().cpu().numpy())
124             sc, pb = np.concatenate(preds), np.concatenate(probs)
125             v_mae[mask] = -float(np.abs(sc - y).mean())
126             try:
127                 v_f1[mask] = float(f1_score(yc, pb.argmax(1), average="macro"))
128             except Exception:
129                 v_f1[mask] = np.nan
130             print(" [GLOBAL] S=%-12s MAE=%0.4f MacroF1=%0.4f"
131                   % (NAMES[mask], -v_mae[mask], v_f1[mask]))
132     return v_mae, v_f1
133
134
135 def main():
136     ap = argparse.ArgumentParser()
137     ap.add_argument("--pkl", default=None, help="默认附件2 aligned (用 --split valid)")
138     ap.add_argument("--split", default=None, help="pkl 内的 split 名 (附件2 需指定 valid)")
139     ap.add_argument("--name", default="att4", help="输出文件名后缀")
140     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
141     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
142     ap.add_argument("--batch_size", type=int, default=32)
143     ap.add_argument("--limit", type=int, default=0, help=">0 时只算前 N 条")
144     ap.add_argument("--device", default="cpu")
145     ap.add_argument("--with_global", action="store_true", help="额外算数据集级 Shapley")
146     ap.add_argument("--global_target", default="mu", choices=["mu", "p_pol"])
147     a = ap.parse_args()
148
149     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
150     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
151     split, key = load_split(pkl, stats, a.split)
152     if a.limit:
153         from data_utils import slice_split
154         split = slice_split(split, a.limit)
155     N = split["N"]
156     print("[DATA] %s/%s N=%d" % (os.path.basename(pkl), key, N))
157
158     combos = [("mu", "miss"), ("mu", "pad"), ("p_pol", "miss"), ("p_pol", "pad")]
159     store, v_store = {}, {}

```

```

160 score_all, prob_all = None, None
161 for target, semantic in combos:
162     tag = "%s_%s" % (target, semantic)
163     V = np.zeros((N, 8))
164     phis = np.zeros((N, 3))
165     sc, pb = [], []
166     for i0 in range(0, N, a.batch_size):
167         idxs = list(range(i0, min(i0 + a.batch_size, N)))
168         b = make_batch(split, idxs, a.device)
169         v, full = run_batch(models, b, target, semantic)
170         for r in range(len(idxs)):
171             phis[i0 + r] = phi_from_values(v[r])
172         V[i0:i0 + len(idxs)] = v
173         sc.append(full["score"])
174         pb.append(full["prob"])
175     store[tag], v_store[tag] = phis, V
176     resid = float(np.abs(phis.sum(1) - (V[:, 7] - V[:, 0])).max())
177     print(" [%-10s] phi 均值 文/语/视 = %0.4f / %0.4f / %0.4f; 精确性残差 max=%0.2e"
178           % (tag, *phis.mean(0), resid))
179     assert resid < 1e-6, "Shapley 精确性检验失败"
180     if tag == "mu_miss":
181         score_all, prob_all = np.concatenate(sc), np.concatenate(pb)
182
183 # 主口径: mu + miss; 同时给出两套归一化
184 phi_main = store["mu_miss"]
185 pi_relu, pi_soft = normalizers(phi_main)
186 n_relu = int(np.isnan(pi_relu).any(1).sum())
187 print(" [NORM] ReLU 归一在全样本上退化 %d/%d 条; softmax 口径无退化" % (n_relu, N))
188 miss_rate = np.zeros((N, 3))
189 for i in range(N):
190     for k, m in enumerate(MODALITIES):
191         span = float((~split["pad"][m][i]).sum())
192         if span <= 1.0: # 整条全零: 整模态缺失 (口径 D1)
193             miss_rate[i, k] = 1.0
194         else:
195             miss_rate[i, k] = float((split["miss"][m][i] & ~split["pad"][m][i]).sum()) / span
196
197 out = dict(ids=np.asarray(split["ids"]).astype(str),
198           score=score_all, prob=prob_all, phi_mu_miss=phi_main,
199           pi_relu=np.nan_to_num(pi_relu, nan=0.0), pi_soft=pi_soft,
200           miss_rate=miss_rate, subset_names=np.array([NAMES[m] for m in MASKS]))
201 for tag in store:
202     out["phi_" + tag] = store[tag]
203     out["v_" + tag] = v_store[tag]
204 path = save_npz(os.path.join(a.out_dir, "q3_shapley_%s.npz" % a.name), **out)
205
206 # CSV (逐样本)
207 import pandas as pd
208 df = pd.DataFrame({
209     "id": out["ids"],
210     "score": np.round(score_all, 4),
211     "prob_neg": np.round(prob_all[:, 0], 4), "prob_neu": np.round(prob_all[:, 1], 4),
212     "prob_pos": np.round(prob_all[:, 2], 4),
213     "main_modality": [MODALITIES[int(np.argmax(r))] for r in pi_soft],
214     "main_modality_relu": [MODALITIES[int(np.argmax(r))] if not np.isnan(r).any()
215                             else "" for r in pi_relu],

```

```

216     "pi_text": np.round(pi_soft[:, 0], 4), "pi_audio": np.round(pi_soft[:, 1], 4),
217     "pi_vision": np.round(pi_soft[:, 2], 4),
218     "phi_text": np.round(phi_main[:, 0], 4), "phi_audio": np.round(phi_main[:, 1], 4),
219     "phi_vision": np.round(phi_main[:, 2], 4),
220     "phi_text_padsem": np.round(store["mu_pad"][:, 0], 4),
221     "phi_audio_padsem": np.round(store["mu_pad"][:, 1], 4),
222     "phi_vision_padsem": np.round(store["mu_pad"][:, 2], 4),
223     "miss_text": np.round(miss_rate[:, 0], 4), "miss_audio": np.round(miss_rate[:, 1], 4),
224     "miss_vision": np.round(miss_rate[:, 2], 4),
225 })
226 csv = os.path.join(a.out_dir, "q3_shapley_%s.csv" % a.name)
227 os.makedirs(a.out_dir, exist_ok=True)
228 df.to_csv(csv, index=False, encoding="utf-8-sig")
229
230 report = dict(meta=q3_meta(a.ckpt_dir, models, theta,
231                          dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N)),
232              data=os.path.relpath(path, ROOT),
233              phi_mean={k: np.round(store[k].mean(0), 4).tolist() for k in store},
234              pi_soft_mean=np.round(pi_soft.mean(0), 4).tolist(),
235              pi_relu_mean=np.round(np.nan_to_num(pi_relu, nan=0.0).mean(0), 4).tolist(),
236              pi_relu_degenerate=int(n_relu),
237              v_mean_mu_miss={NAMES[m]: round(float(v_store["mu_miss"][:, m].mean()), 4)
238                             for m in MASKS})
239 if a.with_global:
240     print(" [GLOBAL] 数据集级效用 (%s / miss 语义) : " % a.global_target)
241     v_mae, v_f1 = global_values(models, split, a.global_target, "miss", a.batch_size,
242                                a.device)
243     report["global"] = dict(target=a.global_target,
244                             v_negmae={NAMES[m]: round(float(v_mae[m]), 4) for m in MASKS},
245                             v_macrof1={NAMES[m]: round(float(v_f1[m]), 4) for m in MASKS},
246                             phi_negmae=np.round(phi_from_values(v_mae), 4).tolist(),
247                             phi_macrof1=np.round(
248                                 phi_from_values(np.nan_to_num(v_f1)), 4).tolist())
249     print("   phi(-MAE) = %s" % report["global"]["phi_negmae"])
250     print("   phi(MacroF1) = %s" % report["global"]["phi_macrof1"])
251 save_json(os.path.join(a.out_dir, "q3_shapley_%s_report.json" % a.name), report)
252 print("[SAVE] %s\n[SAVE] %s" % (os.path.relpath(path, ROOT), os.path.relpath(csv, ROOT)))
253 print("SHAPLEY_OK")
254 return 0
255
256
257 if __name__ == "__main__":
258     sys.exit(main())

```

2.6 问题三：积分梯度与片段级重要性

code/03_xai/ig_gradcam.py (共 135 行)

```

1  #-*- coding: utf-8 -*-
2  r"""S2: 片段级时间重要性的梯度类来源（注意力 / 积分梯度 / 注意力 x 梯度）。
3
4  三个来源（论文 section5.3 的 (a)(b) 两族）：
5  (a) 注意力源      w^att_k = alpha_k          （模型内生，仅作参考）
6  (a') 注意力 x 梯度源  w^agrad_k = alpha_k . ||d t/d x_k||_2    （Grad-CAM 式加权）

```

```

7  (b) 积分梯度       $w^{\wedge}ig\_k = \|(x\_k - x'_k) * (1/L) \text{Sigma}_l \text{grad}_{\{x\_k\}} t(x' + 1/L(x - x'))\|_2$ 
8
9  基线 x': 'zero' (归一化空间中即训练集均值, 主口径) / 'mean' (本 batch 有效位均值, 对照)。
10
11 用法:
12 python ig_gradcam.py --pk1 <pk1> --name att4 --steps 20 --target mu --out_dir <runs/q3>
13 """
14 import argparse
15 import os
16 import sys
17
18 import numpy as np
19 import torch
20
21 HERE = os.path.dirname(os.path.abspath(__file__))
22 if HERE not in sys.path:
23     sys.path.append(HERE)
24 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
25                          predict_batch, save_npz, save_json, q3_meta, ROOT)
26
27
28 def ig_batch(models, b, steps=20, baseline="zero", target="mu", need_agrad=True):
29     """返回 (w_ig (B,3,L), w_att (B,3,L), w_agrad (B,3,L))。"""
30     x0 = {m: b["x"][m].detach().clone() for m in MODALITIES}
31     if baseline == "zero":
32         base = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
33     else:
34         # mean: 该样本有效位均值
35         base = {}
36         for m in MODALITIES:
37             valid = (~(b["miss"][m] | b["pad"][m])).float().unsqueeze(-1)
38             mu = (x0[m] * valid).sum(1, keepdim=True) / valid.sum(1, keepdim=True).clamp(min=1)
39             base[m] = mu.expand_as(x0[m]).contiguous()
40     gsum = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
41     for l in range(1, steps + 1):
42         a = 1 / float(steps)
43         xs = {m: (base[m] + a * (x0[m] - base[m])).requires_grad_(True) for m in MODALITIES}
44         out = predict_batch(models, dict(x=xs, miss=b["miss"], pad=b["pad"]), target=target)
45         g = torch.autograd.grad(out["target"].sum(), [xs[m] for m in MODALITIES],
46                                 retain_graph=False)
47         for i, m in enumerate(MODALITIES):
48             gsum[m] = gsum[m] + g[i].detach()
49     w_ig = torch.stack([(x0[m] - base[m]) * (gsum[m] / steps)].norm(dim=-1)
50                        for m in MODALITIES], dim=1)      # (B,3,L)
51
52     # 注意力源 + 注意力 x 梯度源 (一次前向 + 一次反向)
53     xg = {m: x0[m].clone().requires_grad_(True) for m in MODALITIES}
54     out = predict_batch(models, dict(x=xg, miss=b["miss"], pad=b["pad"]),
55                         target=target, aux_keys=("alpha",))
56     grads = torch.autograd.grad(out["target"].sum(), [xg[m] for m in MODALITIES],
57                                 retain_graph=False)
58     gnorm = torch.stack([grads[i].detach().norm(dim=-1) for i in range(3)], dim=1)  # (B,3,L)
59     B, _, L = gnorm.shape
60     alpha = out.get("alpha")
61     if alpha is not None and alpha.size(0) == B and alpha.size(1) == 3 * L:
62         w_att = alpha.detach().reshape(B, 3, L)
63     else:
64         # 兜底: 无法取注意力时退化为梯度幅值

```

```

63     w_att = gnorm.clone()
64     w_agrad = w_att * gnorm
65     return (w_ig.detach().cpu().numpy(), w_att.cpu().numpy(), w_agrad.detach().cpu().numpy())
66
67
68 def main():
69     ap = argparse.ArgumentParser()
70     ap.add_argument("--pkl", default=None)
71     ap.add_argument("--split", default=None)
72     ap.add_argument("--name", default="att4")
73     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
74     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
75     ap.add_argument("--steps", type=int, default=20)
76     ap.add_argument("--baseline", default="zero", choices=["zero", "mean"])
77     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
78     ap.add_argument("--batch_size", type=int, default=8)
79     ap.add_argument("--limit", type=int, default=0)
80     ap.add_argument("--device", default="cpu")
81     a = ap.parse_args()
82
83     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
84     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
85     split, key = load_split(pkl, stats, a.split)
86     if a.limit:
87         from data_utils import slice_split
88         split = slice_split(split, a.limit)
89     N = split["N"]
90     L = split["L"] ["text"]
91     print("[DATA] %s/%s N=%d L=%d | IG steps=%d baseline=%s target=%s"
92           % (os.path.basename(pkl), key, N, L, a.steps, a.baseline, a.target))
93
94     W_IG = np.zeros((N, 3, L), np.float64)
95     W_AT = np.zeros((N, 3, L), np.float64)
96     W_AG = np.zeros((N, 3, L), np.float64)
97     for i0 in range(0, N, a.batch_size):
98         idxs = list(range(i0, min(i0 + a.batch_size, N)))
99         b = make_batch(split, idxs, a.device)
100         ig, att, ag = ig_batch(models, b, a.steps, a.baseline, a.target)
101         W_IG[i0:i0 + len(idxs)] = ig
102         W_AT[i0:i0 + len(idxs)] = att
103         W_AG[i0:i0 + len(idxs)] = ag
104         if (i0 // a.batch_size) % 20 == 0:
105             print(" %d/%d" % (min(i0 + a.batch_size, N), N))
106
107     # 有效位掩码（重要性只在有效位内有意义）
108     valid = np.zeros((N, 3, L), bool)
109     for i in range(N):
110         for k, m in enumerate(MODALITIES):
111             valid[i, k, :int((~split["pad"] [m] [i]).sum())] = True
112     for W in (W_IG, W_AT, W_AG):
113         W[~valid] = 0.0
114     path = save_npz(os.path.join(a.out_dir, "q3_temporal_ig_%s.npz" % a.name),
115                    ids=np.asarray(split["ids"]).astype(str),
116                    w_ig=W_IG, w_att=W_AT, w_agrad=W_AG, valid=valid,
117                    meta=np.array([a.steps, a.baseline, a.target], dtype=object))
118     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,

```

```

119         dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N,
120              steps=a.steps, baseline=a.baseline, target=a.target)),
121         w_ig_mean=[round(float(W_IG[:, k].sum(1).mean()), 4) for k in range(3)],
122         w_att_mean=[round(float(W_AT[:, k].sum(1).mean()), 4) for k in range(3)],
123         sparsity_ig_top5=[round(float(np.sort(W_IG[i, k, valid[i, k]][::-1][:5].sum() /
124                                     max(W_IG[i, k, valid[i, k]].sum(), 1e-9)), 3)
125                            for i in range(min(N, 20)) for k in range(1)],
126         data=os.path.relpath(path, ROOT))
127     save_json(os.path.join(a.out_dir, "q3_temporal_ig_%s_report.json" % a.name), rep)
128     print("[SAVE] %s" % os.path.relpath(path, ROOT))
129     print("IG_GRADCAM_OK")
130     return 0
131
132
133 if __name__ == "__main__":
134     sys.exit(main())

```

2.7 问题三：遮挡重要性（窗宽 3 滑窗）

code/03_xai/occlusion.py（共 117 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S3: 模型无关的遮挡重要性（作为解释的验证基准）。
3
4  定义：把模态 m 的窗口 [start, start+w) 用**相邻段**替换（保持序列长度与形态，
5  避免“删掉一段导致长度变短”引入的伪影），测量目标标量的变化
6      d_k = t(X) - t(X - window_k),
7  位置 k 的重要性取覆盖它的所有窗口的 |d| 均值。
8
9  为什么用相邻段替换而不是置零：置零在归一化空间里等价于“该段取训练均值”，
10  会同时改变“语义内容”与“是否存在信息”两件事；相邻段替换只改变内容，
11  更接近“如果这段时间说的是别的内容”的反事实，且对输出的扰动量级可解释。
12
13  用法：
14  python occlusion.py --pk1 <pk1> --name att4 --width 3 --stride 1 --out_dir <runs/q3>
15  python occlusion.py --pk1 ../附件2\\aligned_50.pk1 --split valid --name valid --stride 2
16  """
17  import argparse
18  import os
19  import sys
20
21  import numpy as np
22  import torch
23
24  HERE = os.path.dirname(os.path.abspath(__file__))
25  if HERE not in sys.path:
26      sys.path.append(HERE)
27  from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
28                          predict_batch, occlude_window, save_npz, save_json,
29                          q3_meta, ROOT)
30
31
32  def occl_batch(models, b, width=3, stride=1, target="mu"):
33      """返回 (w_occ(B,3,L), d_win 列表, 窗口起点列表)。”"""

```

```

34     B = b["x"]["text"].size(0)
35     L = b["x"]["text"].size(1)
36     base = predict_batch(models, b, target=target)["target"].detach()
37     W = np.zeros((B, 3, L), np.float64)
38     CNT = np.zeros((B, 3, L), np.float64)
39     d_all = {}
40     starts_all = {}
41     for k, m in enumerate(MODALITIES):
42         nvalid = int((~(b["pad"][m][0])).sum().item()) if B else L
43         starts = list(range(0, max(1, nvalid - width + 1), stride))
44         if not starts:
45             starts = [0]
46         starts_all[m] = starts
47         d = np.zeros((B, len(starts)), np.float64)
48         for j, st in enumerate(starts):
49             bo = occlude_window(b, m, st, width)
50             t = predict_batch(models, bo, target=target)["target"].detach()
51             dd = (base - t).cpu().numpy()
52             d[:, j] = dd
53             for r in range(B):
54                 e = min(L, st + width)
55                 W[r, k, st:e] += np.abs(dd[r])
56                 CNT[r, k, st:e] += 1.0
57         d_all[m] = d
58     W = W / np.maximum(CNT, 1.0)
59     return W, d_all, starts_all
60
61
62 def main():
63     ap = argparse.ArgumentParser()
64     ap.add_argument("--pkl", default=None)
65     ap.add_argument("--split", default=None)
66     ap.add_argument("--name", default="att4")
67     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
68     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
69     ap.add_argument("--width", type=int, default=3)
70     ap.add_argument("--stride", type=int, default=1)
71     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
72     ap.add_argument("--batch_size", type=int, default=16)
73     ap.add_argument("--limit", type=int, default=0)
74     ap.add_argument("--device", default="cpu")
75     a = ap.parse_args()
76
77     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
78     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
79     split, key = load_split(pkl, stats, a.split)
80     if a.limit:
81         from data_utils import slice_split
82         split = slice_split(split, a.limit)
83     N, L = split["N"], split["L"]["text"]
84     print("[DATA] %s/%s N=%d L=%d | 窗口 w=%d 步长 %d target=%s"
85           % (os.path.basename(pkl), key, N, L, a.width, a.stride, a.target))
86
87     W = np.zeros((N, 3, L), np.float64)
88     for i0 in range(0, N, a.batch_size):
89         idxs = list(range(i0, min(i0 + a.batch_size, N)))

```

```

90     b = make_batch(split, idxs, a.device)
91     w, _, _ = occl_batch(models, b, a.width, a.stride, a.target)
92     W[i0:i0 + len(idxs)] = w
93     if (i0 // a.batch_size) % 20 == 0:
94         print(" %d/%d" % (min(i0 + a.batch_size, N), N))
95
96     valid = np.zeros((N, 3, L), bool)
97     for i in range(N):
98         for k, m in enumerate(MODALITIES):
99             valid[i, k, :int((-split["pad"][m][i]).sum())] = True
100     W[~valid] = 0.0
101     path = save_npz(os.path.join(a.out_dir, "q3_temporal_occ_%s.npz" % a.name),
102                    ids=np.asarray(split["ids"]).astype(str), w_occ=W, valid=valid,
103                    meta=np.array([a.width, a.stride, a.target], dtype=object))
104     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,
105                             dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N,
106                                     width=a.width, stride=a.stride, target=a.target)),
107               w_occ_sum_mean=[round(float(W[:, k].sum(1).mean()), 4) for k in range(3)],
108               data=os.path.relpath(path, ROOT))
109     save_json(os.path.join(a.out_dir, "q3_temporal_occ_%s_report.json" % a.name), rep)
110     print("[SAVE] %s" % os.path.relpath(path, ROOT))
111     print("OCCLUSION_OK")
112     return 0
113
114
115 if __name__ == "__main__":
116     sys.exit(main())

```

2.8 问题三：三源融合与 γ 单纯形约束

code/03_xai/temporal_fuse.py (共 153 行)

```

1  #-*- coding: utf-8 -*-
2  r"""S4-a: 三源时间重要性的归一化与融合 (gamma 在验证集上按与遮挡源的一致性标定)。
3
4  三源:
5  w_att 模型内生注意力 (基线, 已知"注意力!=重要性")
6  w_ig 积分梯度 (公理化)
7  w_occ 遮挡 (模型无关, 作为验证基准)
8
9  融合: 先对每个样本、每个模态在**有效位内**做 min-max 归一化, 再线性加权
10     w = (gamma1.w_att + gamma2.w_ig + gamma3.w_occ) / (gamma1+gamma2+gamma3), 最后再归一化一次。
11  gamma 的标定: 在单纯形上枚举 (**约束每个 gamma >= 0.1**, 避免退化为"只留遮挡源"这种平凡解),
12     以"融合结果与遮挡源的 Spearman 一致性 (跨样本、跨模态、有效位内聚合)"为准则;
13     同时报告等权 (1,1,1) 对照。
14
15  用法:
16  python temporal_fuse.py --name valid --out_dir ../runs/q3
17  """
18  import argparse
19  import itertools
20  import json
21  import os
22  import sys

```

```

23
24 import numpy as np
25
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 if HERE not in sys.path:
28     sys.path.append(HERE)
29 from xai_common import MODALITIES, save_npz, save_json, ROOT # noqa: E402
30
31 SOURCES = ("att", "ig", "occ")
32
33
34 def load_sources(out_dir, name):
35     ig = np.load(os.path.join(out_dir, "q3_temporal_ig_%s.npz" % name), allow_pickle=True)
36     oc = np.load(os.path.join(out_dir, "q3_temporal_occ_%s.npz" % name), allow_pickle=True)
37     w = dict(att=ig["w_att"], ig=ig["w_ig"], occ=oc["w_occ"])
38     return w, ig["valid"], np.asarray(ig["ids"]).astype(str)
39
40
41 def norm_valid_mat(W, valid):
42     """对 (N,3,L) 逐样本逐模态在有效位内 min-max 归一化（无效位保持 0）。"""
43     A = np.zeros_like(W, dtype=np.float64)
44     N, K, L = W.shape
45     for i in range(N):
46         for k in range(K):
47             v = valid[i, k]
48             if not v.any():
49                 continue
50             seg = W[i, k, v].astype(np.float64)
51             lo, hi = seg.min(), seg.max()
52             if hi - lo < 1e-12:
53                 continue
54             A[i, k, v] = (seg - lo) / (hi - lo)
55     return A
56
57
58 def spearman(a, b):
59     from scipy.stats import spearmanr
60     if a.size < 3 or np.std(a) < 1e-12 or np.std(b) < 1e-12:
61         return np.nan
62     return float(spearmanr(a, b).statistic)
63
64
65 def agreement(W, ref, valid):
66     """跨样本/跨模态/有效位聚合的 Spearman 一致性。"""
67     a, b = [], []
68     for i in range(W.shape[0]):
69         for k in range(W.shape[1]):
70             v = valid[i, k]
71             if v.sum() < 3:
72                 continue
73             a.append(W[i, k, v])
74             b.append(ref[i, k, v])
75     if not a:
76         return np.nan
77     return spearman(np.concatenate(a), np.concatenate(b))
78

```

```

79
80 def simplex_grid(step=0.1, min_g=0.1):
81     """枚举 gamma1+gamma2+gamma3=1, 每个 >= min_g. """
82     out = []
83     n = int(round(1.0 / step))
84     for i in range(n + 1):
85         for j in range(n + 1 - i):
86             k = n - i - j
87             g = np.array([i, j, k]) * step
88             if g.min() >= min_g - 1e-9 and abs(g.sum() - 1.0) < 1e-9:
89                 out.append(tuple(np.round(g, 3)))
90     return sorted(set(out))
91
92
93 def fuse(Ws, gamma):
94     g = np.asarray(gamma, dtype=np.float64)
95     g = g / g.sum()
96     W = g[0] * Ws["att"] + g[1] * Ws["ig"] + g[2] * Ws["occ"]
97     return W
98
99
100 def main():
101     ap = argparse.ArgumentParser()
102     ap.add_argument("--name", default="valid")
103     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
104     ap.add_argument("--step", type=float, default=0.1)
105     a = ap.parse_args()
106
107     W_raw, valid, ids = load_sources(a.out_dir, a.name)
108     N, K, L = W_raw["occ"].shape
109     print("[DATA] %s N=%d 三源形状=(%d,%d,%d)" % (a.name, N, N, K, L))
110     Ws = {k: norm_valid_mat(W_raw[k], valid) for k in SOURCES}
111     for k in SOURCES:
112         print(" [SRC] %-4s 归一化后均值 文/语/视 = %.4f / %.4f / %.4f"
113               % (k, *[Ws[k][:, i][valid[:, i]].mean() for i in range(3)]))
114
115     print(" [gamma] 单纯形枚举 (每源 >= %.1f), 以与遮挡源的一致性为准则: " % 0.1)
116     rows = []
117     for g in simplex_grid(a.step):
118         W = fuse(Ws, g)
119         ag = agreement(norm_valid_mat(W, valid), Ws["occ"], valid)
120         rows.append((g, ag))
121     rows_sorted = sorted(rows, key=lambda t: -(t[1] if np.isfinite(t[1]) else -9))
122     for g, ag in rows_sorted[:5]:
123         print("   gamma=(%.1f,%.1f,%.1f) 一致性 rho=%.4f" % (*g, ag))
124     g_best = rows_sorted[0][0]
125     g_eq = (1 / 3, 1 / 3, 1 / 3)
126     W_best = fuse(Ws, g_best)
127     W_eq = fuse(Ws, g_eq)
128     print(" [gamma] 选定 gamma_att/gamma_ig/gamma_occ = (%.2f,%.2f,%.2f) (等权对照 rho=%.4f)"
129           % (*g_best, agreement(norm_valid_mat(W_eq, valid), Ws["occ"], valid)))
130
131     Wb = norm_valid_mat(W_best, valid)
132     We = norm_valid_mat(W_eq, valid)
133     path = save_npz(os.path.join(a.out_dir, "q3_temporal_fused_%s.npz" % a.name),
134                    ids=ids, valid=valid, w_fused=Wb, w_fused_equal=We,

```

```

135         w_att=Ws["att"], w_ig=Ws["ig"], w_occ=Ws["occ"],
136         gamma=np.array(g_best))
137     rep = dict(name=a.name, n=N, gamma=list(g_best), gamma_equal=list(g_eq),
138             grid=[dict(gamma=list(g), spearman_occ=(None if not np.isfinite(ag) else round(ag, 4)))
139                 for g, ag in rows_sorted],
140             source_mean={k: [round(float(Ws[k][:, i][valid[:, i]].mean()), 4)
141                             for i in range(3)] for k in SOURCES},
142             agreement_with_occ={k: round(agreement(Ws[k], Ws["occ"], valid), 4)
143                                 for k in SOURCES},
144             data=os.path.relpath(path, ROOT))
145     save_json(os.path.join(a.out_dir, "q3_gamma_%s.json" % a.name), rep)
146     print("[SAVE] %s" % os.path.relpath(path, ROOT))
147     print("FUSE_OK")
148     return 0
149
150
151 if __name__ == "__main__":
152     sys.exit(main())

```

2.9 问题三：证据定位（词元对齐、语音时段映射、关键帧抽取）

code/03_xai/evidence.py（共 327 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S5：证据定位（文本片段 / 语音时段 / 视觉关键帧）——题目要求“可回看原始素材”。
3
4  链路：
5  1. 文本：‘text_bert’ 的词元编号 + ‘vocab.txt’（行号即 id）解码 WordPiece -> 合并 ‘##’ 续接
6     -> 得到“词”序列及其**词元位置集合**（词元位置 = 对齐段号，见 section1.2 的对齐粒度实测）
7     -> 用 ‘difflib’ 把解码词对齐回原始 ‘raw_text’ 的字符区间（失败显式标记，不静默降级）；
8  2. 语音：高重要度的段号合并为连续区间，按比例映射为时间（附件2 不提供逐词时间戳，
9     声明为近似）并用视频音频能量曲线佐证；
10 3. 视觉：高重要度段取段内“视觉活跃度最大”的帧作为关键帧，用 ffmpeg 抽帧存盘。
11
12 输出：
13 submission/evidence/{id}_kf.jpg      关键帧
14 runs/q3/q3_evidence_{name}.json     逐样本证据记录（含字符区间、秒区间、帧时刻）
15 runs/q3/q3_audio_energy_{name}.npz  各样本音频能量曲线（论文解释卡用）
16
17 用法：
18 python evidence.py --name att4 --out_dir ../runs/q3
19 """
20 import argparse
21 import difflib
22 import json
23 import os
24 import re
25 import subprocess
26 import sys
27 import wave
28
29 import numpy as np
30
31 HERE = os.path.dirname(os.path.abspath(__file__))

```



```

88     merged.append((w, poss))
89     # 后处理: 相邻的"词+纯标点"再合并一次(处理句首标点情形)
90     out = []
91     i = 0
92     while i < len(merged):
93         w, poss = merged[i]
94         if (i + 1 < len(merged) and merged[i + 1][0] and
95             all(not ch.isalnum() for ch in merged[i + 1][0]) and len(w) > 1):
96             out.append((w + merged[i + 1][0], poss + merged[i + 1][1]))
97             i += 2
98             continue
99         out.append((w, poss))
100        i += 1
101    return out
102
103
104 def align_to_raw(words, raw_text):
105     """把解码词对齐回 raw_text 的字符区间。
106
107     返回 (spans, info): spans 为逐词的 (start, end) (未对齐项为 None);
108     info 记录对齐率, 供论文如实报告"对齐失败样本数"。
109     """
110     spans = [None] * len(words)
111     raw_words = [(m.group(0), m.start(), m.end()) for m in WORD_RE.finditer(raw_text)]
112     a = [w.lower() for w, _ in words]
113     b = [w.lower() for w, _, _ in raw_words]
114     sm = difflib.SequenceMatcher(a=a, b=b, autojunk=False)
115     hit = 0
116     for blk in sm.get_matching_blocks():
117         for t in range(blk.size):
118             spans[blk.a + t] = (raw_words[blk.b + t][1], raw_words[blk.b + t][2])
119             hit += 1
120     info = dict(n_word=len(words), n_raw=len(raw_words), matched=hit,
121                ratio=round(float(hit / max(len(words), 1)), 3))
122     return spans, info
123
124
125 def merge_runs(pos, gap=1):
126     """把位置列表合并为连续区间 [start, end)。"""
127     pos = sorted(set(int(p) for p in pos))
128     if not pos:
129         return []
130     runs, s, e = [], pos[0], pos[0] + 1
131     for p in pos[1:]:
132         if p <= e + gap - 1:
133             e = p + 1
134         else:
135             runs.append([s, e])
136             s, e = p, p + 1
137     runs.append([s, e])
138     return runs
139
140
141 def seg_to_seconds(seg_start, seg_end, n_valid, duration):
142     """段号区间 -> 秒区间 (比例映射; duration 为该样本视频时长)。"""
143     if not duration or n_valid <= 0:

```

```

144         return None, None
145     per = float(duration) / float(n_valid)
146     return round(seg_start * per, 2), round(min(seg_end, n_valid) * per, 2)
147
148
149 # -----
150 def ffmpeg_exe():
151     import imageio_ffmpeg
152     return imageio_ffmpeg.get_ffmpeg_exe()
153
154
155 def grab_frame(video, t_sec, out_jpg, width=320, q=5):
156     os.makedirs(os.path.dirname(out_jpg), exist_ok=True)
157     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-ss", "%.3f" % max(0.0, t_sec),
158           "-i", video, "-frames:v", "1", "-vf", "scale=%d:-1" % width,
159           "-q:v", str(q), out_jpg]
160     r = subprocess.run(cmd, capture_output=True)
161     return os.path.isfile(out_jpg) and os.path.getsize(out_jpg) > 0, (r.stderr or b"")[-300:]
162
163
164 def audio_energy(video, tmp_wav, hop=0.05):
165     """抽出 16k 单声道 wav 并算短时能量曲线（返回 times,rms）。"""
166     os.makedirs(os.path.dirname(tmp_wav), exist_ok=True)
167     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-i", video, "-vn",
168           "-ac", "1", "-ar", "16000", tmp_wav]
169     r = subprocess.run(cmd, capture_output=True)
170     if not os.path.isfile(tmp_wav):
171         return None, None, (r.stderr or b"")[-300:]
172     with wave.open(tmp_wav, "rb") as w:
173         sr = w.getframerate()
174         n = w.getnframes()
175         data = np.frombuffer(w.readframes(n), dtype=np.int16).astype(np.float32) / 32768.0
176         step = max(1, int(hop * sr))
177         nseg = max(1, len(data) // step)
178         rms = np.sqrt(np.mean(data[:nseg * step].reshape(nseg, step) ** 2, axis=1))
179         times = (np.arange(nseg) + 0.5) * step / sr
180         return times, rms, None
181
182
183 # -----
184 def main():
185     ap = argparse.ArgumentParser()
186     ap.add_argument("--name", default="att4")
187     ap.add_argument("--out_dir", default=os.path.join(W_ROOT, "runs", "q3"))
188     ap.add_argument("--sub_dir", default=os.path.join(W_ROOT, "submission"))
189     ap.add_argument("--meta", default=os.path.join(W_ROOT, "data_att", "att_meta.json"))
190     ap.add_argument("--pkl", default=os.path.join(W_ROOT, "data_att", "att4_aligned.pkl"),
191                     help="缓存 pkl（含 text_bert 与 raw_text，用于文本证据解码）")
192     ap.add_argument("--top_seg", type=int, default=3)
193     ap.add_argument("--top_word", type=int, default=3)
194     ap.add_argument("--phrase_win", type=int, default=2)
195     ap.add_argument("--no_video", action="store_true", help="不抽帧/不提音频（只算文本与时段）")
196     a = ap.parse_args()
197
198     fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_%s.npz" % a.name),
199                     allow_pickle=True)

```

```

200 W = fused["w_fused"]
201 valid = fused["valid"]
202 ids = [str(x) for x in fused["ids"]]
203 meta = json.load(open(a.meta, encoding="utf-8"))
204 recs = {r["id"]: r for r in meta[a.name]}
205 vocab = load_vocab()
206
207 # 文本证据需要 text_bert (词元编号) 与 raw_text: 从重建后的缓存 pkl 读取
208 tb_all, rt_all, idx_map = None, None, {}
209 if a.pkl and os.path.isfile(a.pkl):
210     d = load_pickle(a.pkl)
211     if "id" in d:
212         pkl_ids = [str(x) for x in np.asarray(d["id"]).reshape(-1)]
213         idx_map = {sid: i for i, sid in enumerate(pkl_ids)}
214     if "text_bert" in d:
215         tb_all = np.asarray(d["text_bert"])
216     if "raw_text" in d:
217         rt_all = [str(x) for x in np.asarray(d["raw_text"]).reshape(-1)]
218     print(" [PKL] %s: text_bert=%s raw_text=%s"
219           % (os.path.basename(a.pkl),
220              "有" if tb_all is not None else "无",
221              "有" if rt_all is not None else "无"))
222
223 ev_dir = os.path.join(a.sub_dir, "evidence")
224 out, energies = {}, {}
225 align_bad = []
226 for i, sid in enumerate(ids):
227     r = recs.get(sid, {})
228     k = idx_map.get(sid, None)
229     raw_text = (rt_all[k] if (rt_all is not None and k is not None and k < len(rt_all))
230                else (r.get("raw_text", "") or ""))
231     dur = r.get("duration_s")
232     tb = tb_all[k] if (tb_all is not None and k is not None and k < len(tb_all)) else None
233     n_text = int(valid[i, 0].sum())
234     n_audio = int(valid[i, 1].sum())
235     n_vision = int(valid[i, 2].sum())
236     rec = dict(id=sid)
237
238     # —— 文本证据 ——
239     w_text = np.asarray(W[i, 0], np.float64)
240     words = []
241     if tb is not None:
242         tb = np.asarray(tb)
243         words = decode_words(tb[0], tb[1], vocab)
244     if words:
245         spans, al = align_to_raw(words, raw_text)
246         if al["ratio"] < 0.8:
247             align_bad.append(dict(id=sid, **al))
248         ww = []
249         for (word, poss), sp in zip(words, spans):
250             ww.append((word, poss, sp, float(max(w_text[p] for p in poss if p < len(w_text)))))
251         ww_sorted = sorted(ww, key=lambda t: -t[3])
252         picks = [t for t in ww_sorted[:max(1, a.top_word)]]
253         pick_idx = [ww.index(t) for t in picks]
254         lo = max(0, min(pick_idx) - a.phrase_win)
255         hi = min(len(ww) - 1, max(pick_idx) + a.phrase_win)

```

```

256     phrase_words = [t[0] for t in ww[lo:hi + 1]]
257     phrase = "".join(phrase_words)
258     cspans = [t[2] for t in ww[lo:hi + 1] if t[2]]
259     segs = merge_runs([p for t in picks for p in t[1]])
260     rec["text_evidence"] = dict(
261         top_words=[dict(word=t[0], seg=t[1], char_span=t[2], w=round(t[3], 4))
262                     for t in picks],
263         phrase=phrase,
264         phrase_char_span=(min(s[0] for s in cspans), max(s[1] for s in cspans))
265                           if cspans else None),
266         seg_runs=segs, n_valid=n_text, align=al)
267     else:
268         rec["text_evidence"] = dict(note="无 text_bert (无法解码), 仅给段号",
269                                     seg_runs=[[int(x) for x in rn] for rn in
270                                                 merge_runs(np.argsort(-w_text[:n_text]):a.top_seg)]],
271                                     n_valid=n_text)
272
273     # —— 语音证据 ——
274     w_audio = np.asarray(W[i, 1], np.float64)
275     top_a = [int(x) for x in np.argsort(-w_audio[:n_audio]):a.top_seg]
276     runs_a = merge_runs(top_a)
277     s0, s1 = (seg_to_seconds(runs_a[0][0], runs_a[-1][1], n_audio, dur)
278               if runs_a else (None, None))
279     rec["audio_evidence"] = dict(top_segs=top_a, seg_runs=runs_a,
280                                 start_s=s0, end_s=s1, n_valid=n_audio, duration_s=dur)
281
282     # —— 视觉证据 ——
283     w_vision = np.asarray(W[i, 2], np.float64)
284     top_v = [int(x) for x in np.argsort(-w_vision[:n_vision]):a.top_seg]
285     runs_v = merge_runs(top_v)
286     t0, t1 = (seg_to_seconds(runs_v[0][0], runs_v[0][1], n_vision, dur)
287               if runs_v else (None, None))
288     kf_t = None if t0 is None else round((t0 + (t1 if t1 else t0)) / 2.0, 2)
289     rec["vision_evidence"] = dict(top_segs=top_v, seg_runs=runs_v,
290                                 keyframe_time_s=kf_t, n_valid=n_vision)
291     if (not a.no_video) and r.get("video") and kf_t is not None:
292         out_jpg = os.path.join(ev_dir, "%s_kf.jpg" % sid)
293         ok, err = grab_frame(r["video"], kf_t, out_jpg)
294         rec["vision_evidence"]["keyframe_file"] = os.path.relpath(out_jpg, W_ROOT)
295         rec["vision_evidence"]["keyframe_ok"] = bool(ok)
296         if not ok:
297             rec["vision_evidence"]["keyframe_err"] = err.decode("utf-8", "replace")
298         if sid not in energies:
299             tw = os.path.join(a.out_dir, "_audio", "%s.wav" % sid)
300             tt, rr, err2 = audio_energy(r["video"], tw)
301             if tt is not None:
302                 energies[sid] = dict(t=tt, rms=rr)
303                 np.savez_compressed(os.path.join(a.out_dir, "_audio", "%s_energy.npz" % sid),
304                                    t=tt, rms=rr)
305             else:
306                 rec["vision_evidence"]["audio_err"] = str(err2)[:200]
307             try:
308                 os.remove(tw)
309             except OSError:
310                 pass
311     out[sid] = rec

```

```

312         if (i + 1) % 5 == 0 or i + 1 == len(ids):
313             print(" %d/%d" % (i + 1, len(ids)))
314
315         save_json(os.path.join(a.out_dir, "q3_evidence_%s.json" % a.name), out)
316         if align_bad:
317             print(" [警告] 文本对齐率 <0.8 的样本 %d 条: %s" % (len(align_bad), align_bad[:3]))
318             print(" [KF] 关键帧 %d 张; 音频能量曲线 %d 条"
319                   % (len([1 for v in out.values() if v.get("vision_evidence", {}).get("keyframe_ok")]),
320                     len(energies)))
321             print("EVIDENCE_OK")
322             return 0
323
324
325 if __name__ == "__main__":
326     sys.exit(main())

```

2.10 问题三：并联证据头训练（骨干冻结、忠实性正则）

code/03_xai/train_evidence_head.py（共 206 行）

```

1  #-*- coding: utf-8 -*-
2  r"""D4/D9：可解释性证据头（冻结骨干 + 并联解释头 + 解释性正则训练）。
3
4  设计（论文 section5.3“可解释性模型的网络结构、目标函数、训练方案与关键参数”）：
5  骨干**完全冻结**并保持预测通路不变；证据头挂在其**编码器输出**（逐模态 50x d 表示）上，
6  只用于产生解释： $s^m_k = \text{softmax}_k(w^m_k \tanh(W^m_k h^m_k + b^m_k))$ ，
7  模态作用度  $\pi^m = \text{softmax}_m(u^m \tanh(V^m h^m))$ ， $h^m = \text{Sigma}_k s^m_k h^m_k$ 。
8
9  目标函数（全部可微；用“软删除/软保留”把忠实性写成可训练的损失）：
10  $L = \alpha_f [-(t(X) - t(X; \text{miss}=s))^2 + (t(X) - t(X; \text{miss}=1-s))^2]$ 
11  $+ \alpha_1 \text{Sigma}_m \|s^m\|_1 + \alpha_2 \text{Sigma}_m \text{TV}(s^m) + \alpha_3 \text{Sigma}_m (\pi^m - \pi^m_{\text{Shapley}})^2$ 
12
13 第一项：按 s 软删除后的预测变化越大越好（**忠实性**）；
14 第二项：只保留 s 的软权重时的预测应接近全模态（**充分性**）；
15 第三项/第四项：稀疏性与时间平滑；第五项（可选）：与精确 Shapley 的模态作用度对齐。
16
17 训练方案：只训证据头（骨干冻结、eval 模式），AdamW，lr 1e-3，30 轮，按验证集
18 “忠实性 - 充分性偏差”加权目标早停；关键参数 alpha 在验证集上网格择优（默认给定值）。
19
20 用法：
21 python train_evidence_head.py --epochs 12 --out ../runs/q3
22 """
23 import argparse
24 import json
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30 import torch.nn as nn
31 import torch.nn.functional as F
32
33 HERE = os.path.dirname(os.path.abspath(__file__))
34 if HERE not in sys.path:

```

```

35     sys.path.append(HERE)
36 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
37                          predict_batch, save_json, q3_meta, ROOT)
38
39
40 class EvidenceHead(nn.Module):
41     """逐模态时间证据头 + 模态门控（并联，不接入预测通路）。"""
42
43     def __init__(self, hidden, n_mod=3):
44         super().__init__()
45         self.hidden = hidden
46         self.W = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in MODALITIES})
47         self.w = nn.ParameterDict({m: nn.Parameter(torch.zeros(hidden)) for m in MODALITIES})
48         self.V = nn.Linear(hidden, hidden)
49         self.u = nn.Linear(hidden, 1)
50
51     def forward(self, h, pad):
52         """h: 模态->(B,L,H); pad: 模态->(B,L); 返回 s (B,3,L)、pi (B,3)。"""
53         S, pi_logit = [], []
54         for m in MODALITIES:
55             z = torch.tanh(self.W[m](h[m]))          # (B,L,H)
56             sc = torch.einsum("blh,h->bl", z, self.w[m]) # (B,L)
57             sc = sc.masked_fill(pad[m], -1e9)
58             s = torch.softmax(sc, dim=1)
59             S.append(s)
60             hbar = (s.unsqueeze(-1) * h[m]).sum(dim=1) # (B,H)
61             pi_logit.append(self.u(torch.tanh(self.V(hbar))).squeeze(-1))
62         S = torch.stack(S, dim=1) # (B,3,L)
63         pi = torch.softmax(torch.stack(pi_logit, dim=1), dim=1)
64         return S, pi
65
66
67 def capture(mods, models):
68     """抓取骨干各模态编码器输出 h[m] 的 hook（兼容 [(model, ckpt), ...] 与 [model, ...]）。"""
69     store, handles = {}, []
70     m0 = models[0][0] if isinstance(models[0], tuple) else models[0]
71     for m in mods:
72         def mk(name):
73             def hook(module, inp, out):
74                 store[name] = out
75             return hook
76         handles.append(m0.enc[m].register_forward_hook(mk(m)))
77     return store, handles
78
79
80 def forward_with_miss(models, b, miss_override):
81     """用给定的 miss（可为软值）跑集成前向，返回 (score, prob)。x 同步按软掩码缩权。"""
82     bb = dict(x={}, miss={}, pad={})
83     for m in MODALITIES:
84         bb["pad"][m] = b["pad"][m]
85         bb["miss"][m] = miss_override[m]
86         bad = (miss_override[m] + b["pad"][m].float()).clamp(0, 1).unsqueeze(-1)
87         bb["x"][m] = b["x"][m] * (1.0 - bad)
88     out = predict_batch(models, bb, target="mu")
89     return out["score"], out["prob"]
90

```

```

91
92 def tv(s):
93     return (s[:, :, 1:] - s[:, :, :-1]).abs().mean()
94
95
96 def main():
97     ap = argparse.ArgumentParser()
98     ap.add_argument("--epochs", type=int, default=12)
99     ap.add_argument("--batch_size", type=int, default=64)
100    ap.add_argument("--lr", type=float, default=1e-3)
101    ap.add_argument("--alpha_f", type=float, default=1.0)
102    ap.add_argument("--alpha1", type=float, default=1e-3)
103    ap.add_argument("--alpha2", type=float, default=1e-3)
104    ap.add_argument("--alpha3", type=float, default=0.0,
105                    help="与 Shapley 对齐项权重 (>0 需先算训练集 Shapley)")
106    ap.add_argument("--limit", type=int, default=0)
107    ap.add_argument("--out", default=os.path.join(ROOT, "runs", "q3"))
108    ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
109    ap.add_argument("--device", default="cpu")
110    a = ap.parse_args()
111
112    models, stats, theta = load_backbone(a.ckpt_dir, a.device, verbose=False)
113    for m, _ in models:
114        m.eval()
115        for p in m.parameters():
116            p.requires_grad_(False)
117    tr, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "train")
118    va, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "valid")
119    if a.limit:
120        from data_utils import slice_split
121        tr = slice_split(tr, a.limit)
122    hidden = models[0][1]["cfg"]["hidden"]
123    print("[DATA] train N=%d valid N=%d hidden=%d" % (tr["N"], va["N"], hidden))
124
125    head = EvidenceHead(hidden).to(a.device)
126    opt = torch.optim.AdamW(head.parameters(), lr=a.lr, weight_decay=1e-4)
127    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=a.epochs)
128    hist = []
129    best = dict(score=-1e9, epoch=-1, state=None)
130
131    for ep in range(1, a.epochs + 1):
132        head.train()
133        tot, nb = 0.0, 0
134        perm = np.random.permutation(tr["N"])
135        for i0 in range(0, tr["N"], a.batch_size):
136            idxs = [int(i) for i in perm[i0:i0 + a.batch_size]]
137            b = make_batch(tr, idxs, a.device)
138            store, handles = capture(MODALITIES, models)
139            with torch.no_grad():
140                t_full, _ = forward_with_miss(models, b,
141                                              {m: b["miss"][m].float() for m in MODALITIES})
142            for hd in handles:
143                hd.remove()
144            S, pi = head(store, b["pad"])
145            miss_soft = {m: (S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}
146            keep_soft = {m: (1.0 - S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}

```

```

147     t_del, _ = forward_with_miss(models, b, miss_soft)
148     t_keep, _ = forward_with_miss(models, b, keep_soft)
149     l_f = -((t_full - t_del) ** 2).mean() + ((t_full - t_keep) ** 2).mean()
150     l1 = sum(S[:, k].mean() for k in range(3)) / 3.0
151     l_tv = sum(tv(S[:, k:k + 1]) for k in range(3)) / 3.0
152     loss = a.alpha_f * l_f + a.alpha1 * l1 + a.alpha2 * l_tv
153     opt.zero_grad()
154     loss.backward()
155     nn.utils.clip_grad_norm_(head.parameters(), 1.0)
156     opt.step()
157     tot += float(loss.detach())
158     nb += 1
159     sched.step()
160     # —— 验证集评估: 忠实性 - 充分性偏差 (越大越好) ——
161     head.eval()
162     compr, suff = [], []
163     with torch.no_grad():
164         for i0 in range(0, min(va["N"], 256), 64):
165             idxs = list(range(i0, min(i0 + 64, va["N"])))
166             b = make_batch(va, idxs, a.device)
167             store, handles = capture(MODALITIES, models)
168             t_full, _ = forward_with_miss(models, b,
169                                           {m: b["miss"][m].float() for m in MODALITIES})
170             for hd in handles:
171                 hd.remove()
172             S, pi = head(store, b["pad"])
173             t_del, _ = forward_with_miss(models, b, {m: S[:, k] for k, m in enumerate(MODALITIES)})
174             t_keep, _ = forward_with_miss(models, b,
175                                           {m: (1 - S[:, k]) for k, m in enumerate(MODALITIES)})
176             compr.append((t_full - t_del).abs().mean().item())
177             suff.append((t_full - t_keep).abs().mean().item())
178     c, s = float(np.mean(compr)), float(np.mean(suff))
179     sc = c - s
180     hist.append(dict(epoch=ep, loss=round(tot / max(nb, 1), 5), valid_compr=round(c, 5),
181                    valid_suff=round(s, 5), score=round(sc, 5)))
182     print(" [ep%02d] loss=%5f valid 忠实性=%5f 充分性偏差=%5f 目标=%5f"
183           % (ep, tot / max(nb, 1), c, s, sc))
184     if sc > best["score"]:
185         best = dict(score=sc, epoch=ep,
186                    state={k: v.detach().clone() for k, v in head.state_dict().items()})
187     head.load_state_dict(best["state"])
188     torch.save(dict(state=head.state_dict(), cfg=dict(hidden=hidden, alpha_f=a.alpha_f,
189              alpha1=a.alpha1, alpha2=a.alpha2,
190              alpha3=a.alpha3, epochs=a.epochs,
191              lr=a.lr),
192              meta=q3_meta(a.ckpt_dir, models, theta, dict(train_n=tr["N"])),
193              history=hist, best_epoch=best["epoch"]),
194              os.path.join(a.out, "q3_evidence_head.pt"))
195     save_json(os.path.join(a.out, "q3_evidence_head_report.json"),
196              dict(best_epoch=best["epoch"], best_score=best["score"], history=hist,
197              args=dict(alpha_f=a.alpha_f, alpha1=a.alpha1, alpha2=a.alpha2,
198              alpha3=a.alpha3, epochs=a.epochs, lr=a.lr, batch=a.batch_size)))
199     print("[SAVE] runs/q3/q3_evidence_head.pt (最佳轮 %d, 目标 %5f) " % (best["epoch"], best["score"]))
200     print("EVIDENCE_HEAD_OK")
201     return 0
202

```

```

203
204 if __name__ == "__main__":
205     sys.exit(main())

```

2.11 问题三：解释卡生成

code/03_xai/explain_card.py（共 202 行）

```

1  -*- coding: utf-8 -*-
2  r"""S6-a: 典型样本解释卡（JSON + 三联图）。
3
4  三联图内容（论文 section5.3 典型样本解释卡）：
5  (1) 三模态时间重要性条带（含有效位边界与 top 片段标注）；
6  (2) 文本词着色条（按词级重要性深浅着色，凸显关键证据词）；
7  (3) 关键帧缩略图 + 音频能量曲线（高亮证据时段）。
8
9  用法：
10 python explain_card.py --name att4 --out_dir ../../runs/q3 --fig_dir ../../figs
11 """
12 import argparse
13 import json
14 import os
15 import sys
16
17 import numpy as np
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 if HERE not in sys.path:
22     sys.path.append(HERE)
23 from style import apply_style, COLORS, MOD_COLORS, LABEL_COLORS # noqa: E402
24 from xai_common import MODALITIES, save_json, ROOT # noqa: E402
25
26 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
27
28
29 def load_all(out_dir, name):
30     fused = np.load(os.path.join(out_dir, "q3_temporal_fused_%s.npz" % name), allow_pickle=True)
31     shap = np.load(os.path.join(out_dir, "q3_shapley_%s.npz" % name), allow_pickle=True)
32     ev = json.load(open(os.path.join(out_dir, "q3_evidence_%s.json" % name), encoding="utf-8"))
33     return fused, shap, ev
34
35
36 def card_figure(sid, W, valid, rec, pi, score, prob, theta, energy, out_png,
37                label_cn="", meta=None):
38     plt = apply_style()
39     fig = plt.figure(figsize=(9.2, 6.4))
40     gs = fig.add_gridspec(3, 2, height_ratios=[1.0, 0.75, 1.15], hspace=0.55, wspace=0.22)
41
42     # —— (1) 三模态时间重要性条带 ——
43     ax1 = fig.add_subplot(gs[0, :])
44     for k, m in enumerate(MODALITIES):
45         w = np.asarray(W[k], np.float64)
46         n = int(valid[k].sum())

```

```

47     ax1.bar(np.arange(len(w)) + (k - 1) * 0.0, w, width=0.62, bottom=k, color=MOD_COLORS[k],
48             label="%s (pi=%2f)" % (MOD_CN[m], pi[k]))
49     ax1.axvline(n - 0.5, color="#666666", lw=0.8, ls=":")
50     top = np.argsort(-w[:n])[:3]
51     for p in top:
52         ax1.text(p, k + w[p] + 0.06, "*", ha="center", va="bottom",
53                 fontsize=6.5, color="#333333")
54     ax1.set_yticks([0.5, 1.5, 2.5])
55     ax1.set_yticklabels([MOD_CN[m] for m in MODALITIES])
56     ax1.set_xlabel("对齐段号 $k$ (文本为词元位置; 虚线为该模态有效长度上界)")
57     ax1.set_title("(a) 三模态时间重要性 $w^m_k$ (*=top-3 关键片段)", loc="left")
58     ax1.legend(loc="upper right", ncol=3)
59
60     # —— (2) 文本词着色 ——
61     ax2 = fig.add_subplot(gs[1, :])
62     ax2.axis("off")
63     te = rec.get("text_evidence", {})
64     words = te.get("top_words", [])
65     phrase = te.get("phrase", "")
66     span = te.get("phrase_char_span")
67     raw = (meta or {}).get("raw_text", "")
68     ax2.set_title("(b) 文本证据: 高亮为模型关注词, 方框为可回看的字符区间", loc="left")
69     txt = raw if raw else phrase
70     if txt:
71         hi = {w["word"].lower() for w in words if w.get("word")}
72         # 用等宽示意图: 把原句逐词着色 (步长按字符数估算, 换行留出边距)
73         import re
74         x, y = 0.0, 0.74
75         for tok in re.finditer(r"\S+", txt):
76             word = tok.group(0)
77             key = re.sub(r"^[A-Za-z0-9]", "", word).lower()
78             hot = key in hi
79             ax2.text(x, y, word, fontsize=9.5, va="center",
80                     color=(COLORS["accent"] if hot else "#333333"),
81                     weight=("bold" if hot else "normal"),
82                     bbox=(dict(facecolor="#FBEAD6", edgecolor="none", pad=1.2) if hot else None),
83                     transform=ax2.transAxes)
84             x += 0.0092 * (len(word) + 1.35)
85             if x > 0.995:
86                 x, y = 0.0, y - 0.26
87     else:
88         ax2.text(0.01, 0.5, "(该样本无 raw_text, 无法回看原文)", transform=ax2.transAxes,
89                 fontsize=9)
90     if span:
91         ax2.text(0.0, 0.06, "字符区间 [%d,%d]: %s" % (span[0], span[1], phrase),
92                 transform=ax2.transAxes, fontsize=8.5, color=COLORS["accent"])
93
94     # —— (3) 关键帧 + 音频能量 ——
95     ax3 = fig.add_subplot(gs[2, 0])
96     kf = (rec.get("vision_evidence") or {}).get("keyframe_file")
97     if kf and os.path.isfile(os.path.join(ROOT, kf)):
98         img = plt.imread(os.path.join(ROOT, kf))
99         ax3.imshow(img)
100        ax3.set_title("(c) 视觉关键帧 t=%2fs" % (rec["vision_evidence"].get("keyframe_time_s") or 0.0),
101                    loc="left")
102     else:

```

```

103     ax3.text(0.5, 0.5, "无关键帧", ha="center", transform=ax3.transAxes)
104     ax3.set_title("(c) 视觉关键帧", loc="left")
105     ax3.set_xticks([])
106     ax3.set_yticks([])
107     ax3.grid(False)
108
109     ax4 = fig.add_subplot(gs[2, 1])
110     if energy is not None:
111         t, rms = energy["t"], energy["rms"]
112         ax4.plot(t, rms, color=COLORS["audio"], lw=1.2)
113         ae = rec.get("audio_evidence") or {}
114         if ae.get("start_s") is not None:
115             ax4.axvspan(ae["start_s"], ae["end_s"], color=COLORS["audio"], alpha=0.18)
116         ax4.set_xlabel("时间 / s")
117         ax4.set_ylabel("音频能量")
118         ax4.set_title("(d) 音频能量与证据时段（阴影）", loc="left")
119     else:
120         ax4.text(0.5, 0.5, "无音频能量", ha="center", transform=ax4.transAxes)
121         ax4.set_title("(d) 音频能量", loc="left")
122
123     sup = "预测: %s (强度 %+.3f, theta=%+.3f, 主模态 %s)" % (
124         label_cn, score, theta, MOD_CN[MODALITIES[int(np.argmax(pi))]])
125     fig.suptitle("解释卡. 样本 %s %s" % (sid, sup), fontsize=11.5, y=0.985)
126     os.makedirs(os.path.dirname(out_png), exist_ok=True)
127     fig.savefig(out_png, dpi=300)
128     plt.close(fig)
129     return out_png
130
131
132 def main():
133     ap = argparse.ArgumentParser()
134     ap.add_argument("--name", default="att4")
135     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
136     ap.add_argument("--fig_dir", default=os.path.join(ROOT, "figs"))
137     ap.add_argument("--theta", type=float, default=0.325)
138     ap.add_argument("--ids", default=None, help="逗号分隔; 默认自动挑（正/中/负各一）")
139     a = ap.parse_args()
140
141     fused, shap, ev = load_all(a.out_dir, a.name)
142     W, valid = fused["w_fused"], fused["valid"]
143     ids = [str(x) for x in fused["ids"]]
144     pi_all = shap["pi_soft"]
145     score = shap["score"]
146     prob = shap["prob"]
147     LAB = ("Negative", "Neutral", "Positive")
148     CN = {"Negative": "负向", "Neutral": "中性", "Positive": "正向"}
149     pred = [LAB[i] for i in prob.argmax(1)]
150     meta = {r["id"]: r for r in
151             json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
152                           encoding="utf-8"))[a.name]}
153
154     if a.ids:
155         pick = [s.strip() for s in a.ids.split(",")]
156     else: # 自动挑: 正/中/负各一条（优先解释指标最有说服力的: 置信度高 + pi 集中）
157         pick = []
158         for lab in ("Positive", "Neutral", "Negative"):

```

```

159         cand = [i for i, p in enumerate(pred) if p == lab]
160         if not cand:
161             continue
162         best = max(cand, key=lambda i: (float(prob[i].max()) * float(np.max(pi_all[i])
163                                     - np.min(pi_all[i]))))
164         pick.append(ids[best])
165     print("[PICK] %s" % pick)
166
167     made = []
168     cards = {}
169     for sid in pick:
170         i = ids.index(sid)
171         energy = None
172         ep = os.path.join(a.out_dir, "_audio", "%s_energy.npz" % sid)
173         if os.path.isfile(ep):
174             with np.load(ep) as z:
175                 energy = dict(t=z["t"], rms=z["rms"])
176         card = dict(id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
177                   prob={LAB[k]: round(float(prob[i, k]), 4) for k in range(3)},
178                   confidence=round(float(prob[i].max()), 4), theta=a.theta,
179                   main_modality=MODALITIES[int(np.argmax(pi_all[i]))],
180                   modality_contribution={MODALITIES[k]: round(float(pi_all[i, k]), 4)
181                                         for k in range(3)},
182                   shapley_raw={MODALITIES[k]: round(float(shap["phi_mu_miss"][i, k]), 4)
183                               for k in range(3)},
184                   temporal_importance={MODALITIES[k]: np.round(W[i, k], 4).tolist()
185                                       for k in range(3)},
186                   evidence=ev.get(sid, {}))
187         png = os.path.join(a.fig_dir, "Q3_解释卡_%s_%s_组合图.png" % (sid, pred[i]))
188         card_figure(sid, W[i], valid[i], ev.get(sid, {}), pi_all[i], float(score[i]),
189                   prob[i], a.theta, energy, png, CN[pred[i]], meta.get(sid, {}))
190         cards[sid] = card
191     made.append(os.path.relpath(png, ROOT))
192     print(" [CARD] %s-> %s" % (sid, os.path.relpath(png, ROOT)))
193
194     save_json(os.path.join(a.out_dir, "q3_cards_%s.json" % a.name), cards)
195     print("[SAVE] %d 张解释卡: %s" % (len(made), made))
196     print("CARD_OK")
197     return 0
198
199
200 if __name__ == "__main__":
201     sys.exit(main())

```

2.12 问题三：附件四全量推理与解释输出

code/03_xai/infer_explain_att4.py（共 231 行）

```

1  -*- coding: utf-8 -*-
2  r"""S6-b: 主入口 —— 生成附件4 的”预测 + 解释”提交文件与解释卡。
3
4  产物:
5  submission/pred_explain_att4.csv    题目要求的附件4 预测与解释结果（每行一条样本）
6  submission/explain_cards/{id}.json  逐样本完整解释卡（含 50 步时间重要性）

```

```

7 submission/evidence/{id}_kf.jpg      关键帧（由 evidence.py 生成）
8
9 内建自查断言（任一不过即报错退出）：
10 1. 行数与 id 顺序与附件4 严格一致；
11 2. pred_label / pred_score 与 submission/pred_att4.csv 逐位一致（同一骨干、同一 theta）；
12 3. 解释数值有限、pi 和为 1、时间重要性非负；
13 4. 证据区间合法（ $0 \leq \text{start} < \text{end} \leq \text{有效长度}$ ；秒  $\leq$  视频时长）。
14
15 用法：
16 python infer_explain_att4.py --out_dir ..\..\runs\q3 --sub_dir ..\..\submission
17 """
18 import argparse
19 import json
20 import os
21 import re
22 import subprocess
23 import sys
24
25 import numpy as np
26 import pandas as pd
27 import torch
28
29 HERE = os.path.dirname(os.path.abspath(__file__))
30 if HERE not in sys.path:
31     sys.path.append(HERE)
32 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
33                         predict_batch, save_json, q3_meta, ROOT)
34
35 LABELS = ("Negative", "Neutral", "Positive")
36 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
37
38
39 def probe_fps(video, cache_path):
40     """用 ffmpeg 的 stderr 解析帧率（缓存复用）。"""
41     cache = {}
42     if os.path.isfile(cache_path):
43         try:
44             cache = json.load(open(cache_path, encoding="utf-8"))
45         except Exception:
46             cache = {}
47     key = os.path.basename(video)
48     if key in cache:
49         return cache[key]
50     import imageio_ffmpeg
51     r = subprocess.run([imageio_ffmpeg.get_ffmpeg_exe(), "-i", video],
52                        capture_output=True)
53     txt = (r.stderr or b"").decode("utf-8", "replace")
54     m = re.search(r"(\d+(?:\.\d+)?)s.*fps", txt)
55     fps = float(m.group(1)) if m else None
56     cache[key] = fps
57     try:
58         os.makedirs(os.path.dirname(cache_path), exist_ok=True)
59         json.dump(cache, open(cache_path, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
60     except OSError:
61         pass
62     return fps

```

```

63
64
65 def main():
66     ap = argparse.ArgumentParser()
67     ap.add_argument("--pkl", default=os.path.join(ROOT, "data_att", "att4_aligned.pkl"))
68     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
69     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
70     ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
71     ap.add_argument("--theta", type=float, default=0.325)
72     ap.add_argument("--device", default="cpu")
73     a = ap.parse_args()
74
75     models, stats, theta_ck = load_backbone(a.ckpt_dir, a.device, verbose=False)
76     split, key = load_split(a.pkl, stats)
77     N = split["N"]
78     ids = [str(x) for x in split["ids"]]
79     print("[DATA] att4 N=%d (ckpt theta 均值 %.3f, 提交用 theta=%.3f)" % (N, theta_ck, a.theta))
80
81     # ----- 1) 复现预测 -----
82     b = make_batch(split, list(range(N)), a.device)
83     out = predict_batch(models, b, target="mu")
84     score = out["score"].detach().cpu().numpy()
85     prob = out["prob"].detach().cpu().numpy()
86     pred = [LABELS[i] for i in prob.argmax(1)]
87     ref_path = os.path.join(a.sub_dir, "pred_att4.csv")
88     assert os.path.isfile(ref_path), "缺少 %s (先跑 dl_recalib.py)" % ref_path
89     ref = pd.read_csv(ref_path, dtype={"id": str})
90     ref_ids = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in ref["id"]]
91     assert ref_ids == ids, "附件4 id 顺序不一致: %s" % ref_ids[:4]
92     lab_eq = float((np.array(pred) == ref["pred_label"].values).mean())
93     sc_diff = float(np.abs(score - ref["pred_score"].values).max())
94     print("[CHECK] 与 pred_att4.csv: 标签一致率 %.3f; 分数最大差 %.1e" % (lab_eq, sc_diff))
95     assert lab_eq == 1.0, "标签与 pred_att4.csv 不一致, 禁止输出"
96     assert sc_diff < 1e-3, "分数与 pred_att4.csv 不一致 (差 %.4f)" % sc_diff
97
98     # ----- 2) 读解释与证据 -----
99     fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_att4.npz"), allow_pickle=True)
100     shap = np.load(os.path.join(a.out_dir, "q3_shapley_att4.npz"), allow_pickle=True)
101     met = np.load(os.path.join(a.out_dir, "q3_temporal_occ_att4.npz"), allow_pickle=True)
102     ev = json.load(open(os.path.join(a.out_dir, "q3_evidence_att4.json"), encoding="utf-8"))
103     faith_p = os.path.join(a.out_dir, "q3_faithfulness_att4.csv")
104     faith = pd.read_csv(faith_p, dtype={"id": str}) if os.path.isfile(faith_p) else None
105     meta = {r["id"]: r for r in json.load(
106         open(os.path.join(ROOT, "data_att", "att_meta.json"), encoding="utf-8"))["att4"]}
107
108     W = fused["w_fused"]
109     W_att, W_ig, W_occ = fused["w_att"], fused["w_ig"], fused["w_occ"]
110     valid = fused["valid"]
111     pi = shap["pi_soft"]
112     pi_relu = shap["pi_relu"]
113     phi = shap["phi_mu_miss"]
114     occ = met["w_occ"]
115     fps_cache = os.path.join(a.out_dir, "video_fps.json")
116
117     rows, cards = [], {}
118     for i, sid in enumerate(ids):

```

```

119     n_text = int(valid[i, 0].sum())
120     n_audio = int(valid[i, 1].sum())
121     n_vision = int(valid[i, 2].sum())
122     e = ev.get(sid, {})
123     m = meta.get(sid, {})
124     # 断言: pi 归一化与时间重要性合法性
125     assert abs(float(pi[i].sum()) - 1.0) < 1e-6, "pi 未归一化: %s" % sid
126     assert np.isfinite(W[i]).all() and (W[i] >= -1e-9).all(), "时间重要性非法: %s" % sid
127
128     te = e.get("text_evidence", {}) or {}
129     tr = te.get("seg_runs") or [[None, None]]
130     ae = e.get("audio_evidence", {}) or {}
131     ar = ae.get("seg_runs") or [[None, None]]
132     ve = e.get("vision_evidence", {}) or {}
133     vr = ve.get("seg_runs") or [[None, None]]
134     kf_t = ve.get("keyframe_time_s")
135     fps = probe_fps(m["video"], fps_cache) if m.get("video") else None
136     frame_idx = int(round(kf_t * fps)) if (kf_t is not None and fps) else None
137
138     # 证据合法性断言
139     if tr and tr[0][0] is not None:
140         assert 0 <= tr[0][0] < max(1, n_text), "文本证据段号非法: %s" % sid
141     if ae.get("end_s") is not None and m.get("duration_s"):
142         assert ae["end_s"] <= m["duration_s"] + 0.05, "语音证据超视频时长: %s" % sid
143
144     # 忠实性/充分性/稳定性/稀疏性 (来自 metrics_xai.py, 取融合解释那一行)
145     compr = suff = stab = spar = None
146     if faith is not None:
147         sub = faith[(faith["id"] == sid) & (faith["source"] == "fused")]
148         if len(sub):
149             compr = float(sub["compr_mean"].iloc[0])
150             suff = float(sub["suff_absmean"].iloc[0])
151             spar = float(sub["sparsity_mean"].iloc[0])
152     rep = json.load(open(os.path.join(a.out_dir, "q3_faithfulness_att4_report.json"),
153                               encoding="utf-8")) if os.path.isfile(
154         os.path.join(a.out_dir, "q3_faithfulness_att4_report.json")) else {}
155     stab = (rep.get("stability", {}).get(sid, {}) or {}).get("stability_fused")
156
157     phrase = te.get("phrase", "")
158     row = dict(
159         id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
160         prob_neg=round(float(prob[i, 0]), 4), prob_neu=round(float(prob[i, 1]), 4),
161         prob_pos=round(float(prob[i, 2]), 4),
162         confidence=round(float(prob[i].max()), 4), theta=a.theta,
163         main_modality=MODALITIES[int(np.argmax(pi[i]))],
164         w_text=round(float(pi[i, 0]), 4), w_audio=round(float(pi[i, 1]), 4),
165         w_vision=round(float(pi[i, 2]), 4),
166         w_text_relu=round(float(pi_relu[i, 0]), 4),
167         w_audio_relu=round(float(pi_relu[i, 1]), 4),
168         w_vision_relu=round(float(pi_relu[i, 2]), 4),
169         phi_text=round(float(phi[i, 0]), 4), phi_audio=round(float(phi[i, 1]), 4),
170         phi_vision=round(float(phi[i, 2]), 4),
171         text_evidence=phrase or te.get("note", ""),
172         text_seg_start=tr[0][0], text_seg_end=tr[0][1],
173         text_char_start=(te.get("phrase_char_span") or [None, None])[0],
174         text_char_end=(te.get("phrase_char_span") or [None, None])[1],

```

```

175     audio_start_s=ae.get("start_s"), audio_end_s=ae.get("end_s"),
176     audio_seg_start=ar[0][0], audio_seg_end=ar[0][1],
177     vision_seg_start=vr[0][0], vision_seg_end=vr[0][1],
178     vision_frame_idx=frame_idx, vision_time_s=kf_t,
179     vision_keyframe=ve.get("keyframe_file", ""),
180     compr=None if compr is None else round(compr, 4),
181     suff=None if suff is None else round(suff, 4),
182     stability=stab, sparsity=None if spar is None else round(spar, 4),
183 )
184 rows.append(row)
185 cards[sid] = dict(
186     id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
187     prob={LABELS[k]: round(float(prob[i, k]), 4) for k in range(3)},
188     confidence=round(float(prob[i].max()), 4), theta=a.theta,
189     main_modality=row["main_modality"],
190     modality_contribution={MODALITIES[k]: round(float(pi[i, k]), 4) for k in range(3)},
191     modality_contribution_relu={MODALITIES[k]: round(float(pi_relu[i, k]), 4)
192                                for k in range(3)},
193     shapley_raw={MODALITIES[k]: round(float(phi[i, k]), 4) for k in range(3)},
194     temporal_importance_fused={MODALITIES[k]: np.round(W[i, k], 4).tolist()
195                               for k in range(3)},
196     temporal_importance_att={MODALITIES[k]: np.round(W_att[i, k], 4).tolist()
197                              for k in range(3)},
198     temporal_importance_ig={MODALITIES[k]: np.round(W_ig[i, k], 4).tolist()
199                             for k in range(3)},
200     temporal_importance_occ={MODALITIES[k]: np.round(W_occ[i, k], 4).tolist()
201                              for k in range(3)},
202     gamma=fused["gamma"].tolist(),
203     evidence=dict(text=te, audio=ae, vision=ve),
204     raw_text=m.get("raw_text", ""), video=os.path.basename(m.get("video", "") or ""),
205     duration_s=m.get("duration_s"),
206     faithfulness=dict(comprehensiveness=row["compr"], sufficiency=row["suff"],
207                      stability=stab, sparsity=row["sparsity"]))
208
209 df = pd.DataFrame(rows)
210 out_csv = os.path.join(a.sub_dir, "pred_explain_att4.csv")
211 df.to_csv(out_csv, index=False, encoding="utf-8-sig")
212 os.makedirs(os.path.join(a.sub_dir, "explain_cards"), exist_ok=True)
213 for sid, c in cards.items():
214     with open(os.path.join(a.sub_dir, "explain_cards", "%s.json" % sid), "w",
215              encoding="utf-8") as f:
216         json.dump(c, f, ensure_ascii=False, indent=1)
217 save_json(os.path.join(a.out_dir, "q3_explain_meta.json"),
218          dict(meta=q3_meta(a.ckpt_dir, models, a.theta, dict(pk1=os.path.relpath(a.pk1, ROOT))),
219              csv=os.path.relpath(out_csv, ROOT), n=N,
220              columns=list(df.columns)))
221 print("[SAVE] %s (%d 行 x %d 列)" % (os.path.relpath(out_csv, ROOT), len(df), df.shape[1]))
222 print("[SAVE] submission/explain_cards/*.json (%d 个)" % len(cards))
223 cnt = df["main_modality"].value_counts().to_dict()
224 print("[STAT] 主参考模态分布: %s" % cnt)
225 print("INFER_EXPLAIN_OK")
226 return 0
227
228
229 if __name__ == "__main__":
230     sys.exit(main())

```

2.13 问题三：交付自查（17 项一致性校验）

code/03_xai/verify_q3.py（共 151 行）

```
1  #-*- coding: utf-8 -*-
2  r"""S9: 问题3 交付自查（任一断言不过即失败）。
3
4  检查项:
5  1. pred_explain_att4.csv 行数 = 20、id 顺序与附件4 文件名一致、列齐全;
6  2. 其 pred_label / pred_score 与 submission/pred_att4.csv 逐位一致;
7  3. 每行模式作用度之和为 1; 时间重要性数值合法（在卡片 JSON 中抽检）;
8  4. 证据区间合法（段号、秒区间不超过视频时长）;
9  5. 关键帧文件存在且非空;
10 6. 解释卡 JSON 每个 id 齐全;
11 7. submission/ 总体积 <= 50 MB;
12 8. 匿名性: CSV/JSON 中不含单位/队号/学号/手机号/邮箱等身份信息。
13
14 用法:
15 python verify_q3.py --sub_dir ../submission --out_dir ../runs/q3
16 """
17 import argparse
18 import json
19 import os
20 import re
21 import sys
22
23 import numpy as np
24 import pandas as pd
25
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 if HERE not in sys.path:
28     sys.path.append(HERE)
29 from xai_common import save_json, ROOT          # noqa: E402
30
31 BANNED = [r"参赛队", r"队号", r"学号", r"指导教[师]?", r"大[学]?[\u4e00-\u9fff]{0,6}学院",
32           r"l[3-9]d{9}", r"[\w.-]+@[ \w.-]+\w+", r"[\u4e00-\u9fff]{2,4}(同学|老师)"]
33
34
35 def main():
36     ap = argparse.ArgumentParser()
37     ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
38     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
39     ap.add_argument("--max_mb", type=float, default=50.0)
40     a = ap.parse_args()
41     checks, fails = [], []
42
43     def ck(name, ok, detail=""):
44         checks.append(dict(check=name, ok=bool(ok), detail=str(detail)))
45         if not ok:
46             fails.append("%s: %s" % (name, detail))
47         print(" [%s] %-34s %s" % ("OK" if ok else "FAIL", name, detail))
48
49     # 1) 行数 / id / 列
50     path = os.path.join(a.sub_dir, "pred_explain_att4.csv")
51     ck("文件存在", os.path.isfile(path), os.path.relpath(path, ROOT))
52     df = pd.read_csv(path, dtype={"id": str})
```

```

53 df["id"] = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in df["id"]]
54 meta4 = json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
55                        encoding="utf-8"))
56 ids = [r["id"] for r in meta4["att4"]]
57 ck("行数=20", len(df) == 20, "实为%d" % len(df))
58 ck("id 顺序与附件4 一致", list(df["id"]) == ids, "%s" % list(df["id"])[4])
59 need = ["id", "pred_label", "pred_score", "prob_neg", "prob_neu", "prob_pos",
60         "main_modality", "w_text", "w_audio", "w_vision", "text_evidence",
61         "text_seg_start", "text_seg_end", "audio_start_s", "audio_end_s",
62         "vision_frame_idx", "vision_time_s", "compr", "suff", "stability", "sparsity"]
63 miss = [c for c in need if c not in df.columns]
64 ck("必需列齐全", not miss, "缺%s" % miss)
65
66 # 2) 与 pred_att4.csv 一致
67 ref = pd.read_csv(os.path.join(a.sub_dir, "pred_att4.csv"), dtype={"id": str})
68 ck("标签与 pred_att4.csv 一致",
69     float((df["pred_label"].values == ref["pred_label"].values).mean()) == 1.0)
70 ck("分数与 pred_att4.csv 一致",
71     float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()) < 1e-3,
72     "最大差 %2e" % float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()))
73
74 # 3) pi 归一化
75 s = df[["w_text", "w_audio", "w_vision"]].values.sum(1)
76 ck("模态作用度之和=1", float(np.abs(s - 1).max()) < 2e-3, "最大偏差 %2e"
77     % float(np.abs(s - 1).max()))
78 ck("主模态取值合法",
79     set(df["main_modality"]) <= {"text", "audio", "vision"},
80     "%s" % sorted(set(df["main_modality"])))
81
82 # 4) 证据合法性
83 bad_seg = df[(df["text_seg_start"].notna()) & (df["text_seg_end"].notna()) &
84              (~(df["text_seg_start"] >= 0) & (df["text_seg_end"] <= 50) &
85               (df["text_seg_start"] < df["text_seg_end"])))])
86 ck("文本段号合法", len(bad_seg) == 0, "越界%d行" % len(bad_seg))
87 dur = {r["id"]: r.get("duration_s") for r in meta4["att4"]}
88 bad_t = [i for i, r in df.iterrows()
89          if pd.notna(r["audio_end_s"]) and dur.get(r["id"]) and
90          r["audio_end_s"] > dur[r["id"]] + 0.05]
91 ck("语音时段不超视频时长", len(bad_t) == 0, "越界%d行" % len(bad_t))
92 ck("含文本证据的样本数>0", int(df["text_evidence"].notna().sum()) > 0,
93     "%d/20" % int(df["text_evidence"].notna().sum()))
94
95 # 5) 关键帧
96 kf = [os.path.join(ROOT, p) for p in df["vision_keyframe"].dropna().astype(str) if p]
97 ok_kf = [p for p in kf if os.path.isfile(p) and os.path.getsize(p) > 500]
98 ck("关键帧文件有效", len(ok_kf) >= max(1, int(0.8 * len(kf))),
99     "%d/%d 张可用" % (len(ok_kf), len(kf)))
100
101 # 6) 解释卡
102 cdir = os.path.join(a.sub_dir, "explain_cards")
103 have = sorted(os.path.splitext(f)[0] for f in os.listdir(cdir)) if os.path.isdir(cdir) else []
104 ck("解释卡 JSON 齐全", set(have) == set(ids), "%d/20" % len(have))
105 if set(have) >= set(ids):
106     c = json.load(open(os.path.join(cdir, ids[0] + ".json"), encoding="utf-8"))
107     ti = c["temporal_importance_fused"]["text"]
108     ck("时间重要性长度=50", len(ti) == 50, "len=%d" % len(ti))

```

```

109         ck("时间重要性非负且有界", bool(np.min(ti) >= -1e-6 and np.max(ti) <= 1 + 1e-6))
110
111     # 7) 体积
112     tot = 0
113     for r, _, fs in os.walk(a.sub_dir):
114         if "_backup" in r:
115             continue
116         for f in fs:
117             tot += os.path.getsize(os.path.join(r, f))
118     mb = tot / 1024 / 1024
119     ck("submission 体积 <= %.0fMB" % a.max_mb, mb <= a.max_mb, "%.2fMB" % mb)
120
121     # 8) 匿名性
122     hits = []
123     for r, _, fs in os.walk(a.sub_dir):
124         if "_backup" in r:
125             continue
126         for f in fs:
127             if not f.lower().endswith(("csv", "json", "md", "txt")):
128                 continue
129             p = os.path.join(r, f)
130             try:
131                 txt = open(p, encoding="utf-8", errors="replace").read()
132             except OSError:
133                 continue
134             for pat in BANNED:
135                 for m in re.finditer(pat, txt):
136                     hits.append("%s :: %s" % (os.path.relpath(p, ROOT), m.group(0)))
137     ck("无身份信息", not hits, "%d 命中" % len(hits))
138     if hits:
139         print("    %s" % hits[:5])
140
141     rep = dict(n_check=len(checks), n_fail=len(fails), checks=checks, fails=fails,
142               submission_mb=round(mb, 2))
143     save_json(os.path.join(a.out_dir, "verify_q3.json"), rep)
144     print("\n%s (%d 项检查, %d 项失败)" % ("VERIFY_PASS" if not fails else "VERIFY_FAIL",
145                                           len(checks), len(fails)))
146     return 0 if not fails else 1
147
148
149 if __name__ == "__main__":
150     sys.exit(main())

```